

DATA INTERPRETATION AND VISUALIZATION USING PYTHON

B.A. (PROGRAMME) COMPUTER APPLICATIONS
SEMESTER-II
MAJOR PAPER



DEPARTMENT OF DISTANCE AND CONTINUING EDUCATION
UNIVERSITY OF DELHI



Data Interpretation and Visualization using Python

Editors

*Ms. Asha Yadav
Dr. Charu Gupta*

Content Writers

Lavkush Gupta

Content Reviewer from the DDCE/COL/SOL

*Dr. Reema Thareja
Ms. Aishwarya Anand Arora*

Academic Coordinator

Mr. Deekshant Awasthi

© **Department of Distance and Continuing Education**

ISBN: 978-81-19417-53-7

1st edition: 2024

E-mail: ddceprinting@col.du.ac.in

Published by:

Department of Distance and Continuing Education
Campus of Open Learning/School of Open Learning,
University of Delhi, Delhi-110 007

Printed by:

School of Open Learning, University of Delhi



Corrections/Modifications/Suggestions proposed by Statutory Body, DU/Stakeholder/s in the Self Learning Material (SLM) will be incorporated in the next edition. However, these corrections/modifications/suggestions will be uploaded on the website <https://sol.du.ac.in>. Any feedback or suggestions can be sent to the email- feedbackslm@col.du.ac.in



SYLLABUS

Data Interpretation and Visualization using Python

Syllabus	Mapping
Unit-I: Introduction: Motivation for using Python for Data Visualization, Essential Python. Libraries: NumPy, Pandas, Matplotlib, Import and Export of Data, Import and Export of data using files.	Lesson 1: Introduction to Data Analysis Lesson 2: Statistical Foundations & Python Libraries (Pages 3–48)
Unit-II: Array manipulation using Numpy: Numpy array: Creating Numpy arrays, Data Types for Numpy arrays, Arithmetic with NumPy Arrays, Basic Indexing and Slicing, swapping axes, transposing arrays.	Lesson 3: NumPy: The Art of Array Manipulation (Pages 51–77)
Unit-III: Data Manipulation using Pandas: Data Structures in Pandas: Series, DataFrame, Index objects, Loading data into Pandas DataFrame. Working with DataFrames. Grouped and aggregate calculations.	Lesson 4: Pandas Power Play: Mastering Data Manipulation (Pages 81–126)
Unit-IV: Plotting and Visualization: Using matplotlib to plot data: figures, subplots, markings, colour and line styles, labels and legends, plotting functions in Pandas: Line, bar, Scatter plots, histograms, stacked bars, boxplot.	Lesson 5: Plotting Perfection: Mastering Plotting & Visualization (Pages 129–151)
Unit-V: Data Aggregation and Group operations: Group by Mechanics, Data aggregation, General split-apply combine, Pivot tables and cross tabulation.	Lesson 6: Data Unification: Exploring Aggregation and Grouping (Pages 155–171)



CONTENTS

UNIT I

LESSON 1	INTRODUCTION TO DATA ANALYSIS	3–17
	1.1 Learning Objectives	
	1.2 Introduction	
	1.3 What is Data?	
	1.4 Significance of Data	
	1.5 Why Data Analysis?	
	1.6 Types of Data	
	1.7 Sources of Data Collection	
	1.8 Data Preparation	
	1.9 Exploratory Data Analysis	
	1.10 Summary	
	1.11 Glossary	
	1.12 Answers to In-text Questions	
	1.13 Self-Assessment Questions	
	1.14 References	
	1.15 Suggested Readings	
LESSON 2	STATISTICAL FOUNDATIONS & PYTHON LIBRARIES	19–48
	2.1 Learning Objectives	
	2.2 Introduction	
	2.3 Importance of Statistics	
	2.4 Population	
	2.5 Sampling	
	2.6 Types of Statistics	
	2.7 Measures of Central Tendency	



Data Interpretation and Visualization using Python

- 2.8 Measures of Dispersion
- 2.9 Scaling Features of Data
- 2.10 Relationship between random variables – Covariance & Correlation
- 2.11 Regression Analysis
- 2.12 Statistical Hypothesis Generation and Testing
- 2.13 Essentials and Motivation for using Python for Data
- 2.14 Python Libraries
- 2.15 Summary
- 2.16 Glossary
- 2.17 Answers to In-text Questions
- 2.18 Self-Assessment Questions
- 2.19 References
- 2.20 Suggested Readings

UNIT II

LESSON 3	NUMPY: THE ART OF ARRAY MANIPULATION	51–77
-----------------	---	--------------

- 3.1 Learning Objectives
- 3.2 Introduction
- 3.3 NumPy Array
- 3.4 Creating Nd-array
- 3.5 Attributes of Nd-array
- 3.6 Data types of Nd-array
- 3.7 Mathematical operations in Nd-arrays
- 3.8 Random modules & their usage
- 3.9 Indexing & Slicing
- 3.10 Reshaping and Transposing Operations
- 3.11 Swapping Axes
- 3.12 Summary
- 3.13 Glossary



- 3.14 Answers to In-text Questions
- 3.15 Self-Assessment Questions
- 3.16 References
- 3.17 Suggested Readings

UNIT III

LESSON 4 **PANDAS POWER PLAY: MASTERING DATA MANIPULATION**

81–126

- 4.1 Learning Objectives
- 4.2 Introduction
- 4.3 Pandas Series
 - 4.3.1 Creating a Pandas Series
 - 4.3.2 Accessing Elements in a Pandas Series
 - 4.3.3 Operations on Pandas Series
- 4.4 DataFrame
- 4.5 Index Objects
- 4.6 Working with DataFrame
 - 4.6.1 Arithmetic Operations
 - 4.6.2 Statistical Functions
- 4.7 Binning
- 4.8 Indexing and Reindexing
 - 4.8.1 Indexing
 - 4.8.2 Reindexing
- 4.9 Filtering
- 4.10 Handling Missing Data
- 4.11 Hierarchical Indexing
- 4.12 Data Wrangling
- 4.13 Summary
- 4.14 Glossary



- 4.15 Answers to In-text Questions
- 4.16 Self-Assessment Questions
- 4.17 References
- 4.18 Suggested Readings

UNIT IV

LESSON 5 PLOTTING PERFECTION: MASTERING PLOTTING & VISUALIZATION

129–151

- 5.1 Learning Objectives
- 5.2 Introduction
- 5.3 Matplotlib
 - 5.3.1 Pyplot
 - 5.3.2 Concept of figure, plot, and subplot
- 5.4 Plotting Functions with Examples
 - 5.4.1 Basic Plot Functions
 - 5.4.2 Colours, Markers, and Line Styles
 - 5.4.3 Label and Legend
 - 5.4.4 Saving a Plot
- 5.5 Plotting Functions in Pandas
- 5.6 Summary
- 5.7 Glossary
- 5.8 Answers to In-text Questions
- 5.9 Self-Assessment Questions
- 5.10 References
- 5.11 Suggested Readings



UNIT V

LESSON 6	DATA UNIFICATION: EXPLORING AGGREGATION AND GROUPING	155–171
-----------------	---	----------------

- 6.1 Learning Objectives
- 6.2 Introduction
- 6.3 Data Aggregation
- 6.4 GroupBy MECHANICS
- 6.5 Pivot Tables
- 6.6 Cross-Tabulation
- 6.7 Summary
- 6.8 Glossary
- 6.9 Answers to In-text Questions
- 6.10 Self-Assessment Questions
- 6.11 References
- 6.12 Suggested Readings

UNIT I:

LESSON 1: Introduction to Data Analysis

LESSON 2: Statistical Foundations & Python Libraries



LESSON 1

INTRODUCTION TO DATA ANALYSIS

NOTES

Lavkush Gupta
Assistant Professor
Shyama Prasad Mukherji College (W)
University of Delhi
lavkush.mca16.du@gmail.com

Structure

- 1.1 Learning Objectives
- 1.2 Introduction
- 1.3 What is Data?
- 1.4 Significance of Data
- 1.5 Why Data Analysis?
- 1.6 Types of Data
- 1.7 Sources of Data Collection
- 1.8 Data Preparation
- 1.9 Exploratory Data Analysis
- 1.10 Summary
- 1.11 Glossary
- 1.12 Answers to In-text Questions
- 1.13 Self-Assessment Questions
- 1.14 References
- 1.15 Suggested Readings

1.1 LEARNING OBJECTIVES

After completion of this lesson, students will be able to learn about:

- Importance of data.
- Types of data that exist in the real world.
- Types of data used in the analysis.
- Data collection and various sources.
- Purpose of data preparation and data processing



NOTES

1.2 INTRODUCTION

Data analysis is a multifaceted process crucial for extracting valuable insights from diverse datasets. Beginning with the collection of relevant information from various sources, the analysis involves cleaning and pre-processing data to ensure accuracy and reliability. Exploratory Data Analysis (EDA) follows, employing statistical methods and Visualizations to discern patterns and trends. Subsequently, data modelling utilises mathematical models or machine learning algorithms for predictive analysis. Visualization tools and statistical packages aid in representing complex data sets in a comprehensible manner. Challenges include ensuring data quality, addressing privacy concerns, managing the complexity of large datasets, and effectively communicating results to stakeholders. As technology advances, data analysis continues to evolve, playing a pivotal role in data-driven decision-making across industries.

1.3 WHAT IS DATA?

The word ‘data’ is the plural form of the word ‘datum’, which means a single piece of information. The data word is used to represent the raw information of any organisation, company, population, etc. The raw information refers to the one that are not more organised and suitable but still has its meaning. The scientific meaning of data is “the facts and statistics collected for reference or analysis”.

1.4 SIGNIFICANCE OF DATA

The data always have their significance because we can extract meaningful information from the collection of data. We are all living in the digital era and have evolved with the huge amount of data. Without data, we can not presume our daily needs. Data is like fuel for us. Every day, we generate huge amounts of data by using smart devices like smartwatches and smart TVs, various mobile applications on phones, purchasing and selling items, billing or payments, e-mail writing, and various social media platforms (Facebook, Instagram, WhatsApp, etc.).



1.5 WHY DATA ANALYSIS?

Data analysis is a basic need for every organisation. Every organisation, like schools, businesses, technologists, and hospitals, have bulks of information related to their day-to-day operations. This information is called data. Intuitively, data are available in raw formats, so that the raw data can be organised into a specific format that would be useful for the people. We would need to apply some data handling methods such as data collection, data preparation and data analysis repetitively. Data analysis helps us in making important decisions, predictions, operations, and improvements as per the existing knowledge domain. Data Analysis has become an inevitable part of all business operations as it helps in understanding the customer's requirements, improving sales, optimising costs, and creating better problem-solving strategies. In research, a huge amount of data provides insights to present accurate and reliable models and develop graphical representations.

1.6 TYPES OF DATA

In data analysis, various types of data are encountered, each requiring distinct approaches for effective exploration and interpretation. These types can be broadly categorised into two main groups: quantitative data and qualitative data.

Quantitative Data: Quantitative data represents measurable quantities and is expressed in numerical terms. This type of data is often associated with objective observations and is suitable for statistical analysis. Key subtypes include:

- **Discrete Data:** Comprising distinct, separate values, such as counts of items or whole numbers.
- **Continuous Data:** Involving measurements that can take any value within a given range, often associated with real numbers.

Quantitative data allows for mathematical operations, making it suitable for statistical techniques like mean, median, and regression analysis. Common sources of quantitative data include sensor readings, sales figures, and test scores.



NOTES

Example: Consider a dataset containing the heights (measured in centimetres) of students in a class. Each student's height is a numerical value, and you can perform statistical analyses such as calculating the mean or standard deviation to understand the distribution of heights.

Qualitative Data: Qualitative data, on the other hand, is descriptive and non-numeric, providing insights into the qualities or characteristics of a phenomenon. It is often used to explore subjective aspects of a subject. Key subtypes include:

- **Nominal Data:** Categorizes data into distinct categories without any inherent order, such as colours or types of animals.
- **Ordinal Data:** Represents categories with a meaningful order or rank, like customer satisfaction ratings or educational levels.

Qualitative data is typically analysed through methods such as content analysis, thematic coding, or sentiment analysis. Common sources include open-ended survey responses, interview transcripts, and social media comments.

Example: Imagine a survey asking individuals about their preferred mode of transportation to work, with response options like “car”, “bicycle”, and “public transit.” The collected data represents qualitative information, as it categorises individuals based on their choices.

Time Series Data: Time series data involves observations recorded over time intervals, making it valuable for analysing trends and patterns. Examples include stock prices, weather data, and sales figures over specific periods.

For example, analysing monthly sales figures for a retail store over several years constitutes time series data. The dataset includes timestamps (months) and corresponding sales values, enabling the identification of seasonal patterns or long-term trends.

Spatial Data: Spatial data relates to geographic locations and is often used in geographic information systems (GIS). It includes information such as coordinates, addresses, or boundaries, enabling the analysis of spatial relationships and patterns.

Example: In a geographic information system (GIS), spatial data could include a map of a city's neighbourhoods, each defined by its geographical boundaries. Analysing this spatial data can reveal patterns or correlations between location and various characteristics.



Categorical Data: Categorical data represents distinct categories and is often used to group observations based on qualitative attributes. Analysing this type of data involves techniques like chi-square tests or logistic regression.

Example: Suppose you collect data on the preferred smartphone operating systems among a group of individuals, and the categories include “iOS”, “Android”, and “Other.” This information is categorical, and you can use methods like chi-square tests to determine if there are significant associations between preferences and other factors.

Understanding the type of data at hand is essential for selecting appropriate analysis methods and drawing meaningful conclusions. Often, a combination of quantitative and qualitative analyses are necessary to gain a comprehensive understanding of complex phenomena.

IN-TEXT QUESTIONS

1. Plural of Datum is _____.
2. In a class, the students were categorised into first, second, and third. This type of data is called _____.
3. The full form of EDA is _____.
4. The full form of GIS is _____.
5. The data collected for GIS is called _____.

1.7 SOURCES OF DATA COLLECTION

Data collection is a fundamental step in the data analysis process, and there are various sources from which valuable data can be gathered. These sources can be classified into primary and secondary sources, each providing unique insights into different aspects of the subject under study.

Primary Sources: Primary sources involve the direct collection of data from original or first-hand sources. These sources are often specific to the research or analysis at hand and provide data that has yet to be previously processed. Key examples include:



NOTES

- **Surveys and Questionnaires:** Gathering information by directly asking individuals or groups of people specific questions related to the research objectives.
- **Interviews:** Conducting one-on-one or group discussions with participants to gather in-depth and qualitative information.
- **Observations:** Systematically observing and recording behaviours, events, or processes in their natural settings.

Example: If studying customer satisfaction with a new product, a company might conduct surveys to collect direct feedback from customers about their experiences.

Secondary Sources: Secondary sources involve the use of existing data that has been collected by someone else for a different purpose. These sources can be valuable for historical analysis, benchmarking, or supplementing primary data.

Key examples include:

- **Published Reports and Studies:** Utilising reports and studies conducted by government agencies, research institutions, or other organisations.
- **Databases:** Accessing existing databases that compile data on various topics, such as demographic information, economic indicators, or health statistics.
- **Literature Review:** Reviewing academic articles, books, and other publications to gather insights and data relevant to the research question.

Example: If analysing trends in global climate change, a researcher might use data collected by meteorological agencies or research institutions.

Internal Sources: Internal sources involve data that is generated within an organisation or collected for internal purposes. This data is often proprietary and specific to the organisation's operations. Key examples include:

- **Sales Records:** Information on products sold, revenue generated, and customer demographics.
- **Employee Databases:** Data related to employee performance, attendance, and other HR metrics.
- **Customer Relationship Management (CRM) Systems:** Information about customer interactions, preferences, and feedback.



Example: A retail company analysing sales performance may use its internal sales records to identify trends and make informed decisions about inventory and marketing strategies.

External Sources: External sources encompass data collected by entities outside the organisation. These sources provide context and external benchmarks for analysis. Key examples include:

- **Government Data:** Census data, economic indicators, and regulatory information provided by government agencies.
- **Industry Reports:** Data and analyses produced by industry associations or market research firms.
- **Open Data:** Publicly available datasets shared by organisations, governments, or communities.

Example: A researcher studying economic trends may use external sources such as government reports on unemployment rates and GDP growth.

Effective data collection involves a thoughtful combination of these sources, depending on the research objectives, available resources, and the nature of the analysis. Researchers and analysts must carefully consider the strengths and limitations of each source to ensure the reliability and validity of the collected data.

1.8 DATA PREPARATION

It is the second most important step in the execution of data. Since the data we collected from any source do not necessarily have to be available in a properly organised format. Various kinds of dearth occur in collected data, such as semantics mistakes, logical mistakes, value errors, poorly formatted, partially available data, and irrelevant and misconfigured data. Before applying data analysis, we apply data formatting in such a way that the collected data which is prepared with correct semantics, is logical and relevant, and is configured for analysis. Data preparation, a crucial facet of the data analysis process, is devoted to the meticulous cleaning, organising, and transformation of raw data into a format conducive to meaningful analysis. This preparatory phase is instrumental

NOTES



NOTES

in guaranteeing the accuracy, completeness, and relevance of the data in alignment with the research or analysis objectives.

1. Data Cleansing:

Data cleansing is a meticulous process involving the identification and rectification of errors, inconsistencies, and missing values within the dataset. This critical step safeguards the integrity of the data, preventing inaccuracies from impeding the analysis. Techniques such as imputation, outlier removal, and rectification of data entry errors are frequently applied during this stage.

2. Data Conversion:

Data conversion is the art of transforming raw data into a standardised format suitable for analysis. This encompasses the normalisation or scaling of numerical variables, encoding of categorical variables, and the creation of novel derived features to encapsulate underlying patterns in the data in a better way.

3. Handling Missing Data:

Addressing missing data is a pivotal aspect of data preparation. Depending on the extent of incompleteness, strategies such as imputation (substituting missing values with estimated ones) or excluding records with missing data may be employed. The objective is to minimise the impact of incomplete data on the analysis.

4. Feature Engineering:

Feature engineering is a creative process involving the creation of new features or the modification of existing ones to enhance the performance of machine learning models or facilitate the interpretation of statistical analyses. This may encompass the generation of interactive features, aggregation of information, or extraction of meaningful indicators from existing variables.

5. Data Integration:

Data integration entails the amalgamation of data from diverse sources to construct a unified dataset. Resolving inconsistencies in variable names, types, or units is often necessary in this process. The overarching goal is to forge a comprehensive database that encapsulates all pertinent information for analysis.



6. Data Formatting:

Ensuring uniformity in data formats is indispensable for authentication. This involves standardising date formats, indicators, and other data representations to facilitate seamless comparison and interpretation.

7. Data Analysis (EDA):

While technically a part of the broader data analysis process, Exploratory Data Analysis (EDA) is intricately linked to data preparation. EDA involves the visual exploration of data to discern patterns, trends, and anomalies. This iterative process informs subsequent data cleaning and transformation steps.

In summation, data preparation serves as a foundational prerequisite for effective data analysis, ensuring that the data is reliable, consistent, and primed for scrutiny. The efficacy of the ensuing analysis and subsequent models hinges upon the quality and diligence applied in the data preparation process.

1.9 EXPLORATORY DATA ANALYSIS

Data analysis (EDA) is an important step in the data analysis process, aiming to understand important features of the data set before design or evaluation. The main purpose of EDA is to find patterns, relationships, inconsistencies and important insights in data. Using Visualization and analysis techniques, EDA helps gain a deeper understanding of the underlying structure of the data, allowing informed decisions to be made and the next steps to be taken.

EDA works as a search engine, allowing analysts and data scientists to obtain data while searching. This phase is accomplished through a combination of statistical concepts, graphical representation, and data visualization techniques that help uncover patterns and trends, thereby generating hypotheses for further investigation. The key components are:

Descriptive Statistics: Descriptive statistics provide a summary of the main characteristics of the dataset.

- Mean: The arithmetic average of a variable.
- Median: The middle value of a dataset, separating it into two halves.

NOTES



NOTES

- **Mode:** The most frequently occurring value.
- **Standard Deviation:** A measure of the spread or dispersion of the data around the mean.
- **Percentiles:** Values below which a given percentage of observations fall.

Data Visualization: Visualizations are powerful tools in EDA for representing complex data patterns in a comprehensible manner. Common types of Visualizations include:

- **Histograms:** Display the distribution of a single variable, providing insights into the frequency of different values.
- **Box Plots:** Illustrate the distribution's central tendency and spread and identify potential outliers.
- **Scatter Plots:** Reveal relationships and patterns between two variables, aiding in correlation analysis.
- **Heatmaps:** Visually represent patterns in a matrix, useful for displaying relationships in multivariate data.

Correlation Analysis: Correlation analysis assesses the strength and direction of relationships between pairs of variables. Correlation coefficients, such as Pearson's correlation coefficient, help quantify the degree of linear association.

Outlier Detection: Identifying outliers is crucial in understanding data anomalies. Box plots, scatter plots, or statistical methods like the Z-score or the interquartile range (IQR) are commonly used for outlier detection.

Distribution Analysis: Analysing the distribution of variables provide insights into the shape of the data.

- **Normal Distribution:** Bell-shaped curve indicating a symmetric distribution of values.
- **Skewness:** Measures the asymmetry of a distribution.
- **Kurtosis:** Measures the "tailedness" of a distribution.

Pattern Recognition: EDA involves recognising patterns or trends within the data. This could include identifying seasonality in time series data, discerning clusters in spatial data, or recognising trends in scatter plots.



Introduction to Data Analysis

- Time Series Analysis: Identifies trends, seasonality, and cycles in time-dependent data.
- Spatial Analysis: Uncovers geographic patterns and relationships in spatial datasets.
- Cluster Analysis: Groups similar observations based on certain characteristics.

NOTES

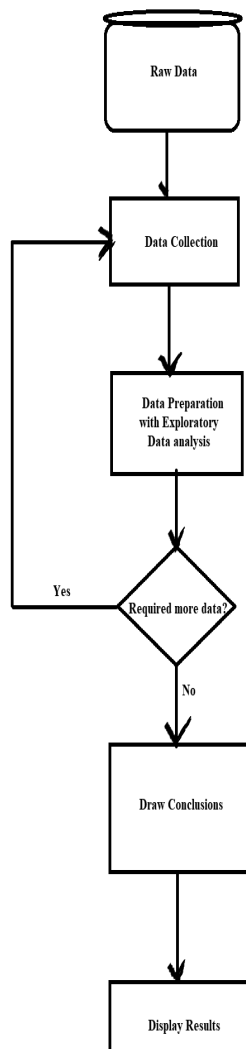


Fig 1.1: Data Analysis Flowchart

Self-Instructional
Material

13



NOTES

Benefits of Exploratory Data Analysis:

- EDA helps analysts familiarise themselves with the structure, patterns, and characteristics of the dataset.
- By visually exploring data, analysts can form hypotheses about potential relationships or patterns, guiding subsequent investigation.
- Identification of outliers and anomalies during EDA aids in the formulation of effective data-cleaning strategies.
- EDA provides insights that support decision-making processes, helping stakeholders make informed choices based on a deep understanding of the data.
- Visualizations created during EDA serve as powerful tools for communicating insights, making complex patterns accessible to both technical and non-technical audiences.
- EDA can guide the selection of appropriate modelling techniques based on the distribution and characteristics of the data.

Exploratory Data Analysis is a dynamic and iterative process that involves continuous questioning, visual exploration, and refinement. It not only uncovers patterns but also sets the stage for hypothesis testing, model building, and more advanced statistical analyses in the subsequent phases of the data analysis pipeline.

IN-TEXT QUESTIONS

6. Grouping observations based on similar characteristics is called _____.
7. A chart that can be used to represent multivariate data visually is _____.
8. Kurtosis measures _____ of the data distribution.
9. The full form of CRM is _____.
10. The most occurring value in a data is called _____.



1.10 SUMMARY

Data serves as a valuable tool for uncovering concealed insights, forecasting future trends, and scrutinising the demands and prerequisites. It has become an integral component of every business organisation, facilitating the decision-making process and enhancing the understanding of customers' needs, thereby boosting sales and optimising costs. The practice of Data Analysis extracts trends and patterns from raw data, providing crucial support for decision-making processes. Widely employed in identifying correlations and relationships for business requirements, detecting anomalies, enhancing performance, predictive modelling, and designing solutions for research problems, Data Analysis has become a pervasive and indispensable practice.

1.11 GLOSSARY

- **Data:** The data is the plural of the word 'datum', which means the piece of information. It can be available in any format like text, table, graph, etc.
- **Semantics:** It describes the meaning of data.
- **Database:** It is the collection of related data that have implicit meaning.
- **Flowchart:** It is a pictorial representation of some algorithmic task or problem.
- **Dataset:** It is the collection of data in a single file with an ordered and well-structured format. It helps to summarise the properties of data.
- **Data Wrangling:** It is the process of cleaning, transforming and structuring data from one raw form into a desired format to improve data quality and make it more useful for data analysis.

1.12 ANSWERS TO IN-TEXT QUESTIONS

1. Data
2. Ordinal Data



NOTES

3. Exploratory Data Analysis
4. Geographic Information System
5. Spatial Data
6. Cluster Analysis
7. Heat Map
8. Tailedness
9. Customer Relationship Management
10. Mode

1.13 SELF-ASSESSMENT QUESTIONS

1. What are the sequential steps of data analysis?
2. Differentiate between exploratory data analysis and data analysis.
3. What are the various ways of collecting data?
4. List a few examples of quantitative and categorical data.
5. Describe the process of data analysis with a pictorial diagram.
6. Define the qualitative and quantitative data. Illustrate with the examples.
7. Identify various forms of error that may exist in raw data.
8. Human errors are recording data in multiple versions, such as City being recorded as Delhi, DLI, and Dilli. Identify at least five different formats, each of
 - a. Date
 - b. Arrival and Departure of a train, flight (Date-Time)
 - c. Date of Birth
 - d. Annual income of an individual
 - e. Name of a person (include initials, full name, with/ without title, etc.)



9. Select any topic (educational, industrial, organisation) as per your interest, explore and perform the following activities upon it:

- Identify different sources of public use open data repositories (Kaggle, Google Data, Github, etc.)
- Download the dataset and identify the file (XML, .csv, .xls, .txt).
- Identify the number of rows (data) and columns (attributes) from the data set downloaded.
- Extract the useful information from collected data.
- Define the attributes according to the properties of specific information.
- Insert the specific information into the correct attribute.
- Identify the columns or rows with missing values, duplicate rows, and different formats (e.g. varied date formats, etc.)

NOTES

1.14 REFERENCES

- Gupta, S. C., & Kapoor, V. K. (2020). Fundamentals of Mathematical Statistics. Sultan Chand & Sons.
- Molin, S. (2019). Hands-On Data Analysis with Pandas. Packt Publishing.
- Hamid, A. O., Titi, S., & Alodat, T. (Year of Publication). Introduction to Statistics Made Easy. (2nd ed.).

1.15 SUGGESTED READINGS

- Statistics and Data Visualization with Python, Jesus Rogel-Salazar, Chapman and Hall/CRC, 2023.
- STATISTICS AND DATA VISUALIZATION WITH PYTHON, CRC Press, 2022.



LESSON 2

STATISTICAL FOUNDATIONS & PYTHON LIBRARIES

NOTES

Lavkush Gupta
Assistant Professor
Shyama Prasad Mukherji College (W)
University of Delhi
lavkush.mca16.du@gmail.com

Structure

- 2.1 Learning Objectives
- 2.2 Introduction
- 2.3 Importance of Statistics
- 2.4 Population
- 2.5 Sampling
- 2.6 Types of Statistics
- 2.7 Measures of Central Tendency
- 2.8 Measures of Dispersion
- 2.9 Scaling Features of Data
- 2.10 Relationship between random variables – Covariance & Correlation
- 2.11 Regression Analysis
- 2.12 Statistical Hypothesis Generation and Testing
- 2.13 Essentials and Motivation for using Python for Data
- 2.14 Python Libraries
- 2.15 Summary
- 2.16 Glossary
- 2.17 Answers to In-text Questions
- 2.18 Self-Assessment Questions
- 2.19 References
- 2.20 Suggested Readings

2.1 LEARNING OBJECTIVES

After completion of this lesson, students will be able to learn about:

- Role of Statistics in data analysis.
- Descriptive vs Inferential Statistics.

*Self-Instructional
Material*

19



NOTES

- Measure of central tendency – mean, median, and mode.
- Measure of Dispersion – variance, standard deviation.
- Scaling of data – min-max range, Standardisation method.
- Relationship between two random variables – covariance and correlation.
- Concepts of predictions or estimations – Illustrated with examples.
- Regression techniques – Why regression is used, mathematical interpretation.
- Introduction of different libraries – NumPy, Pandas, Matplotlib.

2.2 INTRODUCTION

This lesson serves as a gateway to a comprehensive exploration of the synergy between statistical principles and Python's robust libraries. Delving into the essential concepts of statistics, this lesson establishes a solid groundwork for understanding data and its nuances. Simultaneously, it seamlessly integrates hands-on applications through popular Python libraries like Pandas, NumPy, and Matplotlib. This cohesive approach ensures that readers not only grasp the theoretical underpinnings of statistics but also acquire practical skills to analyse and visualize data effectively using Python. Whether a novice aiming to comprehend statistical concepts or a practitioner seeking to enhance data analysis proficiency, this lesson provides a harmonious blend of theory and practical implementation, fostering a holistic understanding of statistical foundations within the Python programming landscape.

2.3 IMPORTANCE OF STATISTICS

Statistics stands as a vital and interdisciplinary branch of mathematics, playing a pivotal role in the realms of data interpretation, analysis, and visualization. As one of the foundational pillars of data science, its significance cannot be overstated. To truly grasp the intricacies of data analysis and visualization, a solid understanding of the basics of statistics is imperative. Beyond its foundational



role, the scope and importance of statistics extend across diverse domains, influencing the planning of economic development and the analysis of business data and making substantial contributions to fields such as biology, medical science, and industry. Serving as the lifeline of data analysis, statistics delineates the methods for accurate data collection, handling, and analysis. It introduces a plethora of measures that enhance the precision, predictions, accuracy, and estimation of data, making it an indispensable tool for anyone navigating the vast landscape of information and insights within diverse fields.

NOTES

2.4 POPULATION

In statistics, a “population” refers to the entire group that is the subject of a study or analysis. It includes all possible individuals, items, or observations that share a common characteristic or a set of characteristics. The key distinction is that a population encompasses every element of interest, not just a subset. Some examples are:

- Suppose you want to study the average height of all students in a school. The population, in this case, would be every student enrolled in that school, regardless of their age, grade, or other characteristics.
- Suppose a company wants to understand the average monthly spending of all its customers. The population would be every customer who has ever purchased with the company, regardless of when they started or how often they buy.
- In a medical study examining the prevalence of a genetic trait in a region, the population would be all individuals living in that region.
- During a national census, the population comprises every individual living in the country, regardless of age, gender, or any other specific criteria.

Understanding the population is crucial in statistical analysis because it helps researchers draw accurate and generalisable conclusions about the entire group based on a subset known as a sample. Due to practical constraints, it is often more feasible to study a subset of the population rather than the entire group. The insights gained from analysing a sample are then extrapolated to make inferences about the population.



NOTES

2.5 SAMPLING

When undertaking the analysis of extensive datasets or a large volume of data, it is advisable to initiate the process by examining a subset of the data, known as a sample, rather than attempting to analyse the entire set simultaneously. This approach, termed sampling, involves extracting a representative subset from the entire population or universe of data. The selection of samples is typically conducted randomly to ensure an unbiased representation.

In statistical investigations, the sample serves as a microcosm of the broader dataset, allowing for insights and inferences to be drawn without the need to analyse the entirety of the data. It is crucial to recognise that the population encompasses all possible data points, while samples are smaller, carefully selected subsets.

However, caution must be exercised when sampling, as biased sampling can lead to inaccurate results in data analysis. Biased sampling occurs when the selection process favours specific parts of the dataset over others. For instance, if samples are taken predominantly from a region or category within the dataset, the resulting analysis may not accurately reflect the characteristics of the entire dataset.

To ensure the validity and reliability of statistical analyses, it is imperative that sampling methods remain unbiased and are not skewed in favour of any observations. By maintaining a random and representative sampling approach, analysts can draw more accurate conclusions about the entire dataset based on the insights gained from the carefully selected subset.

For example, Suppose the government wants to survey their schemes, how many persons or families are getting the benefits of their scheme. If we are doing a survey, we can collect the data on a sample basis of families of a village, town, or region. After collecting the samples from different small regions, summarise the data and send the survey reports to the government. The report will be summarised by region, village, or town and how many families are getting the benefits of the government scheme.



2.6 TYPES OF STATISTICS

The field of statistics is principally categorised into two branches:

1. Descriptive Statistics:

Descriptive statistics, as its name implies, focuses on the elucidation of data. This branch aims to portray sample data in a manner that is not only meaningful but also easily comprehensible. Various methods, such as graphs, charts, tables, or spreadsheets, are employed to represent the data meaningfully. The goal is to provide a clear and representative depiction of the sample.

2. Inferential Statistics:

Inferential statistics, on the other hand, involves making inferences about populations based on data drawn from that population. This branch applies to a subset of the population and attempts to derive results, subsequently extending those findings to the entire population. Inferential statistics encompass activities like comparing data, testing hypotheses, and making predictions.

For Example, consider a scenario where you want to understand if 80% to 90% of a larger population (say, 500 people nearby) favour online shopping at Amazon. Conducting a survey with a sample of 500 people can result in descriptive statistics, like a bar chart representing “yes” or “no” responses. Alternatively, inferential statistics come into play when you analyse the samples and infer that most of the entire population likely shares the same preference for shopping at Amazon. Inferential statistics can be broadly categorised into two areas:

- **Estimating Parameters:** This involves utilising sample data to estimate population parameters. Parameters, such as sample mean, sample median, or mode, are inferred to make predictions about population parameters like population mean, population median, or mode.

Hypothesis Testing: Hypothesis testing is a statistical method used to make decisions based on experimental data. It involves forming assumptions about population parameters, and statistical conclusions are drawn to either accept or reject these assumptions.



NOTES

For Example: Suppose in a class of 100 students, there is an assumption that 40 are average, 20 are good, and 20 are poor students. Hypothesis testing provides a statistical framework to validate or challenge these assumptions, requiring a mathematical analysis to determine the veracity of the considered facts.

2.7 MEASURES OF CENTRAL TENDENCY

In the realm of data analysis, measures of central tendency play a pivotal role in distilling the essence of a dataset, revealing the central or representative values around which the data tends to cluster. These measures, comprising the mean, median, and mode provide valuable insights into the typical characteristics of a dataset, aiding analysts in understanding its central tendencies.

Mean: The mean, often referred to as the average, is a fundamental measure of central tendency. It is calculated by summing all values in the dataset and then dividing the total by the number of observations. The mean is particularly useful when seeking a balance point in the data.

For Example, let us consider a dataset representing the daily incomes of a group of individuals:

Income = [100, 200, 300, 500, 600]

The mean is calculated as:

Mean = $(100 + 200 + 300 + 500 + 600)/5 = 340$

It is important to note that the mean is sensitive to extreme values. In cases where outliers exist, the mean may be skewed towards these extremes.

Median: The median is another crucial measure of central tendency. It represents the middle value of a dataset when it is arranged in ascending or descending order. Unlike the mean, the median is less influenced by extreme values, making it a robust indicator of the central point.

For Example: Using the same income dataset:

Income = [100, 200, 300, 500, 600]

The median is the middle value (or the average of the two middle values,



in this case) after sorting the data. Here, the median = 300.

The median is particularly valuable when dealing with skewed distributions or datasets containing outliers.

Mode: The mode represents the most frequently occurring value in a dataset. A dataset can be unimodal (one mode), bimodal (two modes), or multimodal (more than two modes). The mode is especially useful for identifying peaks or concentrations within the data.

For Example: In a dataset of exam scores:

Scores = [75, 85, 90, 75, 92, 85, 88, 90]

Here, the mode is 75 and 85 since these values occur more frequently than others.

While the mode is straightforward to identify, it may not exist in every dataset, and a dataset can have more than one mode.

Collectively, these measures offer a nuanced understanding of the central tendencies within a dataset. Analysts choose the appropriate measure based on the characteristics of the data and the specific insights sought, recognising that each measure brings a unique perspective to the analysis.

2.8 MEASURES OF DISPERSION

The measure of dispersion is also known as the measure of scatter or variation. The dispersion describes the differences between the observations and the central value, i.e., the arithmetic mean. If the distance is high, dispersion would be high, which means observations would be more scattered (i.e. more variations among the observations). In the measure of dispersion, we talk about two types of variation – Absolute measures and Relative measures. The measures of dispersion in statistics that come under the category of absolute measures are.

- **Range**
- **Variance**
- **Standard Deviation**
- **Coefficient of Variation**

NOTES



NOTES

Range: In a measure of dispersion, the range is defined as the difference between the highest and lowest observed values in a given dataset. The range does not depend upon the frequencies of the observations in the dataset. To find the range of any dataset, all the individual data of the sets should be of the same unit. This measure gives information about the range of values, not given any information about how data are dispersed around its centre or mean.

Range = highest value – lowest value

For Example: Calculate the range of given data.

Data (x)	10	29	12	11	16
Frequency (f)	4	5	7	9	1

Firstly, check the highest and lowest values in the given data table, highest value = 29 and lowest value = 10, irrespective of the frequency of data.

We know that range = highest value – lowest value
 $= 29 - 10 = 19$

Variance: As we discussed in range, we would not be able to get detailed information on data and how it is dispersed. The variance measures dispersion around the mean and overcomes the measure of range. It is denoted by σ^2 . The variance is computed by the ratio of the number of observations to the average squared difference from the mean.

$$\sigma^2 = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n}$$

For Example: Calculate the variance of given data.

Data (x)	10	29	12	11	18
-----------------	----	----	----	----	----

Variance is calculated by - $\sigma^2 = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n}$



First, calculate the mean,

$$\begin{aligned}\bar{x} &= \frac{\sum_{i=1}^n x_i}{n} \\ &= \frac{10+29+12+11+18}{5} = \frac{80}{5} = 16\end{aligned}$$

$$\begin{aligned}\text{Now, variance} &= \frac{(10-16)^2 + (29-16)^2 + (12-16)^2 + (11-16)^2 + (18-16)^2}{5} \\ &= \frac{(-6)^2 + (13)^2 + (-4)^2 + (-5)^2 + (2)^2}{5} \\ &= \frac{36+169+16+25+4}{5} \\ &= \frac{250}{5} = 50\end{aligned}$$

Standard Deviation: It overcomes the limitations of variance measure because the variance computation is done by taking the square of differences of all observations with mean. The standard deviation measure of dispersion is the square root of the variance. It is denoted by σ .

$$\sigma^2 = \sqrt{\frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n}}$$

For example: Find the standard deviation of the above example.

In the above example, we calculated a variance of 50.

$$\begin{aligned}\text{Since standard deviation} &= \sqrt{\text{variance}} \\ &= \sqrt{50}\end{aligned}$$

Coefficient of variation: Unlike variance and standard deviation, the coefficient of variation is a relative measure of dispersion. This type of measure of dispersion has no units. The ratio of standard deviation to the mean of observations is called the coefficient of variation.

NOTES



NOTES

$$\text{Coefficient of variation} = \frac{\text{standard deviation}}{\text{mean}}$$

For example, Calculate the coefficient of variation of given data.

Data (x)	10	29	12	11	18
----------	----	----	----	----	----

We know that *the coefficient of variation* = $\frac{\text{standard deviation}}{\text{mean}}$

First, calculate the variance, then use the formula of

$$\text{Standard Deviation} = \sqrt{\frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n}}$$

Finally, we can calculate the ratio of standard deviation and mean, which is the coefficient of variation.

Variance is calculated by,

$$\sigma^2 = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n}$$

First, calculate the mean, $\bar{x} = \frac{\sum_{i=1}^n x_i}{n}$

$$= \frac{10 + 29 + 12 + 11 + 18}{5} = \frac{80}{5} = 16$$

$$\text{Now, variance} = \frac{(10-16)^2 + (29-16)^2 + (12-16)^2 + (11-16)^2 + (18-16)^2}{5}$$

$$= \frac{(-6)^2 + (13)^2 + (-4)^2 + (-5)^2 + (2)^2}{5}$$

$$= \frac{36 + 169 + 16 + 25 + 4}{5}$$

$$= \frac{250}{5} = 50$$



Now, standard deviation = $\sqrt{50}$

Hence, the coefficient of variation = $\sqrt{50} / 16$

2.9 SCALING FEATURES OF DATA

The scaling of data is the need, and comparison of data is very difficult when you do the distribution of data in either different measurement units or if data is having different values. The scaling of data helps in a comparison of different data values by applying the scaling features. For example, suppose the dataset has a few columns in which their columns are of different measurement units like meters and kilograms. In that case, it will be challenging to compare the data of both columns due to different measuring units. Here, we are discussing two methods of scaling data:

1. Min-Max scaling.

In min-max scaling of data, find the range using minimum and maximum values of data. We must consider every data point, let us say x of the dataset, and subtract the minimum value of the dataset; after that, take the ratio of the resultant value with the range. Then, our data will be scaled or normalised, and we can do comparisons. The formula of min-max scaling is:

$$x_{scaled} = \frac{x - x_{minimum}}{range}$$

2. Standardisation method

In the standardisation method of scaling the data, we take the ratio of differences of each data point to mean with the standard deviation. The formula is given below:

$$z = \frac{x - \bar{x}}{s.d.}$$

Here, z is the new value, x is the observation of old values or data points, $\bar{x}$ is the mean, and s.d. is the standard deviation.



NOTES

For example, suppose we have a dataset of a few vehicles containing data about their model, volume, weight, etc. We want to compare the volume and weight, but both columns have different measurement units, and the values have huge differences between them. Like volume 1.0 is very far from weight 600 in magnitude, 1.5 is also far from 700.

```
import pandas as pd
import numpy as np
# Sample DataFrame
data = {'Vehicle': ['Car', 'Maruti', 'Swift', 'Dzire', 'Bolero'],
        'Model': ['M1', 'M2', 'M3', 'M4', 'M5'],
        'Volume': [1, 1.5, 1.3, 1.8, 1.7],
        'Weight': [600, 700, 750, 800, 850]}
df = pd.DataFrame(data)
df
```

	Vehicle	Model	Volume	Weight
0	Car	M1	1.0	600
1	Maruti	M2	1.5	700
2	Swift	M3	1.3	750
3	Dzire	M4	1.8	800
4	Bolero	M5	1.7	850

Fig 2.1: DataFrame (Vehicle)

First, calculate the mean of the weight column of the dataset by using the NumPy library.

```
wt_mean = df['Weight']
#Finding the mean value of weight column
print('The Mean of weight =', np.mean(wt_mean))

The Mean of weight = 740.0
```

Secondly, find the standard deviation:



```
wt_std = df['Weight']
#Finding the standard deviation value:
print('The standard deviation of weight =', np.std(wt_std))

The standard deviation of weight = 86.02325267042627
```

Similarly, find the mean and standard deviation of the volume column using code:

```
v = df['Volume']
#Finding the mean value of volume column
print('The Mean of volume =', np.mean(v))
#Finding the standard deviation value:
print('The standard deviation of volume =', np.std(v))

The Mean of volume = 1.46
The standard deviation of volume = 0.2870540018881465
```

If you take the **weight** column from the data set above, the first value is 600, and the scaled value will be $(600 - 740.0) / 86.02 = -1.6275$. If you take the **volume** column from the data set above, the first value is 1.0, and the scaled value will be $(1.0 - 1.46) / 0.2870 = -1.6027$.

Now, we can compare -1.6275 and -1.6027 (since both values are now very close to each other) instead of comparing 600 and 1.0.

IN-TEXT QUESTIONS

1. How are mean, median, and mode related to each other?
2. Which measure of central tendency includes the magnitude of scores?
3. Mode refers to the value within a series that occurs _____ number of times.
a) Maximum b) Minimum c) Zero d) Infinite
4. The sum of deviations from the _____ is always zero.
a) Median b) Mode c) Mean d) None of the above



NOTES

2.10 RELATIONSHIP BETWEEN RANDOM VARIABLES – COVARIANCE & CORRELATION

Covariance

Covariance describes how two random variables depend on each other. If change occurs in one of them, then it affects another dependent variable too. The mathematical values of correlation lie in between $-\infty$ and ∞ .

For example, suppose Mr. X wants to purchase a house; he asks for the price of different houses in the same area from a property dealer. The property dealer provides a price list and size of houses to him, like below.

Size (In Sq. meter)	Price (In Rs.)
1000	1500000
1200	1800000
1500	2000000
2000	2500000

In the above list of prices and sizes, we can see that as the size of the houses grows, prices also increase. So, we can say the variable price depends on the other variable size. It means both variables are quantifying to each other in a way that if one increases (or decreases), then the other also increases (or decreases). It is the case of positive covariance.

NOTE: If two variables depend on each other in a way that if the value of one increases (or decreases), then the value of others decreases (or increases), then the covariance will be negative.

Mathematical Equation of Covariance: If X and Y are two variables that depend on each other, then covariance of X and Y is given by:

$$Cov(X, Y) = \frac{1}{n} \sum_{i=1} (X_i - \bar{X})(Y_i - \bar{Y})$$



Where n is the number of data points, $\bar{X}$ is the mean of all variables, X and $\bar{Y}$ is the mean of all variables, Y and X_i are some individual data points of the sample set, and Y_i are some individual data points of the sample set.

Correlation:

Correlation is the statistical term that tells how a pair of random variables are strongly related to each other. The mathematical values of correlation lie in a closed interval of $[-1, 1]$. It means if the values are close to -1 , then data points will have a strong negative correlation, and if values are close to $+1$, then data points will have a strong positive correlation among them. In special cases, if values are nearly 0 , then data points will not be correlated.

The mathematical Equation of Correlation is:

$$\frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum_{i=1}^n (X_i - \bar{X})^2 \sum_{i=1}^n (Y_i - \bar{Y})^2}}$$

Where $\bar{X}$ is the mean of all variables, X and $\bar{Y}$ is the mean of all variables Y and X_i and Y_i are the individual data points of the sample set.

The measure of correlation is known as the correlation coefficient or correlation index.



Fig 2.2: Covariance



NOTES

Pearson Correlation Coefficient:

The measure of correlation is known as the correlation coefficient or correlation index. The correlation coefficient talks about the direction and degree of the correlation. The Pearson correlation coefficient is the ratio of covariance of (X, Y) and the product of standard deviation with respect to x and y. It is denoted by the symbol ρ (rho).

The equation of the Pearson correlation coefficient is;

$$\rho(X, Y) = \frac{\text{cov}(X, Y)}{\sigma_X \sigma_Y}$$

The values of the coefficient lies in $-1 \leq \rho \leq 1$

Where $\text{cov}(X, Y)$ is the covariance between X and Y, σ_X represents the standard deviation of X, and σ_Y represents the standard deviation of Y.

There are some cases of the correlation coefficient, which is shown in the XY plane graph given below. In the diagram, the * symbol represents the data point, and X and Y are axes, respectively.

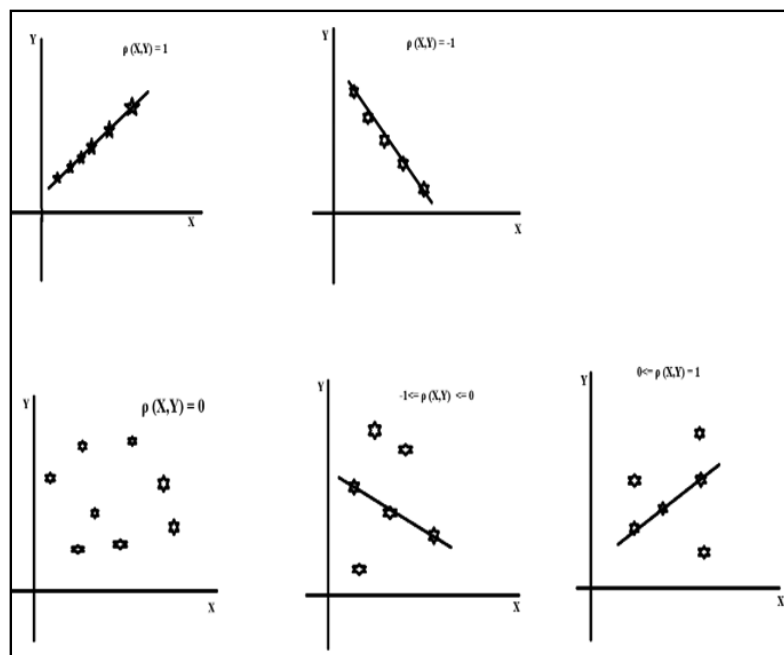


Fig 2.3: Pearson Correlation



2.11 REGRESSION ANALYSIS

Linear Regression: Linear regression is the technique of measuring or estimating the relationship among the variables. The regression approach provides the estimates of the values of the dependent variable from the values of the independent variable. The geometrical shape which is used to accomplish the estimation or prediction is the regression line. The regression line establishes the relationship between existing variables of the dataset. In this section, we will discuss linear regression, which tells about the relationship between a dependent variable and one or more independent variables (the variables that are not fixed are basically inputs or features that generate the value of a dependent variable).

For example: The regression approach estimates the house prices based on various input features like size, number of rooms, number of floors, locality, etc.

NOTE: The two most used measures of central tendency are mean and median for handling numerical data in case of regression problems.

The equation that describes linear regression is the slope form of a line in mathematics.

$$y = mx + c$$

Where y is the dependent or response variable, x is the independent or control variable, m is the slope of the line, and c is the intercept. In the terminology of data science, the above equation is called a regression equation, and the parameters m, y, and c are called regression coefficients.

The goal of regression is to learn the parameters of the model and minimise the error in prediction or forecasting.

2.12 STATISTICAL HYPOTHESIS GENERATION AND TESTING

A hypothesis is a statement about a given problem. Hypothesis testing is a statistical method to make a statistical decision using experimental data. Hypothesis testing is an assumption that we make about a population parameter.



NOTES

It evaluates two mutually exclusive statements about a population to determine which statement is best supported by the sample data.

For example:

- 1) Average number of patients visiting a hospital is 57.
- 2) Plants growing in sunny areas have darker leaves.

Hypothesis Testing

- **Null Hypothesis (H_0):** The null hypothesis is a general given statement or default position that there is no relationship between two measured cases or no relationship among groups. In other words, it is a basic assumption or made based on the problem knowledge.
- **Alternative Hypothesis (H_1):** The alternative hypothesis is the hypothesis used in hypothesis testing that is contrary to the null hypothesis.
- **Level of significance:** It refers to the degree of significance in which we accept or reject the null hypothesis. 100% accuracy is not possible for accepting a hypothesis; a level of significance that is usually 5% is selected. This is normally denoted with α , and generally, it is 0.05 or 5%, which means the output should be 95% confident to give a similar kind of result in each sample.
- **P-value:** The P value, or calculated probability, is the probability of finding the observed/extreme results when the null hypothesis (H_0) of a study-given problem is true. If the P-value is less than the chosen significance level, then reject the null hypothesis, i.e. accept that your sample claims to support the alternative hypothesis.
- **Steps in Hypothesis Testing:**
 - Step 1: Identify the problem and make an assumption statement so that assumption statements are contradictory to one another
 - Step 2: Consider statistical assumptions such as whether the data is normal or not and statistical independence between the data.
 - Step 3: Decide the test data on which the hypothesis will be tested.
 - Step 4: The data for the tests are evaluated. Evaluate various scores like z-score and mean values.



Step 5: Decide whether to accept the null hypothesis or reject the null hypothesis.

- **Formula for Hypothesis Testing**

To validate our hypothesis about a population parameter, we use statistical functions. We use the z-score, p-value, and level of significance (alpha) to make evidence for our hypothesis.

Z-TEST

✚ Formula to find the value of Z (z-test) is:

$$Z = \frac{\bar{x} - \mu_0}{\sigma / \sqrt{n}}$$

✚ $\bar{x}$ = mean of sample
✚ μ_0 = mean of population
✚ σ = standard deviation of population
✚ n = no. of observations

Fig 2.4: Z-test formula

IN-TEXT QUESTIONS

5. The statistical term which tells about how the pair of random variables are strongly related to each other is called _____.
6. The values of the coefficient lie in _____.
7. The _____ approach can be used to estimate the house prices based on various input features like size, number of rooms, number of floors, locality, etc.
8. In a hypothesis test, the p-value is compared to the significance level (α) to make a decision. What happens if the p-value is less than α ?
 - a. Reject the null hypothesis.
 - b. Fail to reject the null hypothesis.
 - c. Accept the null hypothesis.
 - d. The significance level is adjusted.

NOTES



NOTES

2.13 ESSENTIALS AND MOTIVATION FOR USING PYTHON FOR DATA VISUALIZATION

Python's rich ecosystem, led by libraries like Matplotlib and Seaborn, empowers you to create insightful and compelling visualizations effortlessly. Its simplicity, versatility, and readability make Python accessible for beginners while offering advanced capabilities for seasoned professionals. Harness the seamless integration with data manipulation libraries like Pandas, ensuring a smooth workflow. Enjoy the vibrant community support, extensive documentation, and constant updates, ensuring you stay at the forefront of visualization techniques. Whether you're a data scientist, analyst, or enthusiast, Python is your key to transforming raw data into meaningful, impactful insights.

The use of python is guided by Python Enhancement Proposals (PEP) and the Zen of Python as they collectively represent the core values and evolutionary mechanisms that guide the development and growth of the Python programming language. PEPs are formal design documents providing information to the Python community or describing a new feature for Python or its processes. These proposals undergo rigorous review and discussion within the community before being accepted or rejected. On the other hand, the Zen of Python, a collection of aphorisms by Tim Peters, encapsulates the philosophy that shapes Python's design principles. It serves as a set of guiding ideals, emphasizing simplicity, readability, and practicality in code. PEPs and the Zen of Python together exemplify the commitment to transparency, collaboration, and a mindful approach to software development that has contributed to Python's success and popularity in the programming world. They provide a framework that not only governs the language's evolution but also fosters a sense of community and shared values among Python developers worldwide.

Few aphorisms of Zen of python are given below that should be kept in mind of every programmer using python:

- Beautiful is better than ugly.
- Explicit is better than implicit.
- Simple is better than complex.



- Complex is better than complicated.
- Flat is better than nested.
- Sparse is better than dense.
- Readability counts.
- Special cases aren't special enough to break the rules.
- Although practicality beats purity.
- Errors should never pass silently.

Data visualization is the graphical representation of data to uncover insights, patterns, and trends that may not be immediately apparent in raw data. It involves using visual elements such as charts, graphs, and maps to present information in a way that is easily understandable and digestible. The primary goal of data visualization is to communicate complex data in a visual format, enabling individuals to comprehend, analyse, and make informed decisions. Visualisation enhances data exploration, facilitates the identification of correlations, and aids in storytelling by turning abstract numbers and statistics into compelling visual narratives. Various tools and techniques, including programming languages like Python and specialized software, are employed to create effective data visualisations across different domains, including business, science, and journalism.

Data Visualisation is a critical aspect of extracting meaningful insights from complex datasets, and Python stands out as an exceptional tool for this purpose. Its popularity in the field of data science, analytics, and visualization is well-earned, offering a plethora of advantages that make it the go-to language for professionals and enthusiasts alike.

Rich Ecosystem of Libraries:

Python boasts a robust ecosystem of libraries specifically designed for data visualization. Prominent among them are Matplotlib, Seaborn, Plotly, and Bokeh. Matplotlib, the foundational library, provides a high degree of customization, enabling users to create a wide range of static plots and charts. Seaborn, built on top of Matplotlib, simplifies the process of creating aesthetically pleasing statistical graphics. Plotly and Bokeh, on the other hand, excel in interactive visualizations, allowing users to create dynamic and engaging plots.

NOTES



NOTES

Ease of Learning and Use:

Python's syntax is clear and readable, making it an ideal language for beginners. The straightforward syntax reduces the learning curve, enabling users to quickly grasp the fundamentals and start creating visualizations. The simplicity of Python makes it accessible to a diverse audience, from data scientists to business analysts, ensuring that individuals with various backgrounds can harness its power for effective data representation.

Seamless Integration with Data Manipulation Libraries:

Python seamlessly integrates with powerful data manipulation libraries such as Pandas and NumPy. Pandas, in particular, excels at data cleaning, manipulation, and analysis. The ability to effortlessly import, manipulate, and visualize data in the same environment streamlines the workflow, eliminating the need to switch between different tools and languages.

Community Support and Documentation:

Python has a vast and active community that contributes to its continuous improvement. The wealth of online resources, forums, and community-driven documentation ensures that users can find solutions to problems and stay updated on the latest developments. This support network is invaluable for both beginners seeking guidance and experienced practitioners looking to enhance their skills.

Versatility and Flexibility:

Python's versatility extends beyond data visualization. It is a general-purpose programming language that can be used for various tasks, from web development to machine learning. This versatility allows professionals to use Python throughout the entire data science pipeline, fostering consistency and efficiency in their projects.

Open Source Philosophy:

Python follows an open-source philosophy, meaning that its source code is freely available to the public. This fosters collaboration and innovation within the community. Users can contribute to the development of libraries, report issues, and benefit from the collective expertise of a global community of developers.



Integration with Jupyter Notebooks:

Python integrates seamlessly with Jupyter Notebooks, a popular tool in the data science community. Jupyter Notebooks provide an interactive environment where users can create and share documents containing live code, equations, visualizations, and narrative text. This integration enhances the reproducibility of analyses and facilitates collaborative work.

State-of-the-Art Machine Learning Integration:

Python is the language of choice for many machine learning practitioners. The integration of data visualization with machine learning workflows is crucial for understanding model performance, exploring feature importance, and communicating results. Python's popularity in both domains ensures a smooth transition between data exploration and machine learning model development. In conclusion, Python's dominance in the realm of data visualization is a result of its powerful libraries, ease of use, seamless integration with data manipulation tools, strong community support, and versatility. Whether you are a seasoned data scientist or someone new to the field, Python provides the tools and resources needed to transform raw data into meaningful, actionable insights. By leveraging Python for data visualization, you not only unlock a world of possibilities but also join a thriving community that is shaping the future of data science and analytics.

2.14 PYTHON LIBRARIES

Python libraries are pre-written sets of code that extend the functionality of the Python programming language, making it more powerful and versatile for a wide range of tasks. These libraries encapsulate reusable modules, functions, and classes, allowing developers to save time and effort by leveraging existing code rather than building everything from scratch. Python's extensive library ecosystem is a key factor in its popularity, contributing to its versatility across diverse domains such as data science, machine learning, web development, and more. Some well-known Python libraries include NumPy for numerical computing, Pandas for data manipulation, Matplotlib for data visualization, TensorFlow and PyTorch for machine learning, and Django for web development.

NOTES



NOTES

The wealth of open-source Python libraries encourages collaboration, facilitates code reuse, and empowers developers to efficiently address complex challenges in various fields. The vibrant community support around these libraries is a testament to Python's strength as a language of choice for developers worldwide. Some of the popular Python Libraries are given below:

Numpy:

NumPy, short for Numerical Python, is a fundamental library for numerical computing in Python. It provides support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these arrays. NumPy is a cornerstone in the Python data science ecosystem and is widely used for tasks involving numerical computations, linear algebra, statistics, and more.

To install NumPy, you can use Python's package manager, pip. Open a terminal or command prompt and run the following command:

```
pip install numpy
```

This command fetches the latest version of NumPy from the Python Package Index (PyPI) and installs it on your system. Alternatively, if you're using a Jupyter Notebook or an Integrated Development Environment (IDE) like Anaconda, you can install NumPy using their respective package management systems.

Once installed, you can import NumPy into your Python scripts or notebooks using:

```
import numpy as np
```

This standard import statement is a convention, and it allows you to use the alias np when referring to NumPy in your code, making it more concise and readable. With NumPy, you can efficiently perform array manipulations, mathematical operations, and other numerical tasks in a Pythonic and efficient manner.

Pandas

Pandas is a powerful and widely-used open-source data manipulation and analysis library for Python. It provides data structures like Series and DataFrame, which



are designed to handle and manipulate structured data seamlessly. Pandas excels in tasks related to cleaning, transforming, and analyzing data, making it an essential tool in the data science and analytics toolkit.

To install Pandas, you can use Python's package manager, pip. Open a terminal or command prompt and run the following command:

```
pip install pandas
```

This command fetches the latest version of Pandas from the Python Package Index (PyPI) and installs it on your system. If you're using a Jupyter Notebook or an integrated development environment (IDE) like Anaconda, you can install Pandas using their respective package management systems.

Once installed, you can import Pandas into your Python scripts or notebooks using:

```
import pandas as pd
```

The standard import statement uses the alias `pd` for Pandas, making it a common and convenient convention. With Pandas, you can efficiently handle and manipulate tabular data, perform operations like filtering, grouping, and aggregation, and seamlessly integrate data from various sources for in-depth analysis.

Matplotlib

Matplotlib is a popular 2D plotting library for Python that produces high-quality static, animated, and interactive visualizations. It provides a wide range of customizable charts and plots, making it a versatile tool for data visualization.

To install Matplotlib, you can use Python's package manager, pip. Open a terminal or command prompt and run the following command:

```
pip install matplotlib
```

This command fetches the latest version of Matplotlib from the Python Package Index (PyPI) and installs it on your system. If you're using a Jupyter Notebook or an integrated development environment (IDE) like Anaconda, you can install Matplotlib using their respective package management systems.

NOTES



NOTES

Once installed, you can import Matplotlib into your Python scripts or notebooks using:

```
import matplotlib.pyplot as plt
```

The standard import statement uses the alias `plt` for Matplotlib's `pyplot` module, a widely adopted convention for brevity. With Matplotlib, you can create line plots, scatter plots, bar charts, histograms, and more, enabling you to effectively visualize and communicate insights from your data. Its flexibility and integration with other libraries like NumPy make it a go-to choice for data scientists, researchers, and analysts for generating high-quality visualizations.

IMPORTING AND EXPORTING DATA AND FILES

In Python, there are several libraries and modules that facilitate the import and export of data in various formats. Two commonly used libraries for this purpose are Pandas and NumPy. Here's a brief overview of how to import and export data using these libraries:

Importing Data:

1. Using Pandas:

Importing data from a CSV file:

```
import pandas as pd  
df = pd.read_csv('filename.csv')
```

Importing data from an Excel file:

```
import pandas as pd  
df = pd.read_excel('filename.xlsx', sheet_name='Sheet1')
```

Importing data from a SQL database:

```
import pandas as pd  
import sqlite3  
conn = sqlite3.connect('database.db')  
query = 'SELECT * FROM table_name'  
df = pd.read_sql(query, conn)
```



2. Using NumPy:

Importing data from a text file:

```
import numpy as np
data = np.loadtxt('filename.txt')
```

Importing data from a CSV file:

```
import numpy as np
data = np.genfromtxt('filename.csv', delimiter=',')
```

Exporting Data:

1. Using Pandas:

Exporting data to a CSV file:

```
import pandas as pd
df.to_csv('output_filename.csv', index=False)
```

Exporting data to an Excel file:

```
import pandas as pd
df.to_excel('output_filename.xlsx', index=False, sheet_name='Sheet1')
```

Exporting data to a SQL database:

```
import pandas as pd
import sqlite3
conn = sqlite3.connect('output_database.db')
df.to_sql('output_table_name', conn, index=False, if_exists='replace')
```

2. Using NumPy:

Exporting data to a text file:

```
import numpy as np
np.savetxt('output_filename.txt', data)
```

Exporting data to a CSV file:

```
import numpy as np
np.savetxt('output_filename.csv', data, delimiter=',')
```



NOTES

These examples showcase the basic usage of Pandas and NumPy for importing and exporting data in various formats. Depending on your specific use case and the type of data you are working with, you may choose the appropriate library and method for your needs.

2.15 SUMMARY

Statistics is the foundation and pillar of data science. It is the interdisciplinary field of mathematics and computer science. Suppose you want to achieve expertise in data analysis. In that case, you should be aware of the different statistical terminologies - mean, median, mode, range, variance, standard deviation, and coefficient of variation. It provides help in solving real problems easily. Statistics are mainly divided into two parts – descriptive statistics and inferential statistics. Descriptive statistics analyses the collection and summarisation of the data. Inferential statistics works on a sample data and infers predictions for the population. The regression approach is widely used to solve the problems of labelled data and make predictions.

2.16 GLOSSARY

- **Observations:** These are all the individual values of any data collection (sample or population) or dataset.
- **Variables:** The properties or characteristics of any object that will be analysed by using statistical techniques.
- **Frequency:** It is the occurrences of any data item(s) in a given data collection/dataset.
- **Absolute measure:** It is the measure of dispersion that can be measurable in some units with their data. It can be in meters, cm, grams, kg, etc.
- **Relative measure:** It is the measure of dispersion that has no measurable units with their data. These are all coefficients based on coefficient of variation, coefficient of range, coefficient of mean, coefficient of standard deviation, etc.



- **Dataset:** It is the collection of large data items/ data points, which have their values. Most of the time, we use the CSV (comma-separated values) or XLS (excel spreadsheets) format of the dataset to solve the problems.
- **Distribution:** It means spreading of data across any point or region.
- **Libraries:** Python has a wide collection of built-in libraries, which contain bunches of modules or groups of functions which help solve real problems. It makes Python a more popular programming language for data analysis.

NOTES

2.17 ANSWERS TO IN-TEXT QUESTIONS

1. Mode= 3 Median- 2 Mode
2. Mean
3. Maximum
4. Mean
5. Correlation
6. $-1 \leq \rho \leq 1$
7. regression
8. a. Reject the null hypothesis

2.18 SELF-ASSESSMENT QUESTIONS

1. What is the role of statistics in data analysis?
2. Differentiate descriptive and inferential statistics.
3. Define population and sampling; how are both related? Explain with an example.
4. Illustrate the measures of central tendency.
5. Explain covariance and correlation with suitable examples.
6. Describe the various categories of measures of dispersion with examples.



NOTES

7. What do you mean by regression? Give its mathematical interpretation.
8. Create or download any small dataset (of two or three numerical columns) from any source and write a program using Python libraries and functions to apply the following statistical operations over it:
 - a) Find out the mean, median, and mode of any numerical data type column(s)
 - b) Find the range, variance, and standard deviation of any numerical data type column(s).

2.17 REFERENCES

- Gupta S.C., Kapoor V.K., Fundamentals of Mathematical Statistics, Sultan Chand & Sons, 2020.
- Molin S. Hands-On Data Analysis with Pandas, Packt Publishing, 2019
- A. O. Hamid, Titi S., Alodat T., Introduction to Statistics Made Easy, Second edition.

2.18 SUGGESTED READINGS

- Mastering Python Data Visualization, Kirthi Raman, O'Reilly, 2015
- Statistics and Data Visualization with Python, Jesus Rogel-Salazar, Chapman and Hall/CRC, 2023
- Statistics and Data Visualization With Python, CRC Press, 2022

UNIT II:

LESSON 3: NumPy: The Art of Array Manipulation



LESSON 3

NUMPY: THE ART OF ARRAY MANIPULATION

NOTES

Lavkush Gupta

Assistant Professor

Shyama Prasad Mukherji College (W)

University of Delhi

lavkush.mca16.du@gmail.com

Structure

- 3.1 Learning Objectives
- 3.2 Introduction
- 3.3 NumPy Array
- 3.4 Creating Nd-array
- 3.5 Attributes of Nd-array
- 3.6 Data types of Nd-array
- 3.7 Mathematical operations in Nd-arrays
- 3.8 Random modules & their usage
- 3.9 Indexing & Slicing
- 3.10 Reshaping and Transposing Operations
- 3.11 Swapping Axes
- 3.12 Summary
- 3.13 Glossary
- 3.14 Answers to In-text Questions
- 3.15 Self-Assessment Questions
- 3.16 References
- 3.17 Suggested Readings

3.1 LEARNING OBJECTIVES

After completion of this lesson, students will be able to learn about:

- Usage of NumPy arrays.
- Different mathematical operations with NumPy arrays.
- Conversion of list into Nd-arrays.
- Functions and attributes of NumPy.

*Self-Instructional
Material*

51



NOTES

- NumPy Data types.
- Working with random modules and their built-in functions
- Concepts of Indexing & Slicing with various operations.
- Transposition & Swapping axes

3.2 INTRODUCTION

Python is a very popular programming language which is widely used for data analysis. Python has various highly well-defined, essential, and popular libraries, which are the reason behind its use for data analysis and visualization. These libraries save users time and are well-optimised in the implementation of tasks for analysis and visualization purposes. NumPy refers to the “Numerical Python”. In this lesson, we delve into the powerful capabilities of NumPy, a fundamental library for numerical computing in Python. NumPy is an essential tool for many data manipulation tasks, including cleaning, sub-setting, and transformation, because it offers quick and vectorised array operations. We examine common array algorithms such as set operations, unique identification, and sorting, as well as effective methods for data summarisation and descriptive statistics. This lesson also discusses relational manipulations and data alignment, showing how NumPy makes it easier to combine and merge disparate datasets.

3.3 NUMPY ARRAY

In NumPy, an array is the fundamental object. A pivotal feature of NumPy is its N-dimensional array object, known as Nd-array. Arrays are used to store similar data type elements in a sequential block of memory. An Nd-array is a fast and flexible container for large datasets (collection of huge amounts of data), while an array is faster but not flexible enough to adjust a large amount of data; it may take more time to insert or delete the elements in the middle positions. NumPy can do complex calculations on entire arrays without using Python for loops. Arrays allow you to execute mathematical operations on entire blocks of data using a syntax like the operations between individual scalar elements. We can



NumPy: The Art of Array Manipulation

create the Nd-array object by using the array function and passing a list to it.

There are the following steps in creating a NumPy array:

Step 1. Firstly, import the NumPy library.

Step 2. Convert the list elements into a Numpy array using the array() function and assign them to any variable.

Step 3. Print the NumPy array.

```
import numpy as np
#Numpy array using array() function
ndarray= np.array([1,2,3,4,5,6,7,8,9,10])

print("Output")
ndarray

Output
array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10])

print("Output")
print("Shape of Array is: ",ndarray.shape)
print("Datatype of Array is: ",ndarray.dtype)
print("Dimension of Array is: ",ndarray.ndim)

Output
Shape of Array is:  (10,)
Datatype of Array is:  int64
Dimension of Array is:  1
```

Fig 3.1: NumPy Array Example

3.4 CREATING ND-ARRAY

There are various ways of creating arrays in Python.

NOTES



NOTES

Creating an array from a Python list

```
import numpy as np
print("Output")
# 1D array
arr1d = np.array([1, 2, 3])
print("1D array\n",arr1d)
# 2D array
arr2d = np.array([[1, 2, 3], [4, 5, 6]])
print("2D array\n",arr2d)
# 3D array
arr3d = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])
print("3D array\n",arr3d)
```

```
Output
1D array
[1 2 3]
2D array
[[1 2 3]
 [4 5 6]]
3D array
[[[ 1  2  3]
  [ 4  5  6]]
 [[ 7  8  9]
  [10 11 12]]]
```

Creating arrays with specific values

```
import numpy as np
zeros_arr = np.zeros((2, 3)) # 2x3 array with all zeros
ones_arr = np.ones((3, 4)) # 3x4 array with all ones
full_arr = np.full((2, 2), 7) # 2x2 array with all elements set to 7
print("Output")
print("Zeros\n",zeros_arr)
print("Ones\n",ones_arr)
print("Full\n",full_arr)
```

```
Output
Zeros
[[0. 0. 0.]
 [0. 0. 0.]]
Ones
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]
 [1. 1. 1. 1.]]
Full
[[7 7]
 [7 7]]
```



Creating arrays with a range of values

NOTES

```
import numpy as np
range_arr = np.arange(0, 10, 2) # Array with values from 0 to 10 (exclusive) with a step of 2
linspace_arr = np.linspace(0, 1, 5) # Array with 5 values evenly spaced between 0 and 1
rand_arr = np.random.rand(2, 3) # 2x3 array with random values between 0 and 1
print("Output")
print("Array1\n",range_arr)
print("Arrya2\n",linspace_arr)
print("Array3\n",rand_arr)

Output
Array1
[0 2 4 6 8]
Arrya2
[0.  0.25 0.5  0.75 1. ]
Array3
[[0.77469539 0.36817691 0.16783394]
 [0.22470458 0.97957342 0.42768458]]
```

NOTE- We use the `array()` function to convert the list into Numpy array. Check the type of numpy array using `type(array_name)`

3.5 ATTRIBUTES OF ND-ARRAYS

NumPy's Nd-array (n-dimensional array) is a versatile data structure for representing arrays of numerical data. It comes with various attributes that provide information about the array. There are a few attributes that are frequently used with Nd-arrays listed here:

Shape: Returns a tuple representing the size of each dimension of the array.

```
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
print("Output")
print(arr.shape)

Output
(2, 3)
```

Dimensions: `ndim` returns the number of dimensions (axes) of the array.



NOTES

```
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
print("Output")
print(arr.ndim)
```

Output
2

Size: It returns the total number of elements in the array.

```
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
print("Output")
print(arr.size)
```

Output
6

Dtype: It returns the data type of the elements in the array.

```
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
print("Output")
print(arr.dtype)
```

Output
int64

3.6 DATA TYPES OF ND-ARRAY

Numpy supports a variety of data types for its Nd-array objects, allowing you to choose the appropriate type based on the nature of your data. Some common data types in NumPy are given below:

Integer Types:

np.int8, np.int16, np.int32, np.int64: Signed integers with 8, 16, 32, or 64 bits.



NumPy: The Art of Array Manipulation

np.uint8, *np.uint16*, *np.uint32*, *np.uint64*: Unsigned (non-negative) integers with 8, 16, 32, or 64 bits.

Floating-Point Types:

np.float16, *np.float32*, *np.float64*: Floating-point numbers with 16, 32, or 64 bits.

Complex Types:

np.complex64, *np.complex128*: Complex numbers with 64 or 128 bits.

Boolean Type:

np.bool: Boolean values, representing True or False.

String Types:

np.str_, *np.bytes_*: Variable-length string or byte strings.

Datetime Types:

np.datetime64: Represents date and time.

Object Type:

np.object: A generic object type.

Fixed-Size Unicode Types:

np.unicode_: Fixed-size Unicode string type.

You can specify the data type when creating an array using the *dtype* parameter as given in figure below.

```
import numpy as np

# Creating arrays with specified data types
arr_int32 = np.array([1, 2, 3], dtype=np.int32)
arr_float64 = np.array([1.0, 2.5, 3.7], dtype=np.float64)
arr_complex128 = np.array([1 + 2j, 3 - 4j], dtype=np.complex128)
arr_bool = np.array([True, False, True], dtype=np.bool)
arr_str = np.array(['a', 'b', 'c'], dtype=np.str_)
```

Fig 3.2: Specifying Data types of array

NOTES



NOTES

3.7 MATHEMATICAL OPERATIONS IN ND-ARRAYS

NumPy provides a wide range of mathematical operations that can be performed on Nd-arrays. Some common mathematical operations you can perform on Nd-arrays are listed below:

Element-wise Operations

NumPy allows you to perform element-wise operations, meaning that the operation is applied to each element in the array.

```
import numpy as np

arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
print("Output")
# Addition
result_add = arr1 + arr2
print("Addition",result_add)
# Subtraction
result_sub = arr1 - arr2
print("Subtraction",result_sub)
# Multiplication
result_mul = arr1 * arr2
print("Multiplication",result_mul)
# Division
result_div = arr1 / arr2
print("Division",result_div)
# Exponentiation
result_pow = arr1 ** arr2
print("Exponentiation",result_mul)
```

```
Output
Addition [5 7 9]
Subtraction [-3 -3 -3]
Multiplication [ 4 10 18]
Division [0.25 0.4  0.5 ]
Exponentiation [ 4 10 18]
```



NumPy: The Art of Array Manipulation

Aggregation Functions: Functions that are performed on the whole array cumulatively.

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
print("Output")
# Sum
sum_result = np.sum(arr)
print("Sum",sum_result)
# Mean
mean_result = np.mean(arr)
print("Mean",mean_result)
# Minimum and Maximum
min_value = np.min(arr)
max_value = np.max(arr)
print("Minimum",min_value)
print("Maximum",max_value)
```

```
Output
Sum 15
Mean 3.0
Minimum 1
Maximum 5
```

Exponential and Logarithmic Functions

```
import numpy as np
arr = np.array([1, 2, 3])
print("Output")
# Exponential
exp_values = np.exp(arr)
print("Exponential",exp_values)
# Natural logarithm
log_values = np.log(arr)
print("Natural Logarithm",log_values)
```

```
Output
Exponential [ 2.71828183  7.3890561 20.08553692]
Natural Logarithm [0.          0.69314718 1.09861229]
```

NumPy's extensive library provides many more functions for various mathematical operations, making it a powerful tool for scientific computing and data analysis.

NOTES



NOTES

3.8 RANDOM MODULES AND THEIR USAGE

Random module: The random module is an already built-in module of the Python library. This module helps in the generation of random numbers in statistical tasks where few predictive numeric data are required. The random module is leveraged with many functions or submodule(s), which helps in deep data analysis, visualization and interpretation. A few important functions of random modules are discussed below:

rand(): This function is used to generate the real numbers randomly; it returns a real number between 0 and 1.

`modulename.rand()`

```
import random as rd
print("Output")
print(rd.random( ))
```

Output
0.8515976230943195

The above real number generated using the random module lies between 0 and 1.

randint(): This function returns the integer between the specified range inside the function as an argument.

`modulename.randint(start, end)`

where *start* refers to the starting value of the specified range, and *end* refers to the last value of the specified range.

```
import random as rd
print("Output")
print(rd.randint(1,5))
```

Output
5



NumPy: The Art of Array Manipulation

The above integer number, generated using the random module, lies between a specified range of 1 and 5.

NOTE: The numbers generated using the random module would not be static. It means that as you execute the code again and again, it is not necessary to give the current output equal to the previous output.

Random sampling using NumPy: We can access the random module-based functions using the numpy library also. The NumPy module supports retrieving the random generation object. It helps to generate the random numbers by including the random attribute instead of importing the random library explicitly.

The `numpy.random.randint()` function is used to generate random samples using the numpy module.

```
import numpy as np
d = np.random.randint(1,10)
print("Output")
print(d)
```

Output
6

Generate random arrays using NumPy: As in the above example of generating a random number between a given specific range, we can also generate the random array (collection of numbers) of a specific number of rows and columns. The number of required rows and columns needs to be specified as arguments.

`numpy.random.randn(m_rows , n_cols)`

where `m_rows` refers to the number of rows and `n_cols` refers to the number of columns.

```
import numpy as np
d = np.random.randn(1,10)
print("Output")
print(d)
```

Output
[[1.17907092 0.43232024 -0.47349297 -0.31507151 -0.78652042 -0.33540207
 0.34800347 0.06936398 0.44698435 0.92089976]]

NOTE: The elements of the generated array might be positive or negative real numbers.

NOTES



NOTES

IN-TEXT QUESTIONS

1. What is the term used to describe the number of elements in an array?
2. What is the index of the first element in a one-dimensional array in most programming languages?
3. In NumPy, what does the SHAPE of an array mean?
4. What is the correct syntax to check the number of dimensions in an array?

3.9 INDEXING & SLICING

Indexing is an important list and string operation in Python. It helps in extracting a contiguous subset of the list. This operation is similar to the indexing of an array. The process of extracting a subset of a list or array using indices is called *slicing*. It makes it easy to access the array. The slice operation is applied inside a square bracket [x:y], where x is the start index, and y is the stop or last index of the slice.

NOTE: *The slicing always begins from the start index, which is inclusive, and ends with the last index, which is not inclusive.*

If there are 10 elements in a list or array A, then the indexing will be like below:

Index	0	1	2	3	4
Elements	10	12	14	16	18

Now, we can apply the slicing; suppose we want to extract elements 12, 14, and 16 whose indices are 1 to 4, then write A[1:3], where A is the name of the list, colon(:) denotes the slice operation between starting index 1(inclusive) and last index 4(exclusive).



NumPy: The Art of Array Manipulation

NOTES

```
import numpy as np
arr = np.arange(5) # create a list of five numbers 0 to 4.
print("Output")
print(arr[1]) # extract the first element of array using index 1
print(arr[1:4]) # slicing
```

Output
1
[1 2 3]

In the above code, `arr[1:4]` slices the elements from index 1 to 3, i.e., the last index value -1, since index 4 is not inclusive.

Like Python list, Nd-arrays are also mutable, i.e. we can change or modify the elements of Nd-arrays.

```
import numpy as np
nd_array = np.array([[1,2,3],[4,5,6]])
print("Output")
print(nd_array)
#change 0th row
nd_array[0] = [11,22,33]
#print modified ndarray
print('After modifying the array =',nd_array)
```

Output
[[1 2 3]
[4 5 6]]
After modifying the array = [[11 22 33]
[4 5 6]]

Representation of Nd-array elements

```
import numpy as np
nd_array = np.array([[1,2,3],[4,5,6]])
print("Output")
print(nd_array)
```

Output
[[1 2 3]
[4 5 6]]



NOTES

In the above 2×3 Nd-array, we can represent the elements in the following tabular format :

Index : [0][0] Element =1	Index : [0][1] Element =2	Index : [0][2] Element =3
Index : [1][0] Element =4	Index : [1][1] Element =5	Index : [1][2] Element =6

Table 3.1: Indexing elements in a NumPy array

Indexing with a slice operation: In a numpy array, we can retrieve or access the elements or collection of elements using the slice operator. We use the colon (:) to represent and divide the slices from the list or numpy array in Python.

The syntax to apply the slicing is as follows:

`numpy.array_name[start_index : last_index]`

Where start_index is the first index, last_index refers to the predecessor index value (last_index -1) whose values will be taken into the slice operation respectively.

```
#import the numpy library
import numpy as np
#create a numpy array of first numbers 0 to 4.
arr = np.arange(5)
#Display the first 3 indexed elements arr[0], arr[1] and arr[2]
print("Output")
print(arr[0:3])
```

Output
[0 1 2]

Program: Create a 6×6 matrix using Nd-array and illustrate the indexing and slicing operation



NumPy: The Art of Array Manipulation

NOTES

```
import numpy as np
#Create a 6x6 matrix using nd-array
a = np.array([[0, 1, 2, 3, 4, 5],
              [6, 7, 8, 9, 10, 11],
              [12, 13, 14, 15, 16, 17],
              [18, 19, 20, 21, 22, 23],
              [24, 25, 26, 27, 28, 29],
              [30, 31, 32, 33, 34, 35]])

print("Output")
print("\n Array is:\n ",a)
# slicing and indexing
print("\n a[0, 3:5] = ",a[0, 3:5])
print("\n a[4:, 4:] = ",a[4:, 4:])
print("\n a[:, 2] = ",a[:, 2])
print("\n a[2::2, ::2] = ",a[2::2, ::2])
print("\n Shape of array",a.shape)
print("\n Dimension of array", a.ndim)
```

Output

```
Array is:
[[ 0  1  2  3  4  5]
 [ 6  7  8  9 10 11]
 [12 13 14 15 16 17]
 [18 19 20 21 22 23]
 [24 25 26 27 28 29]
 [30 31 32 33 34 35]]

a[0, 3:5] = [3 4]

a[4:, 4:] = [[28 29]
 [34 35]]

a[:, 2] = [ 2  8 14 20 26 32]

a[2::2, ::2] = []

Shape of array (6, 6)

Dimension of array 2
```



NOTES

Some Advanced types of indexing:

There are some specific types of indexing used in generating random data for Nd-arrays.

i. Boolean Array Indexing

ii. Fancy Indexing

In boolean array indexing, the index can be a conditional expression consisting of $>$, $<$, $=$, $!=$, the elements of the array are returned when it satisfies the indexing of the expression.

```
import numpy as np
arr = np.array([2, 10, 18, 26, 34])
print("Output")
print(arr[arr>5])
```

```
Output
[10 18 26 34]
```

Explanation: In the above example, the name of the numpy array is arr, and the boolean expression 'arr > 5' is used as the index of the array. The condition is true for the elements whose values are greater than 5, i.e., 10, 18, 26, and 34.

```
#import the numpy library
import numpy as np
a = np.array([1, 2, 3])
b = np.array([True, True, False])
c = a[b]
print("Output")
print(c)
```

```
Output
[1 2]
```

For the true value of the array index, elements are displayed, and for false, they are not.

Numpy.empty() function: It returns a new array of a given shape and type with random values.

Syntax: `numpy.empty(shape, dtype = float)`



NumPy: The Art of Array Manipulation

Where **shape** represents a number of rows, and **dtype** is optional attribute that represents the float (by default) data type of the returned array.

NOTE: Here, `empty( )` does not refer to zeros (zero values).

For example: Create a random matrix of shape 2×2 using `numpy.empty( )`

```
import numpy as np
print("Output")
print(np.empty([2, 2]))
```

```
Output
[[4.9e-323 8.9e-323]
 [1.3e-322 1.7e-322]]
```

It displays the 2×2 matrix with exponential real numbers.

Fancy Indexing:

The fancy indexing of numpy in Python provides an advanced facility to retrieve the group of elements in an array assigned to a variable. This indexing provides more efficient features over the numpy array like sorting, filtering, conditional access, etc.

```
import numpy as np
# create a numpy array
arr = np.array([10, 12, 30, 40, 15, 19, 45, 18])
# grouped the elements at index 1, 2, 5, 7
grouped_elements = arr[[1, 2, 5, 7]]
print("Output")
print(grouped_elements)
```

```
Output
[12 30 19 18]
```

In the above code of the example, we selected the values at indices of 1, 2, 5, and 7 into a variable 'grouped_elements'.

Code to create a 5×5 shape matrix and fill the number pattern from 0 to 4 row-wise.

NOTES



NOTES

The following pattern should be displayed.

[0 0 0 0 0]

[1 1 1 1 1]

[2 2 2 2 2]

[3 3 3 3 3]

[4 4 4 4 4]

```
import numpy as np
#create 5x5 shape matrix
arr = np.empty((5,5),dtype =int)
print("Output")
print(arr)
#fill the matrix pattern using loop statement.
for i in range(5):
    arr[i] = i
print("Required pattern :")
print(arr)
```

Output

```
[[ 94395837989049 0 0
 -4616189618054758400 4607182418800017408]
 [-4611686018427387904 4611686018427387904 -4616189618054758400
 4607182418800017408 0]
 [ 4611686018427387904 4607182418800017408 4607182418800017408
 4611686018427387904 0]
 [ 4607182418800017408 -4616189618054758400 4611686018427387904
 -4611686018427387904 4607182418800017408]
 [-4616189618054758400 0 -4611686018427387904
 -4616189618054758400 208]]
```

Required pattern :

```
[[0 0 0 0 0]
 [1 1 1 1 1]
 [2 2 2 2 2]
 [3 3 3 3 3]
 [4 4 4 4 4]]
```

NOTE: You should know one difference between slicing and fancy indexing is that the slicing operation does not create a new numpy array object explicitly when it applies, but in the case of fancy indexing, it creates a new numpy array and copies the operated data into a new array.



3.10 RESHAPING AND TRANSPOSING OPERATIONS

Reshaping

Reshaping in NumPy refers to changing the shape or dimensions of an array while maintaining the total number of elements. This operation is useful in various scenarios, such as preparing data for specific algorithms, combining or splitting arrays, or aligning data for mathematical operations. Below are some examples:

Reshaping a 1D Array to a 2D Array

```
import numpy as np

arr_1d = np.array([1, 2, 3, 4, 5, 6])
reshaped_arr_2d = arr_1d.reshape((2, 3))
print("Output")
print("Original 1D Array:")
print(arr_1d)
print("\nReshaped 2D Array:")
print(reshaped_arr_2d)
```

```
Output
Original 1D Array:
[1 2 3 4 5 6]

Reshaped 2D Array:
[[1 2 3]
 [4 5 6]]
```



NOTES

The reshape function is a convenient way to change the shape of an array.

```
arr = np.array([1, 2, 3, 4, 5, 6])
reshaped_arr = np.reshape(arr, (2, 3))
print("Output")
print("Original Array:")
print(arr)
print("\nReshaped Array:")
print(reshaped_arr)
```

```
Output
Original Array:
[1 2 3 4 5 6]

Reshaped Array:
[[1 2 3]
 [4 5 6]]
```

The -1 argument in the reshape function allows NumPy to calculate the size of one dimension automatically based on the size of the others.

```
arr = np.array([1, 2, 3, 4, 5, 6])
reshaped_arr = np.reshape(arr, (2, -1))
print("Output")
print("Original Array:")
print(arr)
print("\nReshaped Array with automatic size calculation:")
print(reshaped_arr)
```

```
Output
Original Array:
[1 2 3 4 5 6]

Reshaped Array with automatic size calculation:
[[1 2 3]
 [4 5 6]]
```

Flattening an array means converting a multi-dimensional array into a 1D array. This can be achieved using the flatten method.



NumPy: The Art of Array Manipulation

NOTES

```
arr_2d = np.array([[1, 2, 3],  
                  [4, 5, 6]])
```

```
flattened_arr = arr_2d.flatten()  
print("Output")  
print("Original 2D Array:")  
print(arr_2d)  
print("\nFlattened 1D Array:")  
print(flattened_arr)
```

```
Output  
Original 2D Array:  
[[1 2 3]  
 [4 5 6]]  
  
Flattened 1D Array:  
[1 2 3 4 5 6]
```

Reshaping can be used to concatenate multiple arrays or split a single array into multiple ones.

```
arr1 = np.array([[1, 2],  
                [3, 4]])  
  
arr2 = np.array([[5, 6]])  
combined_arr = np.concatenate((arr1, arr2), axis=0)  
split_arr = np.split(combined_arr, [2])  
print("Output")  
print("Array 1:")  
print(arr1)  
print("\nArray 2:")  
print(arr2)  
print("\nCombined Array:")  
print(combined_arr)  
print("\nSplit Array:")  
print(split_arr)
```

```
Output  
Array 1:  
[[1 2]  
 [3 4]]  
  
Array 2:  
[[5 6]]  
  
Combined Array:  
[[1 2]  
 [3 4]  
 [5 6]]  
  
Split Array:  
[array([[1, 2],  
        [3, 4]]), array([[5, 6]])]
```




NOTES

Transposing

Transposing in NumPy involves changing the arrangement of elements in an array by flipping the array along its main diagonal. This operation is quite useful in various mathematical and computational tasks, such as matrix operations, data manipulation, and linear algebra. The transpose of a matrix is obtained by switching its rows with columns.

Even though transposing a 1D array does not have a significant effect, NumPy allows it. The result will be the same as the original array.

```
import numpy as np
arr_1d = np.array([1, 2, 3, 4, 5])
transposed_arr_1d = np.transpose(arr_1d)
print("Output")
print("Original 1D Array:")
print(arr_1d)
print("\nTransposed 1D Array:")
print(transposed_arr_1d)
```

```
Output
Original 1D Array:
[1 2 3 4 5]

Transposed 1D Array:
[1 2 3 4 5]
```

For 2D arrays, transposing involves swapping rows and columns.

```
arr_2d = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])

transposed_arr_2d = np.transpose(arr_2d)
print("Output")
print("Original 2D Array:")
print(arr_2d)
print("\nTransposed 2D Array:")
print(transposed_arr_2d)
```

```
Output
Original 2D Array:
[[1 2 3]
 [4 5 6]
 [7 8 9]]

Transposed 2D Array:
[[1 4 7]
 [2 5 8]
 [3 6 9]]
```



NumPy: The Art of Array Manipulation

NumPy arrays also have a `.T` attribute, which can be used to obtain the transpose of an array.

```
arr_2d = np.array([[1, 2, 3],
                  [4, 5, 6],
                  [7, 8, 9]])

transposed_arr_2d = arr_2d.T
print("Output")
print("Original 2D Array:")
print(arr_2d)
print("\nTransposed 2D Array using T attribute:")
print(transposed_arr_2d)
```

```
Output
Original 2D Array:
[[1 2 3]
 [4 5 6]
 [7 8 9]]

Transposed 2D Array using T attribute:
[[1 4 7]
 [2 5 8]
 [3 6 9]]
```

The transposition of arrays with more than two dimensions involves permuting the axes based on the desired order.

```
arr_3d = np.random.rand(2, 3, 4)
transposed_arr_3d = np.transpose(arr_3d, (1, 2, 0))
print("Output")
print("Original 3D Array:")
print(arr_3d)
print("\nTransposed 3D Array:")
print(transposed_arr_3d)
```

```
Output
Original 3D Array:
[[[0.59952288 0.69537181 0.80287528 0.00942622]
  [0.35494703 0.95186879 0.38003904 0.80111641]
  [0.08395584 0.76156874 0.44031072 0.99664963]]

 [[0.12397223 0.43729182 0.13636865 0.98706989]
  [0.04534977 0.22579839 0.29487329 0.86262962]
  [0.34578642 0.99442415 0.35670222 0.11864484]]]

Transposed 3D Array:
[[[0.59952288 0.12397223]
  [0.69537181 0.43729182]
  [0.80287528 0.13636865]
  [0.00942622 0.98706989]]

 [[0.35494703 0.04534977]
  [0.95186879 0.22579839]
  [0.38003904 0.29487329]
  [0.80111641 0.86262962]]

 [[0.08395584 0.34578642]
  [0.76156874 0.99442415]
  [0.44031072 0.35670222]
  [0.99664963 0.11864484]]]
```

NOTES



NOTES

3.11 SWAPPING AXES

The swapping axes are nothing but an interchange between two axes of the array.

`numpy.swapaxes(arr, axis1, axis2)`

Where the parameters are described as follows: arr is the name of the array, axis1 represents the first axis, and axis2 represents the second axis.

```
# importing python module named numpy
import numpy as np
arr = np.array([[11,22,33]])
# apply numpy.swapaxes() function
swapped = np.swapaxes(arr, 0, 1)
print("Output")
print (swapped)
```

```
Output
[[11]
 [22]
 [33]]
```

NOTE: In case of applying the swap axes function, you should take the parameter tuple of two elements having 1 and 0, both like (1,0) or (0,1). Suppose you take both elements of a tuple the same, like (1,1) or (0,0). Then, the swapping of rows and columns will not be performed.

IN-TEXT QUESTIONS

5. What does the reshape function in NumPy do?
6. What is the purpose of the transpose function in NumPy?
7. In NumPy, how is slicing different from indexing?
8. What does the following NumPy code accomplish: `arr[:,2]`?
9. How can you flatten a multi-dimensional NumPy array?



3.12 SUMMARY

In this lesson, we discussed the numpy library and the basic NumPy mathematical operations. The NumPy is a short form of Numerical Python, which is used for numerical operations. The NumPy array is the main data structure of this library; by using this, we can perform various operations like the creation of a multi-dimensional array and matrix structure and maintaining the records in tabular format. Indexing is an important feature of the NumPy array, which facilitates frequent access to the data. Indexing can be of various types, such as simple indexing, boolean indexing, and fancy or advanced indexing. According to the requirements of data processing, we apply a specific type of indexing. Slicing is also an important feature of a NumPy array, which we can use to retrieve the subset or part of data or records stored in the array. The random module is used frequently for creating random numbers in Python; this concept facilitates solving problems on random password generation, game winner predictions, lucky draw, etc.

3.13 GLOSSARY

- **Numpy:** Numpy is the short form of numerical Python. It is the python numerical library, which is widely used for numerical operations.
- **Nd-array:** It refers to an n-dimensional array; in this textbook, we use the terms Nd-array, numpy array, and Nd-dimensional array. They are all synonymous terms.
- **Variables:** The variables are containers that store any value in a memory block.
- **Rows:** In any Nd-array, the elements of the horizontal line refer to the elements of the row.
- **Columns:** In any Nd-array, the elements of the vertical line refer to the columns elements.
- **Attribute:** The properties of any object are called attributes; here, the

NOTES



NOTES

properties which we are performing over a numpy array represent the attribute.

- **Transposing:** It is the operation of a matrix of Nd-array in which we exchange the position of rows' and columns' elements with each other.

3.14 ANSWERS TO IN-TEXT QUESTIONS

1. size
2. 0
3. The shape is the number of elements in each dimension.
4. arr.ndim
5. Reshapes the dimensions of an array
6. Swaps the axes of an array
7. Indexing is used for accessing individual elements while slicing extracts a subarray
8. Select every second element of the array
9. Using the ravel function

3.15 SELF-ASSESSMENT QUESTIONS

1. Explain any two functions that can be used to create numpy array objects, with suitable examples.
2. Define the following attributes in the context of a numpy array :
 - i. Data type
 - ii. Shape
 - iii. Reshape
 - iv. Dimension
3. Differentiate between numpy array and array.



NumPy: The Art of Array Manipulation

4. What do you mean by indexing and slicing? Discuss types of indexing and illustrate with examples.
5. Describe the random modules and any two functions of it.
6. Write a Python code to create a numpy array of 5×5 and display the shape, dimension, and data type.
7. Write a function in Python to take the list as input and return a numpy array object as output.
8. Write the Python code to generate a 5×5 identity matrix whose elements contain all ones.
9. Write a Python code to find the cube of all elements of the numpy array.
10. Write a Python code to generate the numpy array of any twenty random numbers.

NOTES

3.16 REFERENCES

- McKinney W. Python for Data Analysis: Data Wrangling with Pandas, NumPy and IPython. 2nd edition. O'Reilly Media, 2018.
- Molin S. Hands-On Data Analysis with Pandas, Packt Publishing, 2019

3.17 SUGGESTED READINGS

- Data Visualization with Python, Mario Dobler, Tim Großmann, Packt Publishing Limited, 2019
- Data Visualization in Python, Daniel Nelson, StackAbuse.com, 2020
- Mastering Python Data Visualization, Kirthi Raman, O'Reilly, 2015

UNIT III:

LESSON 4: Pandas Power Play: Mastering Data Manipulation



LESSON 4

PANDAS POWER PLAY: MASTERING DATA MANIPULATION

NOTES

Lavkush Gupta
Assistant Professor
Shyama Prasad Mukherji College (W)
University of Delhi
lavkush.mca16.du@gmail.com

Structure

- 4.1 Learning Objectives
- 4.2 Introduction
- 4.3 Pandas Series
 - 4.3.1 Creating a Pandas Series
 - 4.3.2 Accessing Elements in a Pandas Series
 - 4.3.3 Operations on Pandas Series
- 4.4 DataFrame
- 4.5 Index Objects
- 4.6 Working with DataFrame
 - 4.6.1 Arithmetic Operations
 - 4.6.2 Statistical Functions
- 4.7 Binning
- 4.8 Indexing and Reindexing
 - 4.8.1 Indexing
 - 4.8.2 Reindexing
- 4.9 Filtering
- 4.10 Handling Missing Data
- 4.11 Hierarchical Indexing
- 4.12 Data Wrangling
- 4.13 Summary
- 4.14 Glossary
- 4.15 Answers to In-text Questions
- 4.16 Self-Assessment Questions
- 4.17 References
- 4.18 Suggested Readings



NOTES

4.1 LEARNING OBJECTIVES

After completion of this lesson, students will be able to learn about:

- Usage of Pandas library.
- Creation of DataFrame object.
- Creation of series, empty series.
- Loading of data into DataFrames.
- Arithmetic & Statistical working of DataFrames.
- Filtering of data
- Various types of hierarchical indexing and reshaping.

4.2 INTRODUCTION

Python is a very powerful language, and it is increasingly being used for scientific applications. Matrix and vector manipulations, which involve storing and analysing data in single as well as multi-dimensional arrays, form the backbone of scientific computations. We often use Pandas (“Python Data Analysis Library”) as an essential library for applications, including machine learning and data sciences, due to their extensive functionalities that support high-performance matrix computation capabilities.

Before using this module, it must first be installed. Once installed, the module can be imported through the Python script. Importing a module means loading it in memory to use its functionalities. Pandas can be imported by writing the following statement:

```
import pandas as pd
```



4.3 PANDAS SERIES

In Pandas, a Series is a one-dimensional labelled array that can hold any data type, such as integers, strings, floats, or even Python objects. It provides a powerful and flexible data structure for manipulating and analysing data. All Pandas data structures are mutable. This means that their values can be changed. However, a Series is an exception since it is immutable.

4.3.1 Creating a Pandas Series

Series in Python is a one-dimensional array that can store data of any type (integer, string, float, python objects, etc.). A pandas Series can be created using the following constructor:

```
pandas.Series( data, index, dtype, copy)
```

where data can be a list of values, an nd-array, a dictionary, or even a scalar value, constant index values must be unique and hashable, having the same length as data (default np.arange(n) if no index is passed), dtype specifies the data type and copy means to copy data. By default, its value is False.

Below are some examples of creating a series:

Create an Empty Series

To create a basic empty series, the `pd.series()` function is used without passing any arguments to it.

```
#import the pandas library and aliasing as pd
import pandas as pd
S = pd.Series()
print("Output")
print(S)
```

```
Output
Series([], dtype: float64)
```



NOTES

Creating a Series from a List

You can create a Series by passing a Python list to the `pd.Series` constructor.

```
import pandas as pd
# Creating a Series from a list
data_list = [10, 20, 30, 40, 50]
series_from_list = pd.Series(data_list)
print("Output")
print(series_from_list)
```

```
Output
0    10
1    20
2    30
3    40
4    50
dtype: int64
```

Creating a Series from a NumPy Array

You can also create a Series from a NumPy array.

```
import pandas as pd
import numpy as np
# Creating a Series from a NumPy array
data_array = np.array([10, 20, 30, 40, 50])
series_from_array = pd.Series(data_array)
print("Output")
print(series_from_array)
```

```
Output
0    10
1    20
2    30
3    40
4    50
dtype: int64
```



Pandas Power Play: Mastering Data Manipulation

Creating a Series from a Dictionary

A Series can be created from a Python dictionary where keys become the index labels.

```
import pandas as pd
# Creating a Series from a dictionary
data_dict = {'a': 10, 'b': 20, 'c': 30, 'd': 40, 'e': 50}
series_from_dict = pd.Series(data_dict)
print("Output")
print(series_from_dict)
```

```
Output
a      10
b      20
c      30
d      40
e      50
dtype: int64
```

Creating a Series with a Custom Index

You can explicitly specify the index while creating a Series.

```
import pandas as pd
# Creating a Series with a custom index
data_list = [10, 20, 30, 40, 50]
custom_index_series = pd.Series(data_list, index=['one', 'two', 'three', 'four', 'five'])
print("Output")
print(custom_index_series)
```

```
Output
one      10
two      20
three    30
four     40
five     50
dtype: int64
```

Creating a Series with Scalar Values

You can create a Series with scalar values and a specified index.

NOTES



NOTES

```
import pandas as pd
# Creating a Series with scalar values
scalar_series = pd.Series(5, index=['a', 'b', 'c', 'd', 'e'])
print("Output")
print(scalar_series)
```

```
Output
a      5
b      5
c      5
d      5
e      5
dtype: int64
```

Creating a Series from Date Range

Pandas provides the 'pd.date_range' function to create a Series with a date index.

```
import pandas as pd
#Creating a Series with a date index
date_series = pd.Series([1, 2, 3, 4, 5], index=pd.date_range('2022-01-01', periods=5))
print("Output")
print(date_series)
```

```
Output
2022-01-01    1
2022-01-02    2
2022-01-03    3
2022-01-04    4
2022-01-05    5
Freq: D, dtype: int64
```

These examples show different ways to make Series in Pandas. Depending on your data and what you want to do, you can pick the way that works best for you. Series are important for many things you can do with Pandas, so it is crucial to understand how to create them to work with data effectively.

4.3.2 Accessing Elements in a Pandas Series

There are several ways to access elements in a Pandas Series, including index, label, or conditionally.



Accessing by Index

You can access elements in a Series by their index, like accessing elements in a Python list or array. For example:

```
import pandas as pd
#Creating a Series with a date index
date_series = pd.Series([1, 2, 3, 4, 5], index=pd.date_range('2022-01-01', periods=5))
print("Output")
print(date_series)
print("Access by Index",date_series[3])
```

```
Output
2022-01-01    1
2022-01-02    2
2022-01-03    3
2022-01-04    4
2022-01-05    5
Freq: D, dtype: int64
Access by Index 4
```

Accessing by Label

You can also access elements in a Series using their label instead of the index. To do this, you need to assign labels to the elements when creating the series.

Accessing Conditionally

You can also access elements in a Series based on a condition. For example, to access all elements greater than 30, you can use the following code:

```
import pandas as pd
data = [10, 20, 30, 40, 50]
labels = ['A', 'B', 'C', 'D', 'E']
s = pd.Series(data, index=labels)
print("Output")
print(s)
print("Access by Labels",s['C'])
```

```
Output
A    10
B    20
C    30
D    40
E    50
dtype: int64
Access by Labels 30
```

NOTES



NOTES

In this case, the condition ' $s > 30$ ' returns a Boolean Series where each element is compared to 30. Using this Boolean Series as an index for the original Series ' s ', we can access only the elements that satisfy the condition.

4.3.3 Operations on Pandas Series

Pandas Series supports a wide range of operations, including arithmetic operations, logical operations, statistical calculations, and data manipulations.

Arithmetic Operations

You can perform arithmetic operations on Pandas Series, such as addition, subtraction, multiplication, and division. These operations are performed element-wise.

```
import pandas as pd
s1 = pd.Series([1, 2, 3, 4])
s2 = pd.Series([10, 20, 30, 40])
s3 = s1 + s2
print("Output")
print(s3)
```

```
Output
0    11
1    22
2    33
3    44
dtype: int64
```

In this example, we add two Series ' $s1$ ' and ' $s2$ ' using the '+' operator. The resulting Series ' $s3$ ' contains the element-wise addition of the corresponding elements in ' $s1$ ' and ' $s2$ '.

Logical Operations

You can also perform logical operations on Pandas Series, such as equality ('='), inequality ('!='), greater than ('>'), and less than ('<'). These operations return a Boolean Series representing the result of the logical operation.



```
import pandas as pd
s = pd.Series([1, 2, 3, 4])
condition = s > 2
print("Output")
print(condition)
```

```
Output
0    False
1    False
2     True
3     True
dtype: bool
```

In this example, we perform a logical operation ' $s > 2$ ' on the Series ' s '. The resulting boolean series ' $condition$ ' contains ' $True$ ' if the corresponding element in ' s ' is greater than 2, and ' $False$ ' otherwise.

Statistical Calculations

Pandas Series provides various statistical functions to compute descriptive statistics for the data. Some commonly used functions include ' $mean()$ ', ' $median()$ ', ' $sum()$ ', ' $min()$ ', ' $max()$ ', ' $std()$ ', ' $var()$ ', etc.

```
import pandas as pd
s = pd.Series([10, 20, 30, 40, 50])
print("Output")
print(s.mean())
print(s.max())
print(s.std())
```

```
Output
30.0
50
15.811388300841896
```

In this example, we compute the mean, maximum value, and standard deviation of the Series ' s ' using the functions ' $mean()$ ', ' $max()$ ', and ' $std()$ ', respectively.



NOTES

IN-TEXT QUESTIONS

1. What is a Pandas Series?
2. How can you create a Pandas Series from a Python list?
3. What is the primary purpose of the index in a Pandas Series?

4.4 DATAFRAMES

In the realm of data analysis with Python, the Pandas library stands out for its robust capabilities. At the core of Pandas lies the DataFrame, a versatile two-dimensional data structure that simplifies the handling and manipulation of structured data. It can be likened to a table or spreadsheet, where information is organised into rows and columns. DataFrame offers a powerful toolset for data manipulation, cleaning, and analysis. They provide a structured and efficient way to manage diverse types of data, making them essential for tasks ranging from exploratory data analysis to complex statistical modelling. The basic features of DataFrame can be given as:

- Columns of different data types.
- Size is mutable.
- DataFrame has labelled axes for rows and columns.
- Arithmetic operations can be performed on rows and columns.

Creating a DataFrame

A pandas DataFrame can be created by using the `pandas.DataFrame` function. The syntax of this function can be given as:

```
pandas.DataFrame( data, index, columns, dtype, copy)
```

where,

- data can be an nd-array, series, map, lists, dict, constants or any other DataFrame.
- The index denotes the row labels for the resulting frame. It is an optional argument. By default, if no index is passed, then its value will be equal to `np.arange(n)`.



Pandas Power Play: Mastering Data Manipulation

- column is used for specifying column labels. It is an optional argument. By default, its value is `np.arange(n)` if no index is passed.
- dtype specifies the data type of each column.
- copy is used for copying data if the default is False.

Some examples of creating a DataFrame are given below.

```
import pandas as pd
# Creating a DataFrame
data = {'Name': ['Alice', 'Bob', 'Charlie'],
        'Age': [25, 30, 22],
        'City': ['New York', 'San Francisco', 'Los Angeles']}
df = pd.DataFrame(data)
# Setting 'Name' column as the index
df.set_index('Name', inplace=True)
print("Output")
print(df)
```

Output

	Age	City
Name		
Alice	25	New York
Bob	30	San Francisco
Charlie	22	Los Angeles

NOTES

Loading Data into Pandas DataFrame

One of the key strengths of Pandas is its ability to handle diverse data sources effortlessly. Loading data into a Pandas DataFrame is often the first step in any data analysis or manipulation task. In this section, we will explore different methods for loading data into a Pandas DataFrame, ranging from simple CSV files to more complex data sources.

Reading Data from CSV Files

CSV (Comma-Separated Values) files are a common and straightforward format for storing tabular data. Pandas provides a simple method, `pd.read_csv()`, to read data from a CSV file into a DataFrame.



NOTES

```
import pandas as pd
# Loading data from a CSV file
file_path = '/content/StudentMarks.csv' #path to csv file
df_csv = pd.read_csv(file_path)
# Displaying the first few rows of the DataFrame
print("Output")
print(df_csv.head())
```

Output

	Roll No	Name	Marks
0	1	Rohit	78
1	2	Aishwarya	77
2	3	Vaidehi	68
3	4	Charu	90
4	5	Reema	99

Reading Data from Excel Files

Pandas supports the extraction of data from Excel files, a widely used format in business and research environments.

```
import pandas as pd
# Loading data from an Excel file
excel_path = '/content/StudentMarks.xlsx'#path of file
df_excel = pd.read_excel(excel_path, sheet_name='StudentMarks')
# Displaying the first few rows of the DataFrame
print("Output")
print(df_excel.head())
```

Output

	Roll No	Name	Marks
0	1	Rohit	78
1	2	Aishwarya	77
2	3	Vaidehi	68
3	4	Charu	90
4	5	Reema	99

Customizing Loading Options

Pandas provides various parameters for customising the loading process, such as specifying columns, handling missing values, and setting custom indexes. We can customise loading options as well:



```
df_custom = pd.read_csv(file_path, usecols=['column1',
'column2'], na_values=['NA', 'N/A'], index_col='custom_index')
```

Pandas' flexibility in loading data from a wide range of sources makes it a powerful tool for data analysis. Whether dealing with static files, databases, or real-time web data, Pandas simplifies the process of getting your data into a format suitable for analysis in a DataFrame. Understanding these loading techniques is fundamental for any data analyst or scientist working with Pandas.

4.5 INDEX OBJECTS

In Pandas, an Index is an integral part of a DataFrame, serving as labels for both rows and columns. It allows for streamlined data retrieval, alignment, and manipulation. Each DataFrame has both row and column indexes, and comprehending how to work with these indexes is crucial for effective data handling. We can use the `index.is_object()` method to check if the Pandas Index is of the object dtype. An index can be created using the `pd.index()` method.

Default Numeric Index

By default, if an index is not explicitly specified, Pandas assigns a numeric index starting from 0.

```
import pandas as pd
# Creating a DataFrame without specifying an index
data = {'Name': ['Alice', 'Bob', 'Charlie'],
        'Age': [25, 30, 22],
        'City': ['New York', 'San Francisco', 'Los Angeles']}
df = pd.DataFrame(data)
print("Output")
print(df.index)

Output
RangeIndex(start=0, stop=3, step=1)
```



NOTES

Custom Index

You can set a custom index to make the DataFrame more meaningful. Let us set the 'Name' column as the index.

```
# Setting 'Name' column as the index
df.set_index('Name', inplace=True)
print(df)
```

	Age	City
Name		
Alice	25	New York
Bob	30	San Francisco
Charlie	22	Los Angeles

Index with Date Range

Pandas provides functionality to create an index based on date ranges, which is useful for time series data.

```
# Creating a DataFrame with a date range index
date_index = pd.date_range(start='2022-01-01', periods=3, freq='D')
df_date = pd.DataFrame(data, index=date_index)
print("Output")
df_date
```

Output

	Name	Age	City
2022-01-01	Alice	25	New York
2022-01-02	Bob	30	San Francisco
2022-01-03	Charlie	22	Los Angeles



Hierarchical Index

You can create a DataFrame with a hierarchical index, which allows for multi-level indexing.

```
# Creating a DataFrame with a hierarchical index
data_multi_index = {'Age': [25, 30, 22, 28, 35, 30],
                    'City': ['New York', 'San Francisco', 'Los Angeles', 'New York', 'San Francisco', 'Los Angeles']}
multi_index = pd.MultiIndex.from_product([['Group A', 'Group B'], ['Alice', 'Bob', 'Charlie']], names=['Group', 'Name'])
df_multi_index = pd.DataFrame(data_multi_index, index=multi_index)
print("Output")
print(df_multi_index)
```



Pandas Power Play: Mastering Data Manipulation

NOTES

Output			
	Name	Age	City
Group A	Alice	25	New York
	Bob	30	San Francisco
	Charlie	22	Los Angeles
Group B	Alice	28	New York
	Bob	35	San Francisco
	Charlie	30	Los Angeles

Resetting the Index

You can reset the index, which brings back the default numeric index, and the current index becomes a new column.

```
# Resetting the index
df_reset = df.reset_index()
print("Output")
print(df_reset)
```

Output			
	Name	Age	City
0	Alice	25	New York
1	Bob	30	San Francisco
2	Charlie	22	Los Angeles

Understanding and effectively using index objects in Pandas is crucial for efficient data manipulation and analysis. Whether it is customising indexes, using date ranges, or creating hierarchical indexes, the choice of index can significantly impact how you interact with your data.

4.6 WORKING WITH DATAFRAMES

4.6.1 Arithmetic Operations

Working with arithmetic operations on DataFrames in Pandas is a fundamental aspect of data manipulation and analysis. Pandas allows you to perform element-wise arithmetic operations, broadcasting operations, and various mathematical operations on entire DataFrames. Let us explore these concepts with examples:



NOTES

Element-wise Arithmetic Operations

Element-wise operations involve applying an arithmetic operation to each corresponding element in two DataFrames. This is similar to NumPy array operations.

```
import pandas as pd
# Creating two DataFrames
df1 = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
df2 = pd.DataFrame({'A': [7, 8, 9], 'B': [10, 11, 12]})
# Element-wise addition
df_sum = df1 + df2
# Element-wise subtraction
df_diff = df1 - df2
# Element-wise multiplication
df_prod = df1 * df2
# Element-wise division
df_div = df1 / df2
print("Output")
print("DataFrame 1:")
print(df1)
print("\nDataFrame 2:")
print(df2)
print("\nElement-wise Addition:")
print(df_sum)
print("\nElement-wise Subtraction:")
print(df_diff)
print("\nElement-wise Multiplication:")
print(df_prod)
print("\nElement-wise Division:")
print(df_div)
```

```
Output
DataFrame 1:
   A  B
0  1  4
1  2  5
2  3  6

DataFrame 2:
   A  B
0  7 10
1  8 11
2  9 12
```



```
Element-wise Addition:
   A  B
0   8 14
1  10 16
2  12 18

Element-wise Subtraction:
   A  B
0 -6 -6
1 -6 -6
2 -6 -6

Element-wise Multiplication:
   A  B
0   7 40
1  16 55
2  27 72

Element-wise Division:
           A          B
0  0.142857  0.400000
1  0.250000  0.454545
2  0.333333  0.500000
```

Broadcasting Operations

Broadcasting allows operations between a DataFrame and a scalar value. The scalar value is broadcasted to all elements in the DataFrame.

```
#Broadcasting addition with a scalar
df_add_scalar = df1 + 10
# Broadcasting multiplication with a scalar
df_multiply_scalar = df1 * 2
print("Output")
print("\nBroadcasting Addition with Scalar:")
print(df_add_scalar)
print("\nBroadcasting Multiplication with Scalar:")
print(df_multiply_scalar)
```



NOTES

Output

Broadcasting Addition with Scalar:

```
   A  B
0  11 14
1  12 15
2  13 16
```

Broadcasting Multiplication with Scalar:

```
   A  B
0  2  8
1  4 10
2  6 12
```

Operator	Operation	Description
`+`	Addition	Adds corresponding elements in two dataframes
`-`	Subtraction	Subtracts corresponding elements in two dataframes
`*`	Multiplication	Multiplies corresponding elements in two dataframes
`/`	Division	Divides corresponding elements in two dataframes
`//`	Floor Division	Performs floor division on corresponding elements
`%`	Modulo	Computes the remainder of division of corresponding elements
`**`	Exponentiation	Raises each element in the dataframe to the power of another

Mathematical Functions

Pandas provides a range of mathematical functions that can be applied to entire DataFrames.

```
# Applying mathematical functions
df_sqrt = df1.applymap(lambda x: x**0.5) # Square root of each element
df_square = df1.applymap(lambda x: x**2) # Square of each element
df_exp = df1.applymap(lambda x: 2**x) # Exponential of each element
print("Output")
print("\nSquare Root of DataFrame:")
print(df_sqrt)
print("\nSquare of DataFrame:")
print(df_square)
print("\nExponential of DataFrame:")
print(df_exp)
```



Pandas Power Play: Mastering Data Manipulation

NOTES

Output

Square Root of DataFrame:

```
      A      B
0  1.000000  2.000000
1  1.414214  2.236068
2  1.732051  2.449490
```

Square of DataFrame:

```
      A      B
0  1  16
1  4  25
2  9  36
```

Exponential of DataFrame:

```
      A      B
0  2  16
1  4  32
2  8  64
```

Understanding how arithmetic operations work on DataFrames is crucial for data cleaning, preprocessing, and analysis. Whether you are performing simple element-wise operations or more complex mathematical functions, Pandas provide a powerful and flexible framework for handling numerical data in a tabular format. Some functions are given in Table 4.1.

Function	Description	Example
<code>abs()</code>	Absolute value of each element	<code>df.abs()</code>
<code>sqrt()</code>	Square root of each element	<code>np.sqrt(df)</code>
<code>exp()</code>	Exponential (e^x) of each element	<code>np.exp(df)</code>
<code>log()</code>	Natural logarithm of each element	<code>np.log(df)</code>
<code>log10()</code>	Base-10 logarithm of each element	<code>np.log10(df)</code>
<code>sin()</code> , <code>cos()</code> , <code>tan()</code>	Trigonometric functions	<code>np.sin(df)</code> , <code>np.cos(df)</code> , <code>np.tan(df)</code>
<code>ceil()</code> , <code>floor()</code>	Ceiling and floor of each element	<code>np.ceil(df)</code> , <code>np.floor(df)</code>
<code>round()</code>	Round each element to the nearest integer	<code>np.round(df)</code>

Self-Instructional
Material

99



NOTES

<code>mean()</code>	Mean of each column (axis=0) or row (axis=1)	<code>df.mean(axis=0)</code> , <code>df.mean(axis=1)</code>
<code>sum()</code>	Sum of each column (axis=0) or row (axis=1)	<code>df.sum(axis=0)</code> , <code>df.sum(axis=1)</code>
<code>min()</code> , <code>max()</code>	Minimum and maximum of each column (axis=0) or row (axis=1)	<code>df.min(axis=0)</code> , <code>df.min(axis=1)</code> , <code>df.max(axis=0)</code> , <code>df.max(axis=1)</code>

Table 4.1: Functions

4.6.2 Statistical Functions

Pandas provides a variety of statistical functions that can be applied to DataFrames to obtain summary statistics, descriptive statistics, and other measures. Python supports a large number of methods for computing descriptive statistics and other related operations on Series and DataFrame objects. Some widely used functions are shown below:

1. `describe()`: This function provides a summary of descriptive statistics for each column in the DataFrame.

```
import pandas as pd
# Create a sample DataFrame
data = {'A': [1, 2, 3, 4, 5],
        'B': [10, 20, 30, 40, 50]}
df = pd.DataFrame(data)
# Use describe() to get summary statistics
summary_stats = df.describe()
print("Output")
print(summary_stats)
```

```
Output
      A      B
count 5.000000 5.000000
mean  3.000000 30.000000
std    1.581139 15.811388
min    1.000000 10.000000
25%    2.000000 20.000000
50%    3.000000 30.000000
75%    4.000000 40.000000
max    5.000000 50.000000
```



2. `mean()`: Computes the mean along a specified axis.

```
# Calculate the mean of each column
mean_values = df.mean()
print("Output")
print(mean_values)
```

```
Output
A      3.0
B     30.0
dtype: float64
```

3. `median()`: Computes the median along a specified axis.

```
# Calculate the median of each column
median_values = df.median()
print("Output")
print(median_values)
```

```
Output
A      3.0
B     30.0
dtype: float64
```

4. `std()`: Computes the standard deviation along a specified axis.

```
# Calculate the standard deviation of each column
std_dev_values = df.std()
print("Output")
print(std_dev_values)
```

```
Output
A      1.581139
B     15.811388
dtype: float64
```

NOTES



NOTES

5. `min()` and `max()`: Compute the minimum and maximum values along a specified axis.

```
# Calculate the minimum and maximum of each column
min_values = df.min()
max_values = df.max()
print("Output")
print("Minimum values:", min_values)
print("Maximum values:", max_values)
```

```
Output
Minimum values: A      1
B      10
dtype: int64
Maximum values: A      5
B      50
dtype: int64
```

6. `sum()`: Computes the sum of values along a specified axis.

```
# Calculate the sum of each column
sum_values = df.sum()
print("Output")
print(sum_values)
```

```
Output
A      15
B     150
dtype: int64
```

7. `count()`: Returns the number of non-null observations along a specified axis.



```
# Count non-null values in each column
count_values = df.count()
print("Output")
print(count_values)
```

```
Output
A      5
B      5
dtype: int64
```

8. `corr()`: Computes the correlation between columns.

```
# Calculate the correlation matrix
correlation_matrix = df.corr()
print("Output")
print(correlation_matrix)
```

```
Output
      A      B
A  1.0  1.0
B  1.0  1.0
```

9. `cov()`: Computes the covariance between columns.

```
# Calculate the covariance matrix
covariance_matrix = df.cov()
print("Output")
print(covariance_matrix)
```

```
Output
      A      B
A  2.5  25.0
B  25.0 250.0
```




NOTES

These are just a few examples of the statistical functions available in Pandas. Pandas provides a wide range of statistical functions that can be applied to DataFrames. Here is a list of some commonly used statistical functions in Pandas:

Descriptive Statistics

- `count()`: Count non-null values for each column or row.
- `describe()`: Generate descriptive statistics for each column.
- `min()`, `max()`: Compute the minimum and maximum values.
- `sum()`: Compute the sum of values.
- `mean()`: Compute the mean (average) value.
- `median()`: Compute the median value.
- `std()`, `var()`: Compute the standard deviation and variance.
- `quantile()`: Compute sample quantiles.
- `mode()`: Compute the mode.

Correlation and Covariance

- `corr()`: Compute the correlation matrix.
- `cov()`: Compute the covariance matrix.

Aggregation

- `agg()`: Aggregate using one or more operations over a specified axis.
- `apply()`: Apply a function along the axis of the DataFrame.

Missing Data

- `isna()`, `isnull()`: Detect missing values.
- `dropna()`: Drop missing values.



• <code>fillna()</code>: Fill missing values.
Cumulative and Moving Statistics
• <code>cumsum()</code>: Compute cumulative sum.
• <code>cumprod()</code>: Compute cumulative product.
• <code>cummax()</code>, <code>cummin()</code>: Compute cumulative maximum and minimum.
• <code>rolling()</code>: Provide a rolling view of a windowed operation.
Ranking
• <code>rank()</code>: Assign ranks to data elements.
Miscellaneous
• <code>idxmax()</code>, <code>idxmin()</code>: Return the row labels of the maximum and minimum values.
• <code>mad()</code>: Compute the mean absolute deviation.
• <code>round()</code>: Round each element in the DataFrame.

Key Points to Remember:

- Since DataFrame is a heterogeneous data structure, generic functions do not work with all functions.
- Functions like `sum()` and `cumsum()` work with both numeric and character or string data. Though practically, we will never use character aggregations, these functions will not throw any exception if we try to do so.
- Functions like `abs()` and `cumprod()` throw exceptions when the DataFrame contains character or string data because such operations cannot be performed on non-numerical data.

When a particular value, which is either minimum or maximum, is repeated in a DataFrame or series object, then `idxmin()` and `idxmax()` functions return the index of the first occurrence of that value. Therefore, the code



NOTES

below will give an output 2 since 3 is the maximum value that first occurs at index 2 in the DataFrame.

```
df = pd.DataFrame([1,2,3,3,1,1,3], columns=['A'], index=range(7))  
print("Output")  
print(df['A'].idxmax())
```

Output
2

IN-TEXT QUESTIONS

4. How can you calculate the mean of a specific column 'A' in a Pandas DataFrame named df?
5. What does the describe() function in Pandas do when apply to a DataFrame?
6. What is the purpose of the corr() function in Pandas when applied to a DataFrame?

4.7 BINNING

Binning is a technique used in data analysis to categorise continuous data into discrete bins or intervals. This can be helpful in various situations, such as when you want to convert a continuous variable into a categorical one or when you want to analyse data in a more aggregated or summarised form. In Pandas, you can use the 'cut' function to perform binning. Below is a detailed example of binning in a DataFrame:



Pandas Power Play: Mastering Data Manipulation

NOTES

```
import pandas as pd
import numpy as np
# Create a sample DataFrame with a continuous variable (Age)
data = {'Age': [25, 30, 22, 35, 40, 28, 18, 50, 32, 28]}
df = pd.DataFrame(data)
# Define bins and labels for age groups
bins = [18, 25, 35, 45, 60]
labels = ['18-25', '26-35', '36-45', '46-60']
# Use the cut function to create age groups
df['AgeGroup'] = pd.cut(df['Age'], bins=bins, labels=labels, right=False)
# Display the original DataFrame with age values
print("Output")
print("Original DataFrame:")
print(df[['Age']])
```

Output
Original DataFrame:

	Age
0	25
1	30
2	22
3	35
4	40
5	28
6	18
7	50
8	32
9	28

In this example:

1. Create Sample DataFrame: We start with a simple DataFrame, 'df' that contains a column named 'Age' with continuous age values.
2. Define Bins and Labels: We specify the bin edges using the 'bins' list. In this case, the bins are '[18, 25, 35, 45, 60]', indicating the age intervals. We provide corresponding labels for each bin in the 'labels' list, such as ['18-25', '26-35', '36-45', '46-60'].
3. Use 'cut' Function: The 'pd.cut' function takes the continuous variable ('df['Age']') and bins it based on the specified intervals and labels. The 'right=False' argument means that the intervals are left-closed, and the right endpoint is excluded from the interval.

Self-Instructional
Material

107



NOTES

4. Create a New Column: We create a new column, 'AgeGroup', in the DataFrame to store the age group information obtained from the binning process.

```
# Display the DataFrame with age values and corresponding age groups
print("\nDataFrame with Age Groups:")
print(df[['Age', 'AgeGroup']])
```

DataFrame with Age Groups:

	Age	AgeGroup
0	25	26-35
1	30	26-35
2	22	18-25
3	35	36-45
4	40	36-45
5	28	26-35
6	18	18-25
7	50	46-60
8	32	26-35
9	28	26-35

In the final DataFrame, the 'AgeGroup' column shows everyone's corresponding age group.

This binning technique is useful for transforming continuous data into discrete categories, making it easier to analyse and interpret certain patterns in the data. It is commonly used in data preprocessing steps before performing further analysis or modelling. This binning approach can be useful in scenarios where you want to analyse data based on certain ranges, such as age groups, income brackets, or any other continuous variable that can be logically grouped into intervals.

4.8 INDEXING AND REINDEXING

In Pandas, indexing and reindexing are important concepts that involve selecting, modifying, or rearranging the rows and columns of a DataFrame. Let us explore these concepts with examples:

4.8.1 Indexing

Basic Indexing: You can access specific columns or rows using their labels.



Pandas Power Play: Mastering Data Manipulation

NOTES

```
import pandas as pd
# Create a sample DataFrame
data = {'A': [1, 2, 3], 'B': [4, 5, 6]}
df = pd.DataFrame(data)
# Accessing a column
column_A = df['A']
# Accessing a row using iloc
row_0 = df.iloc[0]
print("Output")
print("Column A:")
print(column_A)
print("\nRow 0:")
print(row_0)
```

```
Output
Column A:
0    1
1    2
2    3
Name: A, dtype: int64

Row 0:
A    1
B    4
Name: 0, dtype: int64
```

Boolean Indexing: Use Boolean conditions to filter rows based on a condition.

```
# Filtering rows where column A is greater than 1
filtered_rows = df[df['A'] > 1]
print("Output")
print("Filtered Rows:")
print(filtered_rows)
```

```
Output
Filtered Rows:
   A  B
1  2  5
2  3  6
```



NOTES

4.8.2 Reindexing

Reindexing in Pandas is used to change the index of rows and columns of a DataFrame. Indexes can be used to reference data structures like Pandas series or Pandas DataFrame.

Reindexing Columns: Change the order of columns or add new columns.

```
# Reindexing columns
new_order = ['B', 'A']
df_reindexed_columns = df.reindex(columns=new_order)
print("Output")
print("Reindexed Columns:")
print(df_reindexed_columns)
```

```
Output
Reindexed Columns:
   B  A
0  4  1
1  5  2
2  6  3
```

Reindexing Rows: Modify the order of rows or add new rows.

```
# Reindexing rows
new_index = [2, 0, 1]
df_reindexed_rows = df.reindex(index=new_index)
print("Output")
print("Reindexed Rows:")
print(df_reindexed_rows)
```

```
Output
Reindexed Rows:
   A  B
2  3  6
0  1  4
1  2  5
```



Fill Values during Reindexing: Specify a method to fill missing values during reindexing.

NOTES

```
# Reindexing with forward fill for missing values
df_forward_fill = df.reindex(index=[0, 1, 2, 3], method='ffill')
print("Output")
print("Reindexed with Forward Fill:")
print(df_forward_fill)
```

```
Output
Reindexed with Forward Fill:
   A  B
0  1  4
1  2  5
2  3  6
3  3  6
```

Setting a New Index: Set a new column as the index. Default values in the new index that are not present in the DataFrame are assigned NaN.

```
# Setting a new index
df_set_index = df.set_index('B')
print("Output")
print("DataFrame with 'B' as Index:")
print(df_set_index)
```

```
Output
DataFrame with 'B' as Index:
   A
B
4  1
5  2
6  3
```

These examples showcase how indexing allows you to access specific elements, rows, or columns, while reindexing allows you to modify the existing DataFrame's structure or create a new DataFrame with a different order of rows and columns. Reindexing is also useful for filling in missing values or setting a new index for the DataFrame.



NOTES

4.9 FILTERING

Filtering in Pandas involves selecting specific rows or columns from a DataFrame based on certain conditions. Here are some examples illustrating different ways of filtering data in Pandas:

Filtering Rows Based on Conditions

- **Boolean Indexing:** Use Boolean conditions to filter rows based on a condition.

```
import pandas as pd
# Create a sample DataFrame
data = {'A': [1, 2, 3, 4, 5], 'B': [10, 20, 30, 40, 50]}
df = pd.DataFrame(data)
# Filtering rows where column A is greater than 2
filtered_rows = df[df['A'] > 2]
print("Output")
print("Filtered Rows where A > 2:")
print(filtered_rows)
```

```
Output
Filtered Rows where A > 2:
   A  B
2  3  30
3  4  40
4  5  50
```

- **Multiple Conditions:** Combine multiple conditions using logical operators ('&' for AND, '|' for OR).

```
# Filtering rows where column A is greater than 2 and column B is less than 40
multiple_conditions = df[(df['A'] > 2) & (df['B'] < 40)]
print("Output")
print("Filtered Rows where A > 2 and B < 40:")
print(multiple_conditions)
```

```
Output
Filtered Rows where A > 2 and B < 40:
   A  B
2  3  30
```



- Isin Method: Filter rows based on values present in a list or another DataFrame.

NOTES

```
# Filtering rows where column A values are in a specified list
filter_isin = df[df['A'].isin([2, 4])]
print("Output")
print("Filtered Rows where A is in [2, 4]:")
print(filter_isin)
```

```
Output
Filtered Rows where A is in [2, 4]:
   A  B
1  2  20
3  4  40
```

Filtering Columns

Selecting Specific Columns: Choose specific columns from the DataFrame.

```
# Selecting specific columns
selected_columns = df[['A', 'B']]
print("Output")
print("Selected Columns A and B:")
print(selected_columns)
```

```
Output
Selected Columns A and B:
   A  B
0  1  10
1  2  20
2  3  30
3  4  40
4  5  50
```



NOTES

'loc' and 'iloc' Methods: Use 'loc' for label-based indexing and 'iloc' for integer-based indexing.

```
# Filtering rows using loc based on index
loc_filter = df.loc[df['A'] > 2, ['A', 'B']]
print("Output")
print("Filtered Rows using loc method:")
print(loc_filter)
```

```
Output
Filtered Rows using loc method:
   A  B
2  3  30
3  4  40
4  5  50
```

These examples demonstrate different techniques for filtering rows and columns in Pandas DataFrames. You can use Boolean indexing, conditions, isin method, query method, and label or integer-based indexing to filter and extract specific data according to your requirements.

4.10 HANDLING MISSING DATA

Handling missing data is a crucial part of data analysis and preprocessing. In Pandas, missing data is often represented as NaN (Not a Number).

Detecting Missing Data: Use methods like 'isna()' or 'isnull()' to identify missing values in a DataFrame.

```
import pandas as pd
# Create a sample DataFrame with missing values
data = {'A': [1, 2, None, 4, 5], 'B': [10, None, 30, 40, 50]}
df = pd.DataFrame(data)
# Check for missing values
missing_values = df.isna()
print("Output")
print("DataFrame with Missing Values:")
print(df)
print("\nMissing Values:")
print(missing_values)
```



```
Output
DataFrame with Missing Values:
   A    B
0  1.0  10.0
1  2.0   NaN
2  NaN  30.0
3  4.0  40.0
4  5.0  50.0

Missing Values:
   A    B
0 False False
1 False  True
2  True False
3 False False
4 False False
```

Dropping Missing Values: Use ‘dropna ()’ to remove rows or columns containing missing values.

```
# Drop rows with missing values
df_dropped_rows = df.dropna()
# Drop columns with missing values
df_dropped_columns = df.dropna(axis=1)
print("Output")
print("DataFrame after Dropping Rows:")
print(df_dropped_rows)
print("\nDataFrame after Dropping Columns:")
print(df_dropped_columns)
```

```
Output
DataFrame after Dropping Rows:
   A    B
0  1.0  10.0
3  4.0  40.0
4  5.0  50.0

DataFrame after Dropping Columns:
Empty DataFrame
Columns: []
Index: [0, 1, 2, 3, 4]
```



NOTES

Filling Missing Values: Use `'fillna()'` to replace missing values with specified values.

```
# Fill missing values with a specific value
df_filled = df.fillna(0)
print("Output")
print("DataFrame after Filling Missing Values:")
print(df_filled)
```

```
Output
DataFrame after Filling Missing Values:
   A    B
0  1.0 10.0
1  2.0  0.0
2  0.0 30.0
3  4.0 40.0
4  5.0 50.0
```

- pad/ffill to fill values forward.
- bfill/backfill to fill values backwards.
- nearest to fill from the nearest index values.

Handling Missing Values During Operations: Use methods like `'skipna'` to handle missing values during operations.

```
# Calculate the sum while skipping missing values
sum_without_missing = df.sum(skipna=True)
print("Output")
print("Sum without Missing Values:")
print(sum_without_missing)
```

```
Output
Sum without Missing Values:
A      12.0
B     130.0
dtype: float64
```



Drop Duplicates: Use `drop_duplicates()` to remove duplicate rows, including those with missing values.

```
# Drop duplicate rows
df_no_duplicates = df.drop_duplicates()
print("Output")
print("DataFrame after Dropping Duplicates:")
print(df_no_duplicates)
```

Output

DataFrame after Dropping Duplicates:

	A	B
0	1.0	10.0
1	2.0	NaN
2	NaN	30.0
3	4.0	40.0
4	5.0	50.0

Handling missing data is a critical step in the data-cleaning process, and the choice of method depends on the nature of the data and the analysis goals. It is essential to carefully consider the impact of different strategies on the integrity and accuracy of your analysis.

4.11 HIERARCHICAL INDEXING

Hierarchical is also known as Multi-level indexing. This type of indexing helps in some quite sophisticated data analysis and manipulation, especially for working with higher dimensional data. In particular, it enables you to store and manipulate data with an arbitrary number of dimensions in lower dimensional data structures like Series (one-dimensional series) and DataFrame (two-dimensional DataFrames). It provides a way of working with high-dimensional data as well.

NOTES



NOTES

```
import pandas as pd
import numpy as np
data = pd.Series(np.random.randn(9),index=[['a', 'a', 'a', 'b', 'b', 'c', 'c', 'd', 'd'],[1, 2, 3, 1, 3, 1, 2, 2, 3]])
print("Output")
print(data)
```

Output

```
a 1 -0.385771
   2 -0.062559
   3 -0.704983
b 1  0.592031
   3 -0.828476
c 1  1.017738
   2 -1.951100
d 2 -0.859680
   3  0.228344
dtype: float64
```

In above example, first three indices 1, 2, 3 are clubbed with the name 'a', second three indices 1, 3, 1 are clubbed with name 'b', next two indices 1, 2 are clubbed with the name 'c' and last two indices 2, 3 are clubbed with name 'd'.

How to print multi-index? To display the multi-index, we use the index attribute with the accessible variable (where the series of data is stored). For the above example, we are displaying the multi-index of the pandas data series.

```
import pandas as pd
import numpy as np
data = pd.Series(np.random.randn(9),index=[['a', 'a', 'a', 'b', 'b', 'c', 'c', 'd', 'd'],[1, 2, 3, 1, 3, 1, 2, 2, 3]])
print("Output")
print(data.index)
```

Output

```
MultiIndex([(('a', 1),
              ('a', 2),
              ('a', 3),
              ('b', 1),
              ('b', 3),
              ('c', 1),
              ('c', 2),
              ('d', 2),
              ('d', 3)),
            ])
```

We can also access the subset of multi-indexed data series, which is called partial indexing.



Pandas Power Play: Mastering Data Manipulation

NOTES

```
import pandas as pd
import numpy as np
data = pd.Series(np.random.randn(9), index=[['a', 'a', 'a', 'b', 'b', 'c', 'c', 'd', 'd'], [1, 2, 3, 1, 3, 1, 2, 2, 3]])
print("Output")
print(data['a'])
print(data['b':'c'])
print(data['d'])
```

Output

```
1    0.393424
2   -0.802687
3   -0.068497
dtype: float64
b 1   -0.127047
   3   -0.731212
c 1   -0.041387
   2   -0.771163
dtype: float64
2    1.828859
3   -0.124793
dtype: float64
```

Hierarchical or Multilevel indexing also helps in reshaping data and group-based operations like creating a pivot table, in which you can rearrange the data into a DataFrame using the `unstack` method. Note that you will study the pivot table in the contents of Lesson 5 in-depth.

```
import pandas as pd
import numpy as np
data = pd.Series(np.random.randn(9), index=[['a', 'a', 'a', 'b', 'b', 'c', 'c', 'd', 'd'], [1, 2, 3, 1, 3, 1, 2, 2, 3]])
print("Output")
print(data.unstack())
```

Output

	1	2	3
a	-0.194264	0.036841	-0.349127
b	-0.180680	NaN	0.155820
c	0.462327	1.318264	NaN
d	NaN	-0.661732	-1.291545

The invert operation of `unstack` is called a `stack`; this operation is again used in making hierarchical indexing. The `stack()` method is always applied with the `unstack()` method only.

```
import pandas as pd
import numpy as np
data = pd.Series(np.random.randn(9), index=[['a', 'a', 'a', 'b', 'b', 'c', 'c', 'd', 'd'], [1, 2, 3, 1, 3, 1, 2, 2, 3]])
print("Output")
print(data.unstack().stack())
```

Output

```
a 1   -0.908235
   2   -0.890774
   3   -1.009713
b 1    0.690932
   3   -0.876532
c 1   -0.563500
   2   -1.776505
d 2    0.863226
   3    0.975929
dtype: float64
```

Self-Instructional
Material

119



NOTES

4.12 DATA WRANGLING

Data Wrangling serves as a fundamental aspect in the realm of Data Science and Data Analysis. For effective data manipulation and transformation, Python's Pandas framework is the go-to tool. Pandas, an open-source library, is purpose-built for Data Analysis and Data Science tasks. Its functionalities include data sorting, filtration, grouping, and more.

Data wrangling in Python encompasses various essential tasks:

- **Data Exploration:** The process involves studying, analysing, and gaining insights into the data through Visualizations.
- **Dealing with Missing Values:** Many datasets contain missing values (NaN), which need careful handling. Techniques such as replacing them with mean, mode, the most frequent value, or dropping rows with NaN values are commonly employed.
- **Reshaping Data:** Data is manipulated to meet specific requirements. This involves adding new data or modifying existing data to suit the analysis objectives.
- **Filtering Data:** Datasets often include unwanted rows or columns. Data wrangling includes removing or filtering out these unwanted elements to focus on relevant information.
- **Other Operations:** After applying the functionalities to the raw dataset, we obtain an efficient and tailored dataset. This curated dataset is then ready for various purposes, such as data analysis, machine learning, data visualization, model training, and more.

Data Wrangling with Pandas is an indispensable process that ensures data is structured, cleaned, and prepared for downstream analysis and modelling activities. We have already discussed these operations in the lesson, so we will focus on merging data in this section.

Merging DataFrames is a common operation in data analysis when you want to combine data from two or more datasets based on a common column or index. In the discussion of this section, you will study a few operations that are



Pandas Power Play: Mastering Data Manipulation

frequently used for combining and merging the datasets.

These are the few operations used for combining and merging of the datasets:

Pandas.merge : This method is used to connect rows (data available in a horizontal manner) in DataFrames on behalf of their keys. This operation is very similar to the join operation in SQL.

Pandas.concat : This method is also used to merge the DataFrame objects along the axis.

Combine_first : This method enables splicing together overlapping data to fill in missing values in one object with values from another.

```
#Apply the pandas.merge method over two data frames.
import pandas as pd
import numpy as np
df1 = pd.DataFrame({'key': ['b', 'b', 'a', 'c', 'a', 'a', 'b'], 'data1': range(7)})
df2 = pd.DataFrame({'key': ['a', 'b', 'd'], 'data2': range(3)})
print("Output")
print(df1)
print(df2)
print(pd.merge(df1, df2))
```

Output

	key	data1
0	b	0
1	b	1
2	a	2
3	c	3
4	a	4
5	a	5
6	b	6

	key	data2
0	a	0
1	b	1
2	d	2

	key	data1	data2
0	b	0	1
1	b	1	1
2	b	6	1
3	a	2	0
4	a	4	0
5	a	5	0

When we apply the merge operation over the DataFrame objects, sometimes it is difficult to understand how the operation is applied. So, for the readability of the Python code, we can use the 'on' attribute with the value 'key', which

NOTES



NOTES

indicates the merge operation performed over the key. This attribute is used when we have the same column names in both the DataFrames. For this, we can write the merge statement like below:

```
pd.merge(df1, df2, on = 'key')
```

In the specific scenario of DataFrames, if there are different column names, then it is better to use the attributes **left_on** and **right_on** to represent the DataFrames, respectively.

```
import pandas as pd
import numpy as np
df1 = pd.DataFrame({'lkey': ['b', 'b', 'a', 'c', 'a', 'a', 'b'], 'data1': range(7)})
df2 = pd.DataFrame({'rkey': ['a', 'b', 'd'], 'data2': range(3)})
print("Output")
print(pd.merge(df1, df2, left_on='lkey', right_on='rkey'))
```

```
Output
  lkey  data1 rkey  data2
0    b      0    b      1
1    b      1    b      1
2    b      6    b      1
3    a      2    a      0
4    a      4    a      0
5    a      5    a      0
```

Joins are of two types: Inner join and Outer join.

By default, the merge operation belongs to the inner join operation. The above example is the case of common keys found in both DataFrames tables. Similarly, we can also apply the left and right outer joins (which are the sub-types of outer join) over it. We use the 'how' attribute to specify the join types in the merge operation of DataFrame objects.

Outer Join

```
import pandas as pd
import numpy as np
df1 = pd.DataFrame({'key': ['b', 'b', 'a', 'c', 'a', 'a', 'b'], 'data1': range(7)})
df2 = pd.DataFrame({'key': ['a', 'b', 'd'], 'data2': range(3)})
print("Output")
print(pd.merge(df1, df2, how='outer'))
```



Pandas Power Play: Mastering Data Manipulation

NOTES

```
Output
  key  data1  data2
0   b    0.0    1.0
1   b    1.0    1.0
2   b    6.0    1.0
3   a    2.0    0.0
4   a    4.0    0.0
5   a    5.0    0.0
6   c    3.0    NaN
7   d    NaN    2.0
```

Different join types with how argument

Options	Use
Inner	Use only the key combinations observed in both table
Left	Use all key combinations found in the left table
Right	Use all key combinations found in the right table
Outer	Use all key combinations observed in both tables together

Left Join

```
import pandas as pd
import numpy as np
df1 = pd.DataFrame({'key': ['b', 'b', 'a', 'c', 'a', 'a', 'b'], 'data1': range(7)})
df2 = pd.DataFrame({'key': ['a', 'b', 'd'], 'data2': range(3)})
print("Output")
print(pd.merge(df1, df2, on='key', how='left'))
```

```
Output
  key  data1  data2
0   b     0    1.0
1   b     1    1.0
2   a     2    0.0
3   c     3    NaN
4   a     4    0.0
5   a     5    0.0
6   b     6    1.0
```



NOTES

Inner Join

```
import pandas as pd
import numpy as np
df1 = pd.DataFrame({'key': ['b', 'b', 'a', 'c', 'a', 'a', 'b'], 'data1': range(7)})
df2 = pd.DataFrame({'key': ['a', 'b', 'd'], 'data2': range(3)})
print("Output")
print(pd.merge(df1, df2, on='key', how='inner'))
```

Output

	key	data1	data2
0	b	0	1
1	b	1	1
2	b	6	1
3	a	2	0
4	a	4	0
5	a	5	0

4.13 SUMMARY

In this lesson, we discussed the Pandas library and the basic panda's mathematical operations. It introduces two main tools in Pandas: Series (for one-dimensional data) and DataFrame (for two-dimensional, tabular data). Series can be thought of as a list with labels, and DataFrame as a table where you can easily organise and analyse your data. This lesson also covers practical skills like loading different types of data into Pandas, doing math and statistics with your data, and dealing with missing information. Towards the end, the lesson introduces data wrangling, which is like getting your data into the perfect shape for analysis.

4.14 GLOSSARY

- **Series:** It is a one-dimensional labelled array in Pandas, like a column in a spreadsheet or a single list in Python.
- **DataFrame:** It is a two-dimensional labelled data structure in Pandas, resembling a table or spreadsheet, where data is organised in rows and columns.



Pandas Power Play: Mastering Data Manipulation

- **Index:** It is an object in Pandas that provides labels to the data, facilitating quick and efficient data retrieval and alignment.
- **Data Wrangling:** It is the process of cleaning, transforming, merging, and reshaping raw data to make it suitable for analysis, often using tools like Pandas.
- **Binning:** It means grouping continuous data into discrete intervals or bins for better analysis or Visualization.
- **Indexing:** It means selecting specific rows or columns in a DataFrame based on labels or positions.
- **Reindexing:** It means creating a new index for a DataFrame or altering the existing index to match a specified set of labels.
- **Attribute:** The properties of any object are called attributes; here, the properties which we are performing over Pandas objects represent the attribute.
- **Transposing:** It is the operation of a matrix of nd-array in which we exchange the position of rows and columns elements with each other.

NOTES

4.15 ANSWERS TO IN-TEXT QUESTIONS

1. A one-dimensional labelled array
2. `pd.Series(my_list)`
3. It allows for quick data retrieval and alignment
4. `df['A'].mean()`
5. Provides basic statistics for each column
6. Computes the correlation matrix between numeric columns

4.16 SELF-ASSESSMENT QUESTIONS

1. Explain the data series and DataFrames objects with suitable examples.
2. Define the following attributes in the context of pandas:

*Self-Instructional
Material*

125



NOTES

- Data type
 - Shape
 - Reshape
 - Dimension
3. Differentiate between NumPy array and pandas.
 4. What do you mean by indexing? Discuss hierarchical indexing and illustrate with examples.
 5. Write a Python code to create a pandas series object and display the shape, dimension, and data type.
 6. Write a Python to create a pandas DataFrames object and display the shape, dimension, and data type.
 7. Write the Python code to generate a stacked and unstacked pivot table.
 8. Write the Python code to apply the merging of any two datasets.

4.17 REFERENCES

- McKinney, W. (2018). Python for Data Analysis: Data Wrangling with Pandas, NumPy and IPython (2nd ed.). O'Reilly Media.
- Molin, S. (2019). Hands-On Data Analysis with Pandas. Packt Publishing.
- Thareja, R. (2017). Python Programming using problem-solving approach.

4.18 SUGGESTED READINGS

- Data Visualization with Python, Mario Dobler, Tim Großmann, Packt Publishing Limited, 2019.
- Data Visualization in Python, Daniel Nelson, StackAbuse.com, 2020.
- Mastering Python Data Visualization, Kirthi Raman, O'Reilly, 2015.

UNIT IV:

LESSON 5: Plotting Perfection: Mastering Plotting & Visualization



LESSON 5

PLOTTING PERFECTION: MASTERING PLOTTING & VISUALIZATION

NOTES

Lavkush Gupta
Assistant Professor
Shyama Prasad Mukherji College (W)
University of Delhi
lavkush.mca16.du@gmail.com

Structure

- 5.1 Learning Objectives
- 5.2 Introduction
- 5.3 Matplotlib
 - 5.3.1 Pyplot
 - 5.3.2 Concept of figure, plot, and subplot
- 5.4 Plotting Functions with Examples
 - 5.4.1 Basic Plot Functions
 - 5.4.2 Colours, Markers, and Line Styles
 - 5.4.3 Label and Legend
 - 5.4.4 Saving a Plot
- 5.5 Plotting Functions in Pandas
- 5.6 Summary
- 5.7 Glossary
- 5.8 Answers to In-text Questions
- 5.9 Self-Assessment Questions
- 5.10 References
- 5.11 Suggested Readings

5.1 LEARNING OBJECTIVES

By the end of this lesson, learners should be equipped with the knowledge and skills needed to create, customise, and interpret a variety of plots using both Matplotlib and Pandas, enhancing their ability to communicate insights derived from data effectively.

*Self-Instructional
Material*

129



NOTES

- Understand the Basics of Matplotlib.
- Comprehend Plotting Functions in Pandas.
- Apply Data Visualization in Real-world Scenarios.
- Interpret and draw insights from visualized data, recognising patterns and trends

5.2 INTRODUCTION

In this lesson, we delve into the art and science of data visualization using the powerful tools of Matplotlib and Pandas. Visualization plays a pivotal role in data analysis, aiding in the exploration, interpretation, and communication of complex datasets. Matplotlib, a widely used plotting library in Python, serves as our primary tool for crafting visually engaging figures and subplots. We embark on a journey to understand the intricacies of Matplotlib, exploring its diverse capabilities, such as customising colours, line styles, and annotations. Learners will grasp the fundamentals of constructing informative plots that not only convey data trends but also cater to the aesthetic considerations crucial for effective communication.

Moving beyond Matplotlib, this lesson introduces the plotting functions available in Pandas, offering a seamless integration of visualization into data manipulation workflows. From basic line plots to intricate heatmaps, we navigate through the spectrum of Pandas plotting functions. The emphasis is not only on creating diverse visualizations but also on understanding when to employ each type of plot for optimal representation of different data scenarios. By the end of this lesson, learners will be equipped with the skills to not only generate compelling plots but also to discern the most suitable visualization techniques for their specific analytical objectives. Whether it is depicting trends over time, comparing distributions, or showcasing relationships between variables, the mastery of Matplotlib and Pandas plotting functions empowers data enthusiasts to unlock meaningful insights from their datasets.



5.3 MATPLOTLIB

Matplotlib stands as an extensive library designed to facilitate the creation of static, animated, and interactive visualizations within the Python programming language. Renowned for its versatility, Matplotlib excels in simplifying straightforward tasks while simultaneously empowering users to tackle more intricate challenges. Its capabilities extend beyond conventional plotting, enabling the generation of publication-quality plots with ease. Matplotlib empowers users to craft interactive figures that facilitate zooming, panning, and real-time updates, enhancing the viewer's engagement with the data. This library also grants users fine-grained control over visual style and layout, ensuring that the aesthetics align with their specific preferences.

Moreover, Matplotlib supports seamless exportation to various file formats and integration into popular environments like JupyterLab and Graphical User Interfaces. Its compatibility with a rich assortment of third-party packages further amplifies its utility, allowing users to leverage an extensive ecosystem built upon the robust foundations of Matplotlib. Matplotlib serves as a powerful and versatile library for data visualization in Python, and understanding the intricacies of figures and subplots is crucial for constructing meaningful visual representations.

5.3.1 PyPlot

PyPlot is a submodule of the Matplotlib library, specifically designed to provide a simplified interface for creating plots and visualizations in Python. Widely used in scientific computing, data analysis, and various fields of research, PyPlot enables users to generate high-quality charts and graphs with relative ease. It serves as a convenient tool for those who prefer a concise and user-friendly approach to data visualization without sacrificing the robust capabilities offered by Matplotlib.

At its core, PyPlot provides a collection of functions that closely resemble those in MATLAB, allowing users to create figures, axes, and plots with minimal boilerplate code. The submodule abstracts away some of the complexities of Matplotlib, making it particularly accessible for users who are new to the library. With PyPlot, one can effortlessly customise the appearance of plots, add labels,



NOTES

legends, and titles, and manipulate various plot elements, all within a single interface. Whether used for quick exploratory data analysis or the creation of publication-ready visuals, PyPlot serves as a valuable tool for harnessing Matplotlib's power while streamlining the plotting process in Python.

5.3.2 Concept of figure, plot, and subplot

A figure is an overall container, a plot is a specific graphical representation of data within that figure, and a subplot is a smaller plot within a figure that allows you to organise multiple plots. Understanding these concepts helps you create and customise visualizations effectively in Matplotlib.

Figure

- A Figure in Matplotlib is the top-level container that represents the entire window or page where your plots are drawn.
- When you create a new plot using `plt.plot()` or a set of subplots using `plt.subplots()`, you are creating them within a figure.
- You can explicitly create a new figure using `plt.figure()`.

Plot

- A “plot” refers to the graphical representation of data within a figure. It could be a line plot, scatter plot, or bar plot.
- Functions like `plt.plot()`, `plt.scatter()`, or `plt.bar()` are used to create specific types of plots within a figure.

Subplot

- A “subplot” is a smaller plot that exists within a single figure. It allows you to organise multiple plots in a grid-like fashion.
- The `plt.subplots()` function is commonly used to create subplots. It returns a figure and an array of subplot axes.



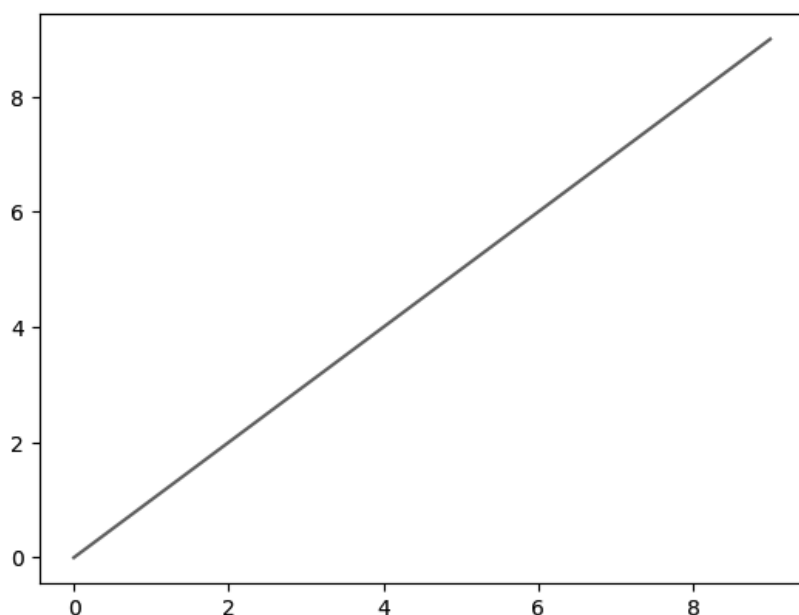
5.4 PLOTTING FUNCTIONS WITH EXAMPLES

5.4.1 Basic Plot Functions

plot() function: This function is used to plot the points in graphical form. It comes after initialising the data points in the variable. It draws the lines from one point to another point over the axes. Sometimes, it takes two arguments to denote the x-axis and y-axis, respectively.

show() function: This function is used to display the graph on the user's output screen.

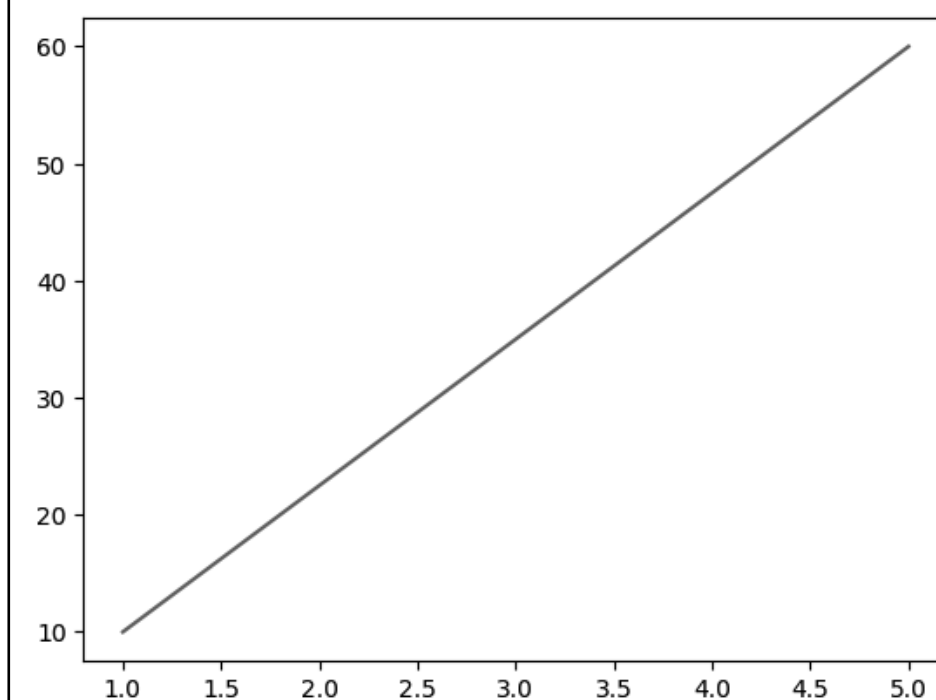
```
#Simple line plot
import numpy as np
#Load the plotting library
import matplotlib.pyplot as plt #plt is alias name for matplotlib
d = np.arange(10)
#Function to plot
plt.plot(d)
#Function to show the plot
plt.show()
```





NOTES

```
# x-axis values
x_coord = [1,5]
# Y-axis values
y_coord = [10,60]
# Function to plot
plt.plot(x_coord,y_coord)
#Function to show the plot
plt.show()
```



5.4.2 Colours, Markers, and Line Styles

The plot() function in the Matplotlib library offers parameters for customising colours, line styles, and markers. These parameters allow explicit control over the appearance of the plotted data. The colour, linestyle, and marker parameters enable users to tailor the visual representation of the plot to their preferences.

Colour Parameter: The colour parameter is used to specify the colour of the line or marker. It accepts various colour notations, such as named colours ('red',



Plotting Perfection: Mastering Plotting & Visualization

'blue'), hexadecimal RGB values ('#FF5733'), or RGB tuples ((1.0, 0.34, 0.20)).

```
plt.plot(x, y, colour='green', label='Green Line')
```

Markers Parameter: The marker parameter allows the selection of different marker styles to represent data points. Common markers include '*' for a star, 'o' for a circle, and 'd' for a diamond shape.

```
plt.plot(x, y, marker='o', label='Circle Markers')
```

Marker Size (ms) Parameter: The ms parameter, short for marker size, controls the size of the markers on the plot. Numeric values can be assigned to adjust the marker size according to specific requirements.

```
plt.plot(x, y, marker='*', ms=8, label='Star Markers  
(Size: 8)')
```

Line Style Parameter: The linestyle parameter defines the style of the line connecting the data points. Common line styles include '-', '--', and ':' for solid, dashed, and dotted lines, respectively.

```
plt.plot(x, y, linestyle='--', label='Dashed Line')
```

5.4.3 Label and Legend

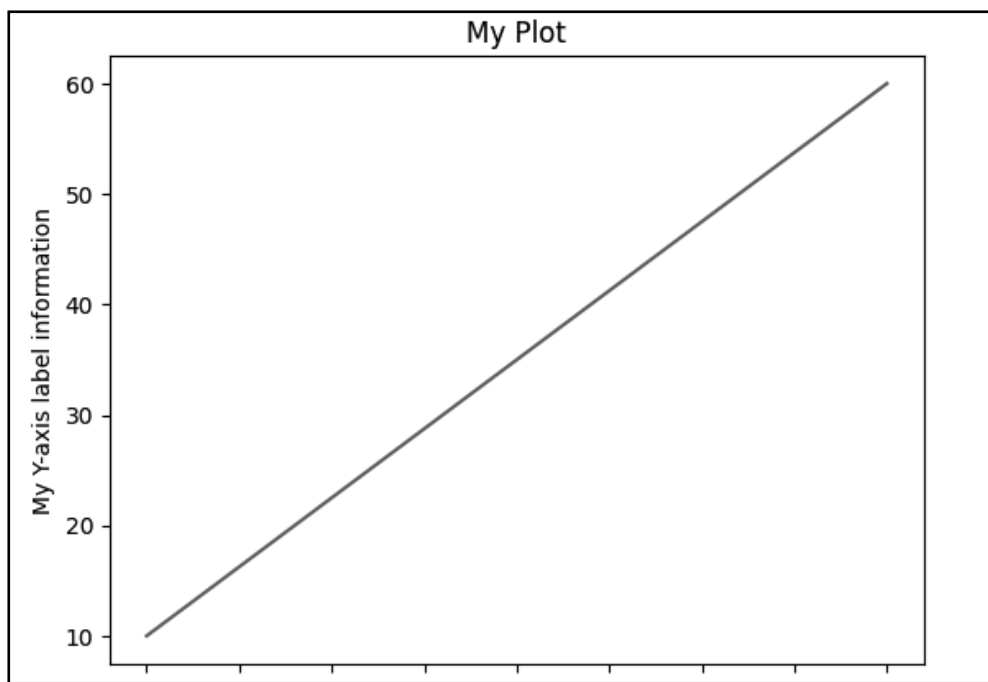
Labels: In data visualization, we use various decorative features to represent the data with more clarity and detailed information that enhances the Visualization. We use the xlabel(), ylabel() function to add the labels at x and y axes respectively. We can add a plot title using the title() function.

```
import matplotlib.pyplot as plt
import numpy as np
# x-axis values
x_coord = [1,5]
plt.xlabel("My X-axis label information")
# Y-axis values
y_coord = [10,60]
# Function to plot
plt.title('My Plot') # add title
plt.plot(x_coord,y_coord)
plt.ylabel("My Y-axis label information")
#Function to show the plot
plt.show()
```

NOTES



NOTES



NOTE: If you want to shift the title location to the left or right, then use the `loc = 'left'` or `loc = 'right'`. Also, if you want to give a common title name for group of subplots then use the `suptitle()` function.

Legends: Generally, we plot single-line data and do visualization. Then, there are no more requirements to distinguish the plotted element (the lines, curves, scatter, spike). However, when we have more than one plotted element, i.e. the group of data is plotted at a single interface. There is a need to distinguish the plotted data elements from one another for analysis and visualization purposes. This is done by using the symbolic notation, known as legends.

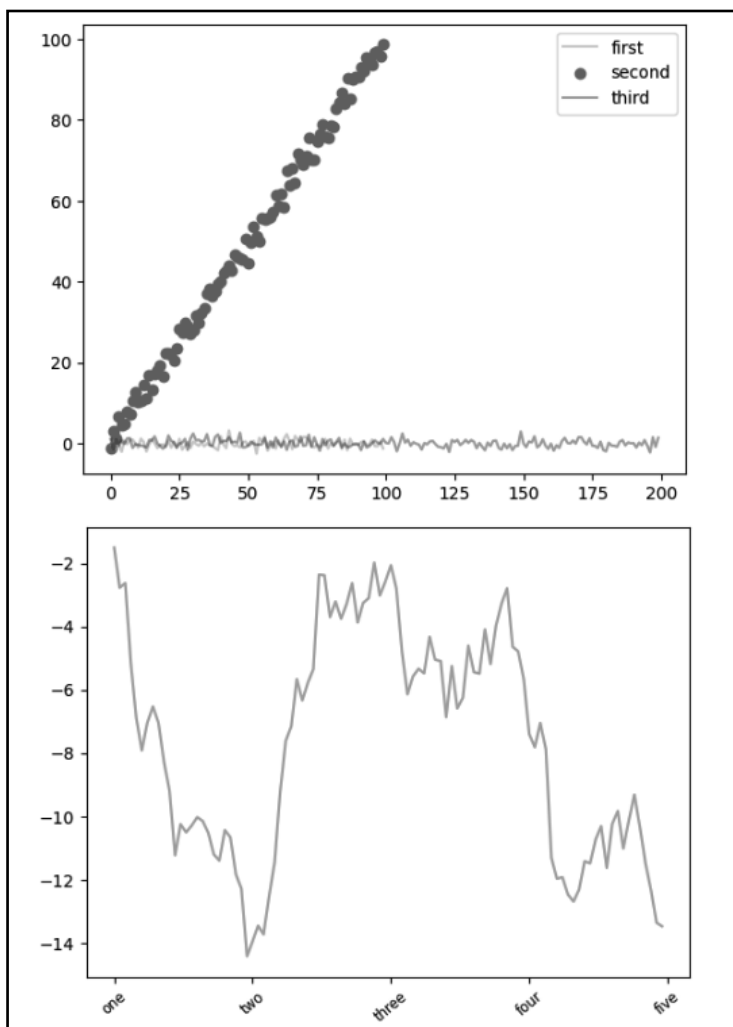


Plotting Perfection: Mastering Plotting & Visualization

NOTES

```
import matplotlib.pyplot as plt
import numpy as np
p1 = plt.subplot(1, 1, 1)
p1.plot(np.random.randn(100), color='c', alpha=0.5, label = "first")
p1.scatter(np.arange(100), np.arange(100) + 2 * np.random.randn(100), label = "second")
p1.plot(np.random.randn(200), color='r', alpha=0.5, label = "third")
p1.legend()
plt.show()

import matplotlib.pyplot as plt
import numpy as np
data = plt.subplot(1,1,1)
data.plot(np.random.randn(100).cumsum(), color = 'c')
ticks = data.set_xticks([0, 25, 50, 75, 100])
labels = data.set_xticklabels(['one', 'two', 'three', 'four', 'five'], rotation=40, fontsize='small')
plt.show()
```



Self-Instructional
Material

137

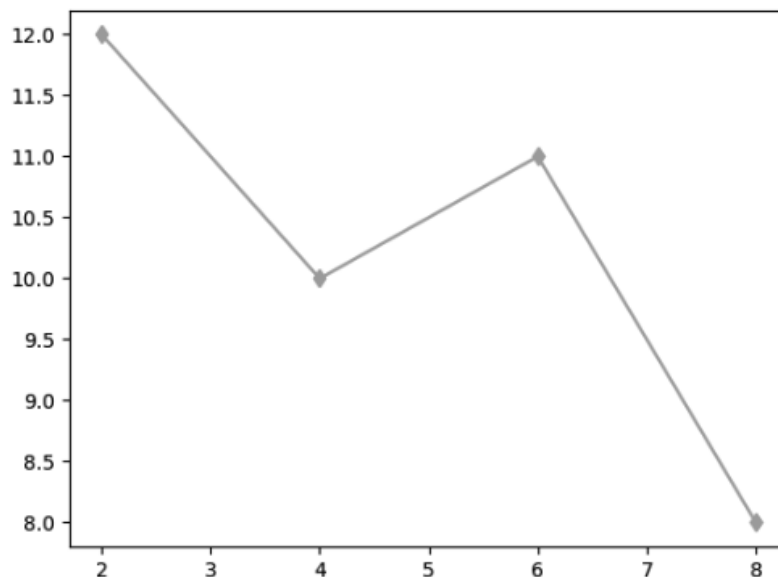


NOTES

5.4.4 Saving a Plot

After creating the plot, if you want to save the plotted output of your data, then you can apply the `savefig()` function. This function will be executed using the `matplotlib` module, which helps you save the figure according to your choice. The figure below shows how to save the plotted figure in `svg` (scalable vector graphics) image format. Similarly, you can save the figure in `pdf`, `jpg` or `png` formats.

```
import matplotlib.pyplot as plt
import pandas as pd
x=np.array([2,4,6,8])
y=np.array([12,10,11,8])
plt.plot(x,y,marker='d', color='c')
#plt.show()
plt.savefig('myplot.svg')
```



NOTE: The figure (plotted image file) will be saved at the same path or folder of your system where your Python source code file is available. Here, I have opened the saved file; it resides on the same path as my source code.



IN-TEXT QUESTIONS

1. What is Matplotlib?
2. Which module in Matplotlib is commonly used for creating basic plots?
3. What does the plot function in Matplotlib do?
4. How can you create multiple subplots in a single figure in Matplotlib?
5. Which parameter is used to customise the colour of a plot in Matplotlib?
6. What does the xlabel function in Matplotlib do?
7. How can you save a Matplotlib plot to a file?
8. Which type of plot is best suited for visualising the distribution of a continuous variable?
9. What does the tight_layout() function in Matplotlib do?

5.5 PLOTTING FUNCTIONS IN PANDAS

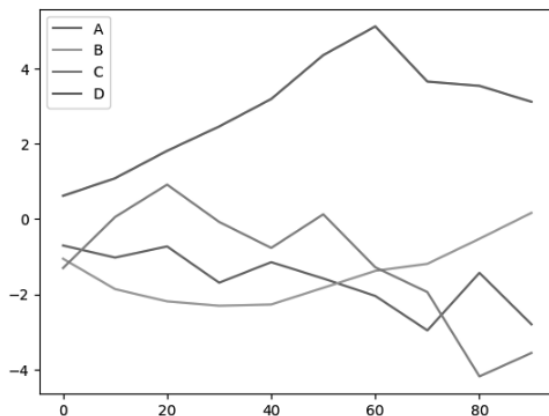
Pandas is the Python library used in analysing the data. It uses the concepts of DataFrames. It has various functionalities and offers solutions for data cleaning, analysis, exploration, and editing. Apart from data manipulation with Pandas, you can visually represent your findings through expressive plots. Whether you are exploring data for insights or presenting results, the combination of Pandas and Matplotlib provides a powerful toolkit for effective data visualization. Pandas simplifies the process of working with structured data, providing a high-level interface to organise, filter, and analyse datasets. When coupled with Matplotlib, Pandas allows for efficient plotting without the need for extensive code. We discuss various types of plots, for example:

Line plots: The group of lines can be plotted using pandas easily.



NOTES

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
s = pd.Series(np.random.randn(10).cumsum(), index=np.arange(0, 100, 10))
s = pd.DataFrame(np.random.randn(10, 4).cumsum(0), columns=['A', 'B', 'C', 'D'], index=np.arange(0, 100, 10))
s.plot()
plt.show()
```



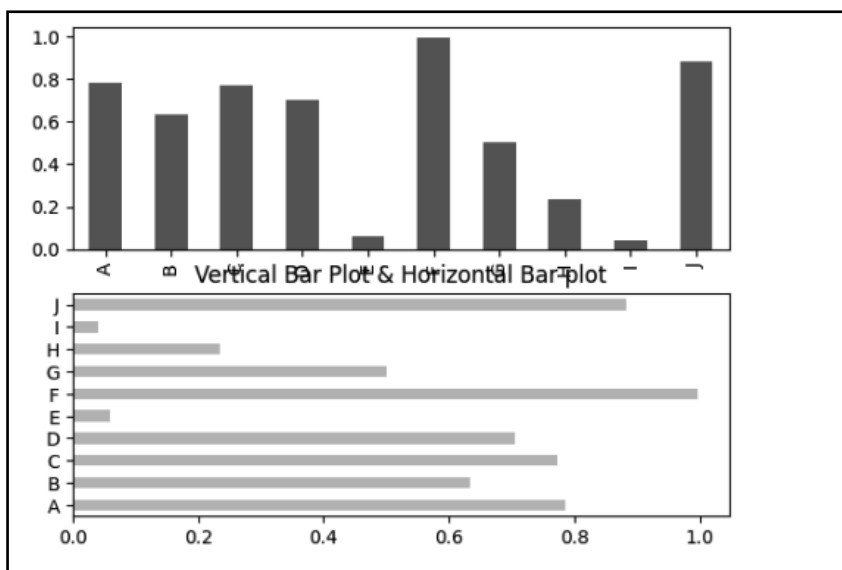
Bar plot: The bar plot key elements are rectangular bars with their length and height, which tells about the data representation. The bar plot is also called as bar chart or bar graph. The bar chart can be made in a horizontal or vertical manner. The `plot.bar()` function is used for vertical bar charts and `plots.barh()` is used for horizontal bar charts. In plotting a bar chart, we can use pandas with matplotlib library (pyplot submodule of matplotlib) or numpy with matplotlib library (pyplot submodule of matplotlib). With pandas module, we use either DataFrame or series for indexing of bar plot.

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
fig, axes = plt.subplots(2,1)
#add title
plt.title('Vertical Bar Plot & Horizontal Bar plot')
data = pd.Series(np.random.rand(10), index=list('ABCDEFGHIJ'))
#Vertical Bar plot
data.plot.bar(ax=axes[0], color='b', alpha=0.7)
#Horizontal Bar plot
data.plot.barh(ax=axes[1], color='c', alpha=0.7)
#Display the Bar plot
plt.show()
```

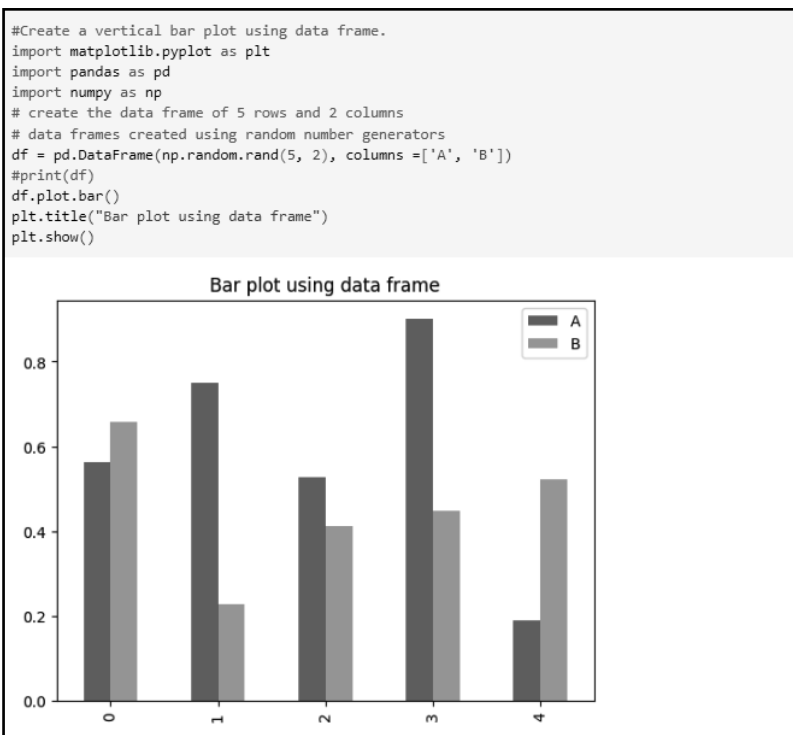


Plotting Perfection: Mastering Plotting & Visualization

NOTES



Barplot using DataFrame: First, we use the `DataFrame()` function to create the DataFrames, then plot the bar chart using `dataframe.plot.bar()` function.



Self-Instructional
Material

141

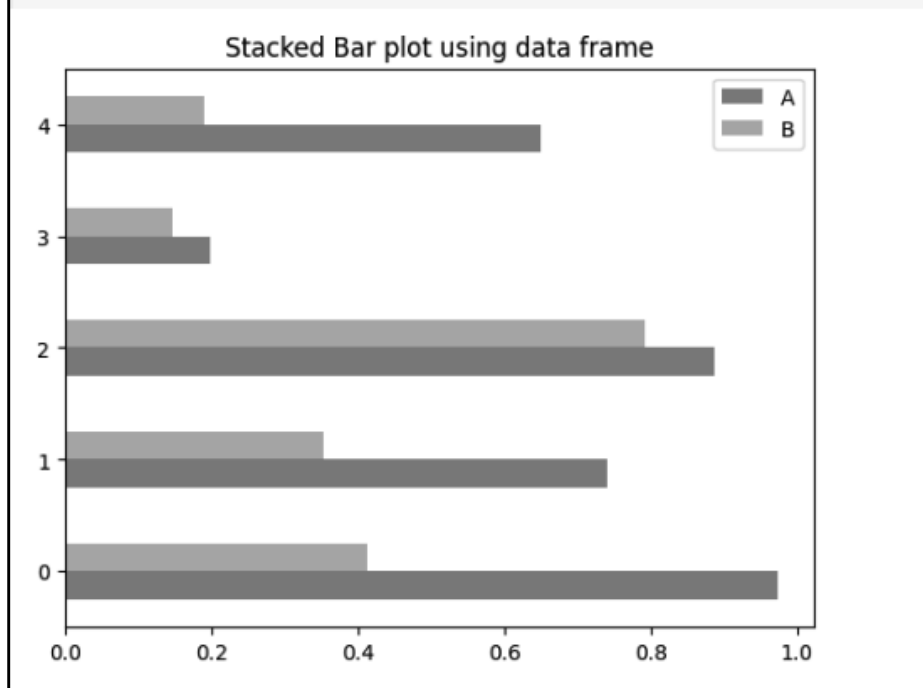


NOTES

NOTE: We can view the DataFrame values from which the bar plots are drawn using the statement: `print(dataframe)`.

Horizontal bar plot: The horizontal barplot is also known as the stacked bar plot.

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
# Create the data frame of 5 rows and 2 columns using random numbers
df = pd.DataFrame(np.random.rand(5, 2), columns=['A', 'B'])
df.plot.barh(alpha = 0.8)
plt.title("Stacked Bar plot using data frame")
plt.show()
```



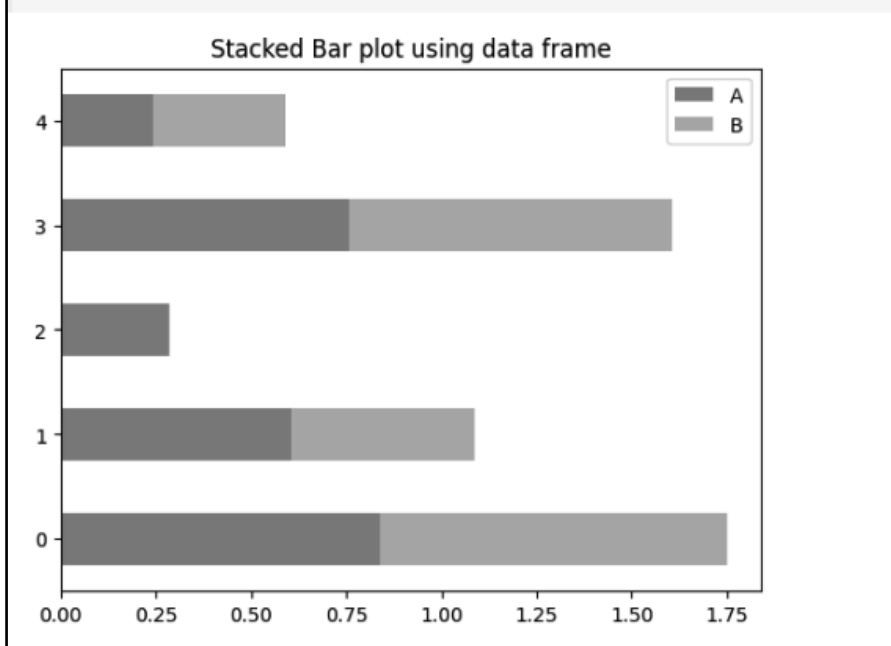
NOTE – We can use a stacked attribute with a `TRUE` boolean value to represent the stacked horizontal bar plot. For the stacked vertical bar plot, we can use the `df.plot.bar()` function.



Plotting Perfection: Mastering Plotting & Visualization

NOTES

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
# create the data frame of 5 rows and 2 columns using random numbers
df = pd.DataFrame(np.random.rand(5, 2), columns=['A', 'B'])
df.plot.barh(stacked=True, alpha=0.8)
plt.title("Stacked Bar plot using data frame")
plt.show()
```

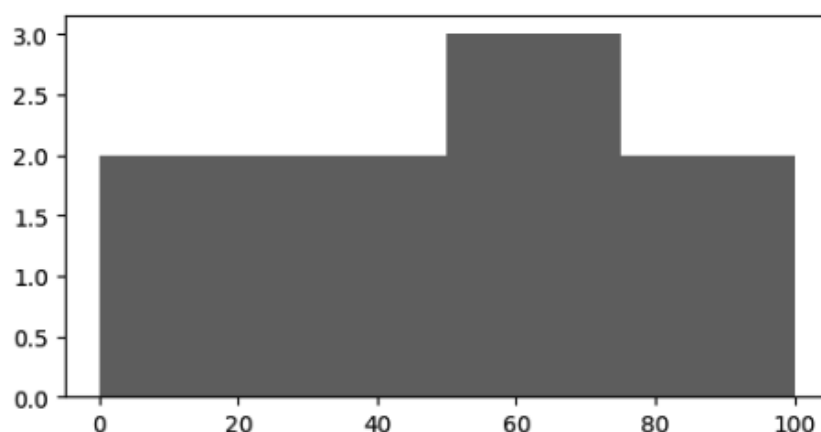


Histogram: Like a bar plot, the histogram is also a plot which represents the group of data points into a specific range; these ranges are called bins. Creating a histogram can provide a visual representation of data distribution with a large set of data. In the 2 D histogram, the horizontal axis (X-axis) represents the bins, and the vertical axis (Y-axis) represents the number of frequent occurrences of data, i.e. frequency. We can use the function `plot.hist()` to draw the histogram.



NOTES

```
import matplotlib.pyplot as plt
import numpy as np
# Creating dataset
a = np.array([10,20,30,40,50,60,70,80,90])
# Creating histogram
fig, ax = plt.subplots(figsize=(6, 3))
ax.hist(a, bins = [0, 25, 50, 75, 100])
# Show plot
plt.show()
```



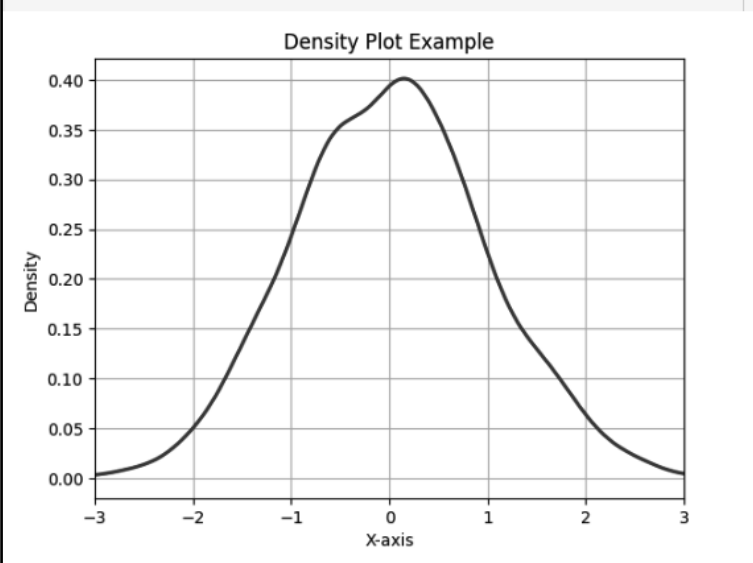
Density plot: Density Plot is the continuous, simple and smoothed version of the histogram estimated from the observed data. It is estimated through Kernel, which is a simpler distribution like the normal probability distribution. That is why the density plots are also called Kernel Density Estimation (KDE) plots. In this method, a Kernel (continuous curve) is drawn at every individual data point, and then all these curves are added together to make a single smoothed density estimation. Histogram fails when we want to compare the data distribution of a single variable over multiple categories. At that time, a density plot is useful for visualising the data.



Plotting Perfection: Mastering Plotting & Visualization

NOTES

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
# Create a sample DataFrame with a column of random data
np.random.seed(42)
data = pd.DataFrame({'RandomData': np.random.normal(size=1000)})
# Plotting a density plot
data['RandomData'].plot(kind='density', xlim=(-3, 3), color='blue', linewidth=2)
# Customize the plot
plt.title('Density Plot Example')
plt.xlabel('X-axis')
plt.ylabel('Density')
plt.grid(True)
# Show the plot
plt.show()
```

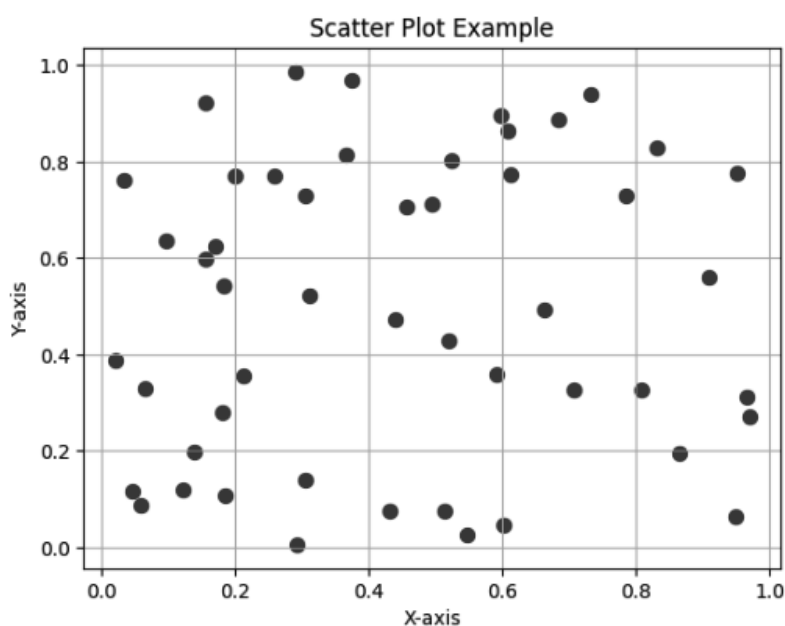


Scatter Plot: A scatter plot is used to visualise the relationship between two continuous variables. In Pandas, you can create a scatter plot using the 'plot' function with 'kind='scatter'.



NOTES

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
# Create a sample DataFrame
np.random.seed(42)
data = pd.DataFrame({
    'X': np.random.rand(50),
    'Y': np.random.rand(50)
})
# Plotting a scatter plot
data.plot(kind='scatter', x='X', y='Y', color='blue', marker='o', s=50)
# Customize the plot
plt.title('Scatter Plot Example')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.grid(True)
# Show the plot
plt.show()
```



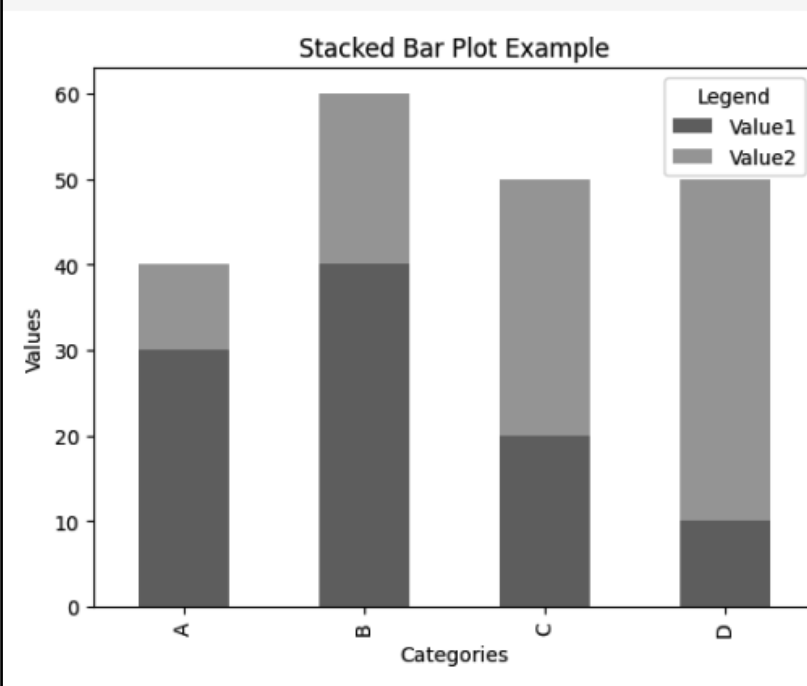
Stacked Bar Plot: A stacked bar plot is useful to show the composition of multiple categories. In Pandas, you can use the `plot` function with `kind='bar'` and set `stacked=True`.



Plotting Perfection: Mastering Plotting & Visualization

NOTES

```
import pandas as pd
import matplotlib.pyplot as plt
# Create a sample DataFrame
data = pd.DataFrame({
    'Category': ['A', 'B', 'C', 'D'],
    'Value1': [30, 40, 20, 10],
    'Value2': [10, 20, 30, 40]
})
# Plotting a stacked bar plot
data.set_index('Category').plot(kind='bar', stacked=True)
# Customize the plot
plt.title('Stacked Bar Plot Example')
plt.xlabel('Categories')
plt.ylabel('Values')
plt.legend(title='Legend')
# Show the plot
plt.show()
```



Heatmap: A heatmap is a graphical representation of data where values are represented as colours. In Pandas, you can create a heatmap using the 'plot' function with `kind='heatmap'`.



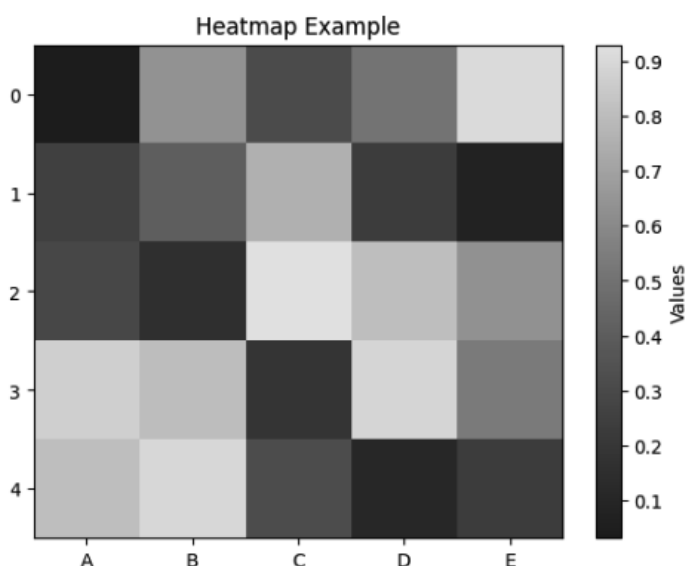
NOTES

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Create a sample DataFrame
data = pd.DataFrame(np.random.rand(5, 5), columns=['A', 'B', 'C', 'D', 'E'])

# Plotting a heatmap
plt.imshow(data, cmap='viridis', interpolation='none', aspect='auto')

# Customize the plot
plt.title('Heatmap Example')
plt.xticks(range(len(data.columns)), data.columns)
plt.yticks(range(len(data)), data.index)
plt.colorbar(label='Values')
# Show the plot
plt.show()
```



IN-TEXT QUESTIONS

10. Which Pandas plotting function is used to create a line plot?
11. How do you create a histogram in Pandas using the plot function?
12. To create a scatter plot in Pandas, which parameters can be used in the plot function?
13. What does stacked=True achieve when creating a bar plot with Pandas?



5.6 SUMMARY

In this lesson, we explored the powerful capabilities of Matplotlib for data visualization, covering the creation of figures and subplots, customisation of plots through markings, colours, and line styles, as well as the addition of labels and legends for enhanced clarity. Matplotlib's flexibility and extensive functionality enable users to convey insights effectively through diverse visualizations. Subsequently, we delved into Pandas' seamless integration with Matplotlib, demonstrating how to leverage the 'plot' function for lines, bars, scatter plots, histograms, stacked bars, and heatmaps. This integration simplifies the plotting process, particularly when working with Pandas DataFrames. Whether using Matplotlib directly or through Pandas, these tools offer a comprehensive toolkit for creating insightful and visually appealing plots, crucial for effective data analysis and communication.

5.7 GLOSSARY

- **Matplotlib:** It is a Python plotting library.
- **Subplots:** It arranges plots in figures.
- **Pandas:** It is a data manipulation library.
- **Scatter Plot:** It visualises two variables distribution.
- **Heatmap:** It is colour-coded matrix representation.

5.8 ANSWERS TO IN-TEXT QUESTIONS

1. A data visualization library
2. matplotlib.pyplot
3. Plot a 2D line or scatter plot

NOTES



NOTES

4. Using the subplots function
5. Colour
6. Set the x-axis label
7. Using `plt.savefig('filename.png')`
8. Density plot
9. Adjust the layout of subplots to prevent overlap
10. `df.plot(kind='line')`
11. `df.plot(kind='hist')`
12. `df.plot(x='X', y='Y', kind='scatter')`
13. Stacks bars on top of each other

5.9 SELF-ASSESSMENT QUESTIONS

1. Can you explain the purpose of Matplotlib in data visualization?
2. How do you create multiple subplots within a single figure using Matplotlib?
3. What parameters in Matplotlib allow you to customise the appearance of a plot, such as colour, line style, and marker?
4. How does Pandas simplify the process of creating plots compared to using Matplotlib directly?
5. Explain the use cases for creating histograms and stacked bar plots in Pandas.
6. When would you choose to use a heatmap, and how is it visualized in Pandas?
7. Can you describe the relationship between Matplotlib and Pandas when it comes to plotting data?
8. Create a Python script that reads a CSV file named “data.csv” containing information about monthly sales data. The file has columns ‘Month’ and ‘Sales’. Your task is to use Matplotlib and Pandas to visualise this data by plotting a line chart showing the sales trend over the months. Additionally, add labels to the axes and a title to the plot, and customise the line style and colour for better clarity.



5.10 REFERENCES

- McKinney, W. (2018). Python for Data Analysis: Data Wrangling with Pandas, NumPy and IPython (2nd ed.). O'Reilly Media.
- Molin, S. (2019). Hands-On Data Analysis with Pandas. Packt Publishing.
- Thareja, R. (2017). Python Programming using problem-solving approach.

5.11 SUGGESTED READINGS

- Data Visualization in Python, Daniel Nelson, StackAbuse.com, 2020
- Mastering Python Data Visualization, Kirthi Raman, O'Reilly, 2015
- Statistics and Data Visualization with Python, Jesus Rogel-Salazar, Chapman and Hall/CRC, 2023

NOTES

UNIT V:

LESSON 6: Data Unification: Exploring Aggregation and Grouping



LESSON 6

DATA UNIFICATION: EXPLORING AGGREGATION AND GROUPING

NOTES

Lavkush Gupta
Assistant Professor
Shyama Prasad Mukherji College (W)
University of Delhi
lavkush.mca16.du@gmail.com

Structure

- 6.1 Learning Objectives
- 6.2 Introduction
- 6.3 Data Aggregation
- 6.4 GroupBy MECHANICS
- 6.5 Pivot Tables
- 6.6 Cross-Tabulation
- 6.7 Summary
- 6.8 Glossary
- 6.9 Answers to In-text Questions
- 6.10 Self-Assessment Questions
- 6.11 References
- 6.12 Suggested Readings

6.1 LEARNING OBJECTIVES

Upon completing the lesson, learners should be able to:

- Define data aggregation and explain its importance in data analysis.
- Recognise scenarios where data aggregation is applicable.
- Utilise the group by function in Pandas to group data efficiently.
- Understand the purpose and use of each aggregation function.
- Create pivot tables using Pandas for complex data summarisation.
- Understand and use cross-tabulations to analyse relationships between variables.

*Self-Instructional
Material*

155



NOTES

6.2 INTRODUCTION

This lesson discusses the fundamental techniques of data manipulation. In the realm of data analysis, the ability to aggregate and perform group operations is indispensable, and this lesson focuses on harnessing the capabilities of Pandas, a versatile Python library. As we delve into the intricacies of the group by function and pivot tables, learners will acquire the skills to distil complex datasets into meaningful summaries. From handling missing data to optimising code efficiency, this lesson equips you with the practical knowledge needed to navigate real-world scenarios, transforming raw data into actionable insights.

6.3 DATA AGGREGATION

We are all living in the digital era and have evolved with the huge amount of data. Without data, we cannot presume our daily needs. Data is like fuel for us. The data aggregation process helps in statistical analysis for the collection of objects and provides useful and summarised information. It helps in industrial analysis, organisational data and many more. Often, the aggregation of data is applied over a large amount of data. It helps in splitting and grouping data in various ways. The group of programs used in the aggregation of data are known as data aggregators.

There are groups of operations used in data aggregation. This process is also called groupby mechanics. The groupby mechanics operations are similar to divide and conquer algorithms; they are performed in the sequences of the split, apply operation and conquer (combined), respectively. The first step, split, means to break or divide the data into sublists or sub-sequences according to the applied data structure, and then we get the small parts of lists that make things easier. In the second step of applying the operation, we can apply the required operation to each subpart of divided data (split group of data). In the last step of combining, the resultant data is combined by making groups.



Data Unification: Exploring Aggregation and Grouping

There are various aggregation functions used with the `groupby()` function in performing the statistical operations and analysis. A few aggregation functions, which are applicable to `groupby` methods, are given below :

- mean
- median
- max
- min
- std
- var
- count
- sum

NOTES

6.4 GROUPBY MECHANICS

The purpose of the `GroupBy` function is to convert the random data sequence into an arranged one. It also works like a filter operation in which we can fetch the required data of a particular group only.

```
import pandas as pd
# Sample DataFrame
data = {'Category': ['A', 'B', 'A', 'B', 'A'],
        'Value1': [10, 20, 30, 40, 50],
        'Value2': [5, 10, 15, 20, 25]}
df = pd.DataFrame(data)
# Grouping by 'Category'
grouped_df = df.groupby('Category')
# Calculating the sum for each group
sum_by_category = grouped_df.sum()
# Displaying the results
print("Output")
print("Original DataFrame:")
print(df)
print("\nGrouped by 'Category' - Sum:")
print(sum_by_category)
```

Self-Instructional
Material

157



NOTES

```
Output
Original DataFrame:
  Category  Value1  Value2
0        A      10       5
1        B      20      10
2        A      30      15
3        B      40      20
4        A      50      25

Grouped by 'Category' - Sum:
      Value1  Value2
Category
A           90      45
B           60      30
```

To retrieve the grouped DataFrames, we need to apply the iteration method or looping statements. It will be discussed in the next topic of the lesson.

NOTE: When we apply the `groupby()` function over the series of (or array), the number of elements in each series should be equal otherwise it throws an error.

Filter operation of the `groupby` can be applied using the `get_groupby()` function.

```
import pandas as pd
# Sample DataFrame
data = {'Category': ['A', 'B', 'A', 'B', 'A'],
        'Value1': [10, 20, 30, 40, 50],
        'Value2': [5, 10, 15, 20, 25]}
df = pd.DataFrame(data)
# Grouping by 'Category'
grouped_df = df.groupby('Category')
# Filtering based on group-specific conditions
filtered_group_A = grouped_df.get_group('A') # Get group 'A'
filtered_group_B = grouped_df.get_group('B') # Get group 'B'
# Displaying the results
print("Output")
print("Original DataFrame:")
print(df)
print("\nFiltered Group A:")
print(filtered_group_A)
print("\nFiltered Group B:")
print(filtered_group_B)
```



Data Unification: Exploring Aggregation and Grouping

NOTES

```
Output
Original DataFrame:
  Category  Value1  Value2
0        A      10       5
1        B      20      10
2        A      30      15
3        B      40      20
4        A      50      25

Filtered Group A:
  Category  Value1  Value2
0        A      10       5
2        A      30      15
4        A      50      25

Filtered Group B:
  Category  Value1  Value2
1        B      20      10
3        B      40      20
```

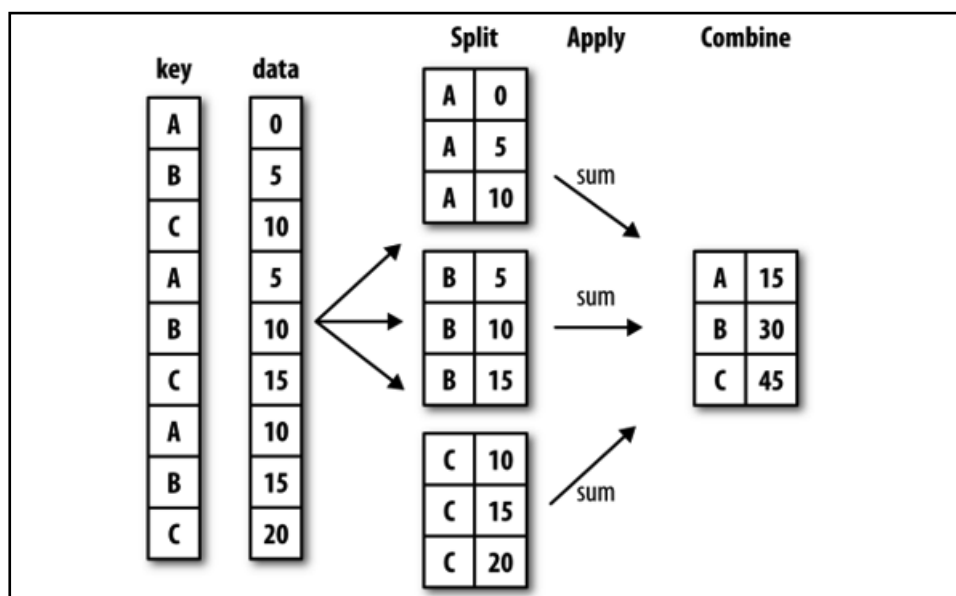


Figure 6.1: Illustration of a group aggregation

Mean of Grouped Data: As we discussed in earlier lessons, the mean as a measure of dispersion helps to find the central value among a number of data. We can find the mean of grouped data using data aggregation methods. We use the `mean()` function to compute the mean of grouped data. When we apply the `mean()` function over the series of data, then all grouped keys are aggregated and mapped by their values.



NOTES

Steps to find the mean of grouped data:

1. First, import the required library for data aggregation.
2. Create the DataFrames and store them in a variable.
3. Apply the mean() with the variable to compute the mean.

```
import pandas as pd
# Sample DataFrame
data = {'Category': ['A', 'B', 'A', 'B', 'A'],
        'Value1': [10, 20, 30, 40, 50],
        'Value2': [5, 10, 15, 20, 25]}
df = pd.DataFrame(data)
# Grouping by 'Category'
grouped_df = df.groupby('Category')
# Calculating the mean for each group
mean_by_category = grouped_df.mean()
# Displaying the results
print("Output")
print("Original DataFrame:")
print(df)
print("\nGrouped by 'Category' - Mean:")
print(mean_by_category)
```

Output

Original DataFrame:

	Category	Value1	Value2
0	A	10	5
1	B	20	10
2	A	30	15
3	B	40	20
4	A	50	25

Grouped by 'Category' - Mean:

	Value1	Value2
Category		
A	30.0	15.0
B	30.0	15.0

Iteration over the Grouped data: The iteration process helps while retrieving the grouped elements (aggregated series data at stored memory location) from contiguous memory blocks. Iterating over grouped data in Pandas involves looping through the groups created by the groupby function. Each group is a subset of the original DataFrame based on the unique values in the grouping column. Here is an example of how to iterate over grouped data:



Data Unification: Exploring Aggregation and Grouping

NOTES

```
import pandas as pd

# Sample DataFrame
data = {'Category': ['A', 'B', 'A', 'B', 'A'],
        'Value1': [10, 20, 30, 40, 50],
        'Value2': [5, 10, 15, 20, 25]}
df = pd.DataFrame(data)

# Grouping by 'Category'
grouped_df = df.groupby('Category')

# Iterate over groups and display information for each group
for group_name, group_data in grouped_df:
    print(f"Group: {group_name}")
    print(group_data)
    print("\n")
```

```
Group: A
  Category  Value1  Value2
0        A      10       5
2        A      30      15
4        A      50      25
```

```
Group: B
  Category  Value1  Value2
1        B      20      10
3        B      40      20
```

In this example, the `groupby` operation creates two groups based on the unique values in the 'Category' column ('A' and 'B'). The loop iterates over these groups, and for each group, it prints the group name (`group_name`) and the corresponding DataFrame (`group_data`). You can then perform specific operations or analyses within the loop for each group.

Additionally, you can use aggregate functions within the loop to calculate group-specific statistics or apply custom functions to each group. The flexibility of iteration over grouped data allows for dynamic and customised analyses based on the unique characteristics of each group.



NOTES

Applying more than one aggregate function: You can apply multiple aggregate functions simultaneously using the `agg` method. The `agg` method allows you to specify different aggregation functions for each column.

```
import pandas as pd

# Sample DataFrame
data = {'Category': ['A', 'B', 'A', 'B', 'A'],
        'Value1': [10, 20, 30, 40, 50],
        'Value2': [5, 10, 15, 20, 25]}
df = pd.DataFrame(data)

# Grouping by 'Category' and applying multiple aggregate functions
aggregated_data = df.groupby('Category').agg({
    'Value1': ['sum', 'mean', 'max'],
    'Value2': ['min', 'median', 'std']
})

# Displaying the aggregated results
print(aggregated_data)
```

	Value1			Value2		
	sum	mean	max	min	median	std
Category						
A	90	30.0	50	5	15.0	10.000000
B	60	30.0	40	10	15.0	7.071068

IN-TEXT QUESTIONS

1. In Python's Pandas library, what function is commonly used for grouping data?
2. Which method is used to apply aggregation functions to grouped data?
3. Which aggregation function in pandas calculates the median of a numeric column within each group?
4. In the context of pandas and data grouping, what does the term “multi-level indexing” refer to?



6.5 PIVOT TABLES

DataFrame: It is a main object in pandas, used to represent data with rows and columns like a tabular form of Excel data. Let's consider a dataframe of weather data that we will use in our next set of examples:

```
import pandas as pd

# Sample DataFrame
data = {'Date': ['5/1/2017', '5/2/2017', '5/3/2017', '5/1/2017', '5/2/2017', '5/3/2017', '5/1/2017', '5/2/2017', '5/3/2017'],
        'City': ['new york', 'new york', 'new york', 'mumbai', 'mumbai', 'mumbai', 'beijing', 'beijing', 'beijing'],
        'Temperature': [65, 66, 68, 75, 78, 82, 80, 77, 79],
        'Humidity': [56, 58, 60, 80, 83, 85, 26, 30, 35]}

df = pd.DataFrame(data)
df
```

	Date	City	Temperature	Humidity
0	5/1/2017	new york	65	56
1	5/2/2017	new york	66	58
2	5/3/2017	new york	68	60
3	5/1/2017	mumbai	75	80
4	5/2/2017	mumbai	78	83
5	5/3/2017	mumbai	82	85
6	5/1/2017	beijing	80	26
7	5/2/2017	beijing	77	30
8	5/3/2017	beijing	79	35

Figure 6.2: DataFrame df of Weather

Pivot() Function: This function allows us to change the shape of DataFrame from rows to columns or vice versa. It has the argument 'index', which refers to the row data, and argument 'columns', which refers to the columns data.



NOTES

```
df.pivot(index="City",columns="Date")
```

Date	Temperature			Humidity		
	5/1/2017	5/2/2017	5/3/2017	5/1/2017	5/2/2017	5/3/2017
City						
beijing	80	77	79	26	30	35
mumbai	75	78	82	80	83	85
new york	65	66	68	56	58	60

NOTE: If you want to display specific column values only, then specify those in the ‘values’ argument.

```
df.pivot(index="City",columns="Date",values="Temperature")
```

Date	5/1/2017	5/2/2017	5/3/2017
City			
beijing	80	77	79
mumbai	75	78	82
new york	65	66	68

Here, only temperature values are displayed, while humidity values are not displayed.

Pivot Table: The pivot table is used to summarise and aggregate data inside the DataFrame. It aggregates the data similar to groupby method of series data using one or more keys.



Data Unification: Exploring Aggregation and Grouping

NOTES

```
df.pivot_table(index=["City","Date"])
```

City	Date	Humidity Temperature	
		Humidity	Temperature
beijing	5/1/2017	26	80
	5/2/2017	30	77
	5/3/2017	35	79
mumbai	5/1/2017	80	75
	5/2/2017	83	78
	5/3/2017	85	82
new york	5/1/2017	56	65
	5/2/2017	58	66
	5/3/2017	60	68

Now, if we have to show only one attribute temperature, then use the values argument with value as temperature. It hides the humidity attribute.

```
df.pivot_table(index=["City","Date"],values="Temperature")
```

City	Date	Temperature
beijing	5/1/2017	80
	5/2/2017	77
	5/3/2017	79
mumbai	5/1/2017	75
	5/2/2017	78
	5/3/2017	82
new york	5/1/2017	65
	5/2/2017	66
	5/3/2017	68



NOTES

Aggregate functions with Pivot Table: As we discussed earlier, aggregate functions can be applied with the groupby methods. Similarly, the aggregate functions(count, sum, mean, median, std, var) are also applicable to the pivot table. In this example, we used the aggregation function ‘aggfunc’ with their value count, which gives the number of available values for a specific column in a specific city.

```
df.pivot_table(index="City",aggfunc="count")
```

	Date	Humidity	Temperature
City			
beijing	3	3	3
mumbai	3	3	3
new york	3	3	3

Note: You can replace aggfunc with any other aggregate function like sum, mean, or avg, as discussed in earlier sections.

Margins Argument of Pivot Table: We could augment this table to include partial totals by passing margins=True. This has the effect of adding All row and column labels, with corresponding values being the group statistics for all the data within a single tier.

```
df.pivot_table(index="Date",columns="City",margins=True)
```

	Humidity				Temperature			
City	beijing	mumbai	new york	All	beijing	mumbai	new york	All
Date								
5/1/2017	26.000000	80.000000	56	54.0	80.000000	75.000000	65.000000	73.333333
5/2/2017	30.000000	83.000000	58	57.0	77.000000	78.000000	66.000000	73.666667
5/3/2017	35.000000	85.000000	60	60.0	79.000000	82.000000	68.000000	76.333333
All	30.333333	82.666667	58	57.0	78.666667	78.333333	66.333333	74.444444



Data Unification: Exploring Aggregation and Grouping

NOTE: If any value is missing or not given in any column of the pivot table, then the cell would be NaN (Not a Number) by default, but if you want to apply the aggregation over the table, then you can use `fill_value = 0` (the missing value will be filled with digit 0) as an argument into `pivot_table()` function.

NOTES

6.6 CROSS-TABULATIONS

This is another form of the pivot table. It gives short form of big table. The use of cross-tables is often for the computation of group frequencies. Sometimes, in short, cross-tabulation is also called a cross table or contingency table. In the example below, a small dataset on student performance of different streams is given:

```
import pandas as pd
# Sample DataFrame
data = {'Name': ['Ram', 'Shyam', 'Mahesh', 'Suresh', 'Dinesh', 'Krishna', 'Rajeev'],
        'Class': [11, 11, 12, 12, 10, 11, 11],
        'Stream': ['Science', 'Commerce', 'Art', 'Commerce', 'Science', 'Commerce', 'Commerce'],
        'Performance': ['Best', 'Best', 'Best', 'Average', 'Good', 'Best', 'Average'],
        'Age': [15, 14, 16, 17, 13, 14, 15]}
df = pd.DataFrame(data)
df
```

	Name	Class	Stream	Performance	Age
0	Ram	11	Science	Best	15
1	Shyam	11	Commerce	Best	14
2	Mahesh	12	Art	Best	16
3	Suresh	12	Commerce	Average	17
4	Dinesh	10	Science	Good	13
5	Krishna	11	Commerce	Best	14
6	Rajeev	11	Commerce	Average	15

Apply the cross-table function: The **crosstab** function is used to create a cross-tabulation from the DataFrame.



NOTES

```
pd.crosstab(df.Stream,df.Performance)
```

Performance	Average	Best	Good
Stream			
Art	0	1	0
Commerce	2	2	0
Science	0	1	1

Apply the margins with cross-tabulations: The margins provide the total aggregate sum of each row and column values separately.

```
pd.crosstab(df.Stream, df.Performance, margins = True)
```

Performance	Average	Best	Good	All
Stream				
Art	0	1	0	1
Commerce	2	2	0	4
Science	0	1	1	2
All	2	4	1	7

Aggregate two indices stream and class in a list with a column of performance.



Data Unification: Exploring Aggregation and Grouping

```
pd.crosstab([df.Stream,df.Class], df.Performance, margins = True)
```

Stream	Class	Performance			
		Average	Best	Good	All
Art	12	0	1	0	1
	11	1	2	0	3
Commerce	12	1	0	0	1
	10	0	0	1	1
	11	0	1	0	1
All		2	4	1	7

NOTES

IN-TEXT QUESTIONS

5. In pandas, which method is used to create a pivot table?
6. What is the primary purpose of a pivot table?
7. In pandas, what does the margins parameter in the pivot_table() function do?
8. In pandas, what is the primary function of the crosstab() function?

6.7 SUMMARY

Data aggregation and group operations involve organising data based on specific criteria, applying operations within these groups, and combining the results. The “Group by” mechanic in databases helps in categorising data for aggregation, allowing the use of functions like sum or average within each group. A broader approach, known as split-apply-combine, involves breaking down data, applying operations independently to each segment, and then merging the outcomes. This flexible method accommodates various analyses and transformations. Pivot tables, commonly found in spreadsheet tools and programming libraries like pandas, simplify data analysis by reshaping and summarising information.



NOTES

Additionally, cross-tabulation, achieved through functions like `crosstab`, helps in summarising and studying the relationship between two categorical variables. In essence, data aggregation and group operations streamline data analysis by grouping, applying operations, and combining results, utilising tools like `group by`, `split-apply-combine`, pivot tables, and cross-tabulation for efficient insights.

6.8 GLOSSARY

- **Group by:** It categorise data for aggregation.
- **Aggregation:** It means combining and summarising data.
- **Pivot Tables:** It means reshaping and summarising data.
- **Cross-tabulation:** It means analysing categorical variable relationships.
- **Split-apply-combine:** It break, operate, and merge data.

6.9 ANSWERS TO IN-TEXT QUESTIONS

1. `groupby()`
2. `agg()`
3. `median()`
4. Grouping data at multiple levels of hierarchy
5. `pivot_table()`
6. Reshaping and summarising data based on specified criteria
7. Adds an extra column and row for subtotals
8. Generating contingency tables

6.10 SELF-ASSESSMENT QUESTIONS

1. How do you use the `groupby` function in the pandas library to group a `DataFrame` by a specific column?
2. Write a Python code snippet to calculate the sum of a numeric column



Data Unification: Exploring Aggregation and Grouping

named 'sales' for each group in a pandas DataFrame after using the groupby function.

3. How to create a pivot table from a DataFrame and show the average values for two columns, 'A' and 'B', based on the grouping of column 'X'.
4. How would you use the crosstab function from the pandas library to create a contingency table for two categorical variables 'Category' and 'Region' from a DataFrame?

NOTES

6.11 REFERENCES

- McKinney, W. (2018). Python for Data Analysis: Data Wrangling with Pandas, NumPy and IPython (2nd ed.). O'Reilly Media.
- Molin, S. (2019). Hands-On Data Analysis with Pandas. Packt Publishing.
- Thareja, R. (2017). Python Programming using problem-solving approach.

6.12 SUGGESTED READINGS

- Data Visualization in Python, Daniel Nelson, StackAbuse.com, 2020
- Mastering Python Data Visualization, Kirthi Raman, O'Reilly, 2015
- Statistics and Data Visualization with Python, Jesus Rogel-Salazar, Chapman and Hall/CRC, 2023

NOTES

[illegible]



**DEPARTMENT OF DISTANCE AND CONTINUING EDUCATION
UNIVERSITY OF DELHI**